

pytop Kullanım Kılavuzu

Nokta-Küme Topolojisi ve Metrik Uzaylar

Kadirhan Polat

2026

İçindekiler

Ön Söz	vi
I Ön Koşullar	1
1 pytop'a Hızlı Giriş	2
1.1 Kurulum ve Import	2
1.2 Uzay Nesneleri	2
1.3 Result Tipi	3
1.4 Temel Uzaylar Turu	4
1.5 Özel Topoloji: make_topology	4
1.6 Tag Sistemi	4
1.7 n Nokta Üzerindeki Topoloji Sayısı	5
1.8 pytop Ne Hesaplar? — Genel Harita	5
1.9 Hesaplamalı Çekirdek: İlk Bakış	5
1.10 Alıştırmalar	7
2 Önerme Mantiğı	8
2.1 Önerme ve Doğruluk Değeri	8
2.2 Mantıksal Bağlaçlar	8
2.3 Algoritmalar	10
2.3.1 Doğruluk Tablosu Üretimi	10
2.3.2 Tautoloji Denetimi	10
2.4 pytop API	10
2.5 Örnekler	10
2.5.1 Doğruluk Tablosu	10
2.5.2 Tautolojiler — De Morgan ve Karşıt	11
2.5.3 Niceleyiciler	12
2.5.4 Topoloji Uygulaması — T0 ve T1	13
2.6 Alıştırmalar	13
3 Küme Teorisi ve Fonksiyon Temelleri	15
3.1 Konu	15
3.2 Teoremler	16
3.3 Algoritmalar	17
3.3.1 Güç Kümesi — $O(2^{ A })$	17
3.3.2 Denklik Bağıntısı Doğrulama — $O(R \cdot A)$	17
3.4 pytop API	17
3.5 Örnekler	18

3.6	Alıştırmalar	20
II	Nokta-Küme Topolojisi	21
4	Topolojik Uzaylar	22
4.1	Konu	22
4.1.1	Sezgisel Giriş	22
4.1.2	Formal Tanım	22
4.1.3	Temel Örnekler	23
4.1.4	Baz ve Alt-Baz	23
4.1.5	Sonlu ve Sonsuz Uzaylar	24
4.2	Teoremler	24
4.3	Algoritmalar	25
4.3.1	Bazdan Topoloji Üretimi	25
4.3.2	İz Sürme: Küçük Bir Girdiyle Adım Adım	26
4.3.3	Alt-Bazdan Topoloji Üretimi	26
4.4	pytop API	26
4.4.1	Sınıflar	26
4.4.2	Oluşturucular	26
4.4.3	Tag Sistemi	27
4.4.4	Tag Gereksinimleri	27
4.5	Örnekler	27
4.5.1	Örnek 1: Sierpiński Uzayı	27
4.5.2	Örnek 2: Ayrık Topoloji	28
4.5.3	Örnek 3: <code>make_topology</code> ile Manuel İnşa	29
4.5.4	Örnek 4: Alexandrov Zincir Uzayı	30
4.5.5	Örnek 5: Bazdan Topoloji Üretimi	30
4.5.6	Örnek 6: Gerçek Doğru	31
4.5.7	Örnek 7: Aksiyom Denetimi ve Kapalı Kümeler	31
4.5.8	Örnek 8: Dışlanan-Nokta Topolojisi ve Komşuluk Karakteri	32
4.6	Alıştırmalar	33
5	Yüklemler ve Altküme Operatörleri	35
5.1	Konu	35
5.1.1	Altküme Yüklemeleri	35
5.1.2	Altküme Operatörleri	35
5.1.3	Komşuluk Sistemi	36
5.2	Teoremler	36
5.3	Algoritmalar	37
5.4	pytop API	38
5.5	Örnekler	38
5.5.1	Örnek 1: Kapanış ve İç — Sierpiński Uzayı	38
5.5.2	Örnek 2: Türev Kümesi	39
5.5.3	Örnek 3: Komşuluk Sistemi	39
5.5.4	Örnek 4: İç / Kapanış / Sınır / Dış Parçalanışı	40
5.5.5	Örnek 5: Kuratowski Aksiyomlarının Doğrulanması	40
5.5.6	Örnek 6: Yoğun Nokta ve Birikme Kümesi	41
5.6	Alıştırmalar	41

6	Ayrılma Aksiyomları	43
6.1	Konu	43
6.2	Teoremler	44
6.3	Algoritmalar	46
6.3.1	İz Sürme: T0 Prosedürü Sierpiński Üzerinde	46
6.4	pytop API	47
6.5	Örnekler	47
6.5.1	Örnek 1: Sierpiński — T0 ✓, T1 ×	47
6.5.2	Örnek 2: Kosonlu $\mathbb{N}$ — T1 ✓, T2 ×	48
6.5.3	Örnek 3: Ayrılma Zinciri — Ayrık	48
6.5.4	Örnek 4: Regüler Ama T3 Değil — Konvansiyon Testi	48
6.5.5	Örnek 5: Dışlanan-Nokta Topolojisi — İkinci Bir “T0’da Takılan”	49
6.5.6	Örnek 6: Metrik Uzaylar — Zincirin Tepesine Kadar	50
6.6	Alıştırmalar	50
7	Kompaktlık	52
7.1	Konu	52
7.1.1	Sezgisel Giriş — “Gizli Sonluluk”	52
7.1.2	Kompaktlık Tanımı	52
7.1.3	Neden “kapalı ve sınırlı” yetmez? — Karşı-örnekler	53
7.1.4	Kompaktlık Varyantları	53
7.2	Teoremler	54
7.3	Algoritmalar	54
7.3.1	Sonlu Uzayda Kompaktlık	54
7.3.2	analyze_compactness_variants — Tag Tabanlı Çıkarım	55
7.4	pytop API	55
7.5	Örnekler	55
7.6	Alıştırmalar	57
8	Bağlantılılık	58
8.1	Konu	58
8.1.1	Sezgisel Giriş — “Tek Parça mı, Kopuk mu?”	58
8.1.2	Bağlantılılık Kavramları	59
8.1.3	Clopen Ayrılma	59
8.1.4	Bağlantılı Bileşenler	59
8.2	Teoremler	60
8.3	Algoritmalar	60
8.3.1	Sonlu Bağlantılılık — $O(\tau ^2)$	60
8.4	pytop API	61
8.5	Örnekler	61
8.6	Alıştırmalar	63
9	Sayılabilirlik Aksiyomları	64
9.1	Konu	64
9.1.1	Sayılabilirlik Aksiyomları	64
9.2	Teoremler	65
9.3	Algoritmalar	66
9.3.1	Sonlu Uzayda Sayılabilirlik	66
9.3.2	Sonsuz Uzayda: Tag-Tabanlı Çıkarım	66

9.4	pytop API	67
9.5	Örnekler	67
9.6	Alıştırmalar	68
10	Süreklili Fonksiyonlar ve Homeomorfizmalar	70
10.1	Konu	70
10.2	Teoremler	71
10.3	Algoritmalar	72
10.3.1	is_continuous_finite_map — $O(\tau_Y \cdot X)$	72
10.4	pytop API	72
10.5	Örnekler	73
10.6	Alıştırmalar	75
11	Alt Uzay ve Çarpım Topolojisi	76
11.1	Konu	76
11.2	Teoremler	77
11.3	Algoritmalar	78
11.3.1	finite_subspace — $O(\tau \cdot A)$	78
11.3.2	binary_product — $O(\tau_X \cdot \tau_Y)$	78
11.4	pytop API	79
11.5	Örnekler	79
11.6	Alıştırmalar	80
12	Bölüm Topolojisi	82
12.1	Konu	82
12.2	Teoremler	83
12.3	Algoritmalar	84
12.3.1	quotient_set — $O(X ^2 \cdot \alpha(X))$	84
12.3.2	finite_quotient_contract — $O(X + P)$	84
12.4	pytop API	84
12.5	Örnekler	85
12.6	Alıştırmalar	88
13	Başlangıç ve Son Topoloji	89
13.1	Başlangıç Topolojisi	89
13.2	Algoritmalar	90
13.2.1	Başlangıç Topolojisi İnşası	90
13.2.2	Son Topoloji İnşası	90
13.3	pytop API	91
13.4	Örnekler	92
13.4.1	Tek Harita ile Başlangıç Topolojisi	92
13.4.2	İki Harita — Topoloji İncelir	92
13.4.3	Altuzay = Başlangıç	92
13.4.4	Çarpım = Başlangıç	93
13.4.5	Son Topoloji — Manuel Hesap	93
13.4.6	Bölüm = Son Topoloji	94
13.4.7	Yeni Örnek — “En Kaba” Olmanın Doğrudan Doğrulanması	94
13.4.8	Yeni Örnek — Evrensel Özelliğin Doğrulanması	95
13.5	Alıştırmalar	96

III Metrik Uzaylar	97
14 Metrik Uzaylar	98
14.1 Konu	98
14.1.1 Sezgisel Giriş — “Mesafe Topolojiyi Doğurur”	98
14.1.2 Metrik Aksiyomları	99
14.1.3 Temel Kavramlar	100
14.1.4 Standart Metrikler	100
14.2 Teoremler	100
14.3 Algoritmalar	101
14.3.1 validate_metric — $O(X ^3)$	101
14.3.2 open_ball — $O(X)$	101
14.4 pytop API	101
14.5 Örnekler	102
14.6 Alıştırmalar	104
15 Metrik Tamlık ve Kompaktlık	105
15.1 Konu	105
15.1.1 Cauchy Dizisi ve Tamlık	105
15.1.2 Tamamen Sınırlılık	106
15.1.3 Metrik Kompaktlık Karakterizasyonu	106
15.2 Teoremler	106
15.3 Algoritmalar	107
15.3.1 Sonlu Metrikte is_complete ve is_totally_bounded	107
15.3.2 metric_compactness_check	107
15.4 pytop API	107
15.5 Örnekler	108
15.6 Alıştırmalar	109
16 Metrik Fonksiyonlar ve Sözleşmeler	111
16.1 Konu	111
16.1.1 Metrik Fonksiyon Sınıfları	111
16.1.2 Lipschitz Sabiti	112
16.2 Teoremler	112
16.3 Algoritmalar	113
16.3.1 classify_finite_metric_map — $O(X ^2)$	113
16.4 pytop API	113
16.5 Örnekler	114
16.6 Alıştırmalar	116
A Alıştırma Çözümleri	117
A.1 Bölüm 4 — Topolojik Uzaylar	117
A.2 Bölüm 6 — Ayrılma Aksiyomları	118
Dizin	121

Ön Söz

Bu kılavuz `pytop` paketinin nokta-küme topolojisi ve metrik uzaylar modüllerini kapsamlı biçimde tanıtmaktadır. Her bölüm aynı yapıyı izler:

1. Teorik giriş ve formal tanımlar
2. Temel teoremler (kanıt iskeletleriyle)
3. Algoritmalar (sözde kod ve karmaşıklık)
4. `pytop` API
5. Çalışan örnekler
6. Alıştırmalar

Örnekler aynı zamanda `docs/user_guide/python/` altındaki Python script'lerinde ve `docs/user_guide/notebook/` altındaki Jupyter defterlerinde çalıştırılabilir biçimde mevcuttur.

Kısım I

Ön Koşullar

Bölüm 1

pytop'a Hızlı Giriş

pytop kurulumu, temel uzay nesneleri, `Result` tipi ve tag sistemi — kılavuzun geri kalanını okumadan önce bu bölümü çalıştırın.

1.1 Kurulum ve Import

```
1 pip install -e .    # git kokundan
```

Her bölümde ihtiyaç duyulan semboller doğrudan `pytop`'tan import edilir. Bu bölüm bir *tur* olduğundan, ilk hücrede sonraki tüm hücrelerin kullanacağı semboller tek seferde içe aktarıyoruz:

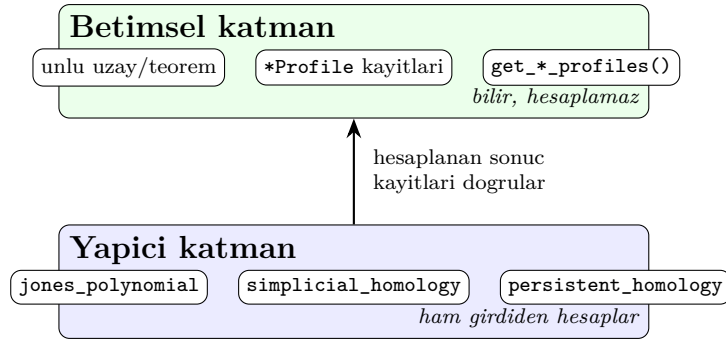
```
1 import pytop
2 from pytop import (
3     discrete_topology, indiscrete_topology, sierpinski_space,
4     make_topology,
5     is_compact, is_connected, is_t0, is_t1, is_t2,
6     count_topologies_on_n_points,
7     simplicial_complex, simplicial_homology, betti_numbers,
8     FiniteMetricSpace, persistent_homology, persistence_betti_numbers
9 )
10 from pytop.experimental.spaces import finite_circle, pi1_space
11 print("pytop surumu:", pytop.__version__)
```

pytop surumu: 1.7.0

Sezgi

`pytop`'u iki katlı bir bina gibi düşünün. Alt kat **betimsel** (descriptive) katmandır: ünlü uzayların ve teoremlerin kayıtlı, kaynaklı bilgisini tutar — bilir ama hesaplamaz. Üst kat **yapıcı** (constructive) katmandır: ham girdiden invariantları *hesaplar* — homoloji, kalıcı homoloji, düğüm polinomları. Bu bölüm her iki kata da kısa bir tur attırır.

1.2 Uzay Nesneleri



Şekil 1.1: pytop'un iki katmanı: betimsel kayıtlar ile yapıcı motorlar.

Kurucu	Topoloji	Sonuç
<code>discrete_topology(*pts)</code>	Her altküme açık	Ayrık
<code>indiscrete_topology(*pts)</code>	Yalnız $\emptyset$ ve X açık	İndiscrete
<code>sierpinski_space()</code>	$\{\emptyset, \{1\}, \{0, 1\}\}$	Sierpiński
<code>make_topology(carrier, *open_sets)</code>	Kullanıcı tanımlı	Özel

Tablo 1.1: Temel uzay kurucuları

```

1 d = discrete_topology(0, 1, 2)
2 print("carrier:", sorted(d.carrier))
3 print("topoloji boyutu:", len(d.topology))
4 print("etiketler:", sorted(d.tags))

```

```

carrier: [0, 1, 2]
topoloji boyutu: 8
etiketler: ['discrete', 'finite', 'hausdorff', 'metrizable', 'normal', 'regular']

```

1.3 Result Tipi

pytop'un çoğu yüklemi bir `Result` nesnesi döner.

```

1 s = sierpinski_space()
2 r = is_t2(s)
3 print(".status:", r.status)           # 'true' | 'false' | 'unknown'
4 print(".value:", r.value)
5 print(".mode:", r.mode)
6 print(".justification:", r.justification)

```

```

.status: false
.value: hausdorff
.mode: exact
.justification: ['The explicit finite topology fails hausdorff.']

```

`.status` her zaman `'true'`, `'false'` veya `'unknown'` dizesidir — Python `bool`'u değil.

1.4 Temel Uzaylar Turu

```

1 spaces = {
2     "Ayrik D(0,1,2)": discrete_topology(0, 1, 2),
3     "Indiscrete I(0,1,2)": indiscrete_topology(0, 1, 2),
4     "Sierpinski S": sierpinski_space(),
5 }
6 for name, sp in spaces.items():
7     print(name,
8           is_compact(sp).status,
9           is_connected(sp).status,
10          is_t2(sp).status)

```

Ayrik D(0,1,2)	true	false	true
Indiscrete I(0,1,2)	true	true	false
Sierpinski S	true	true	false

1.5 Özel Topoloji: make_topology

```

1 X = [1, 2, 3, 4]
2 tau = [set(), {1,2}, {3,4}, {1,2,3,4}]
3 fts = make_topology(X, *tau)
4 print("topoloji boyutu:", len(fts.topology))
5 print("kompakt:", is_compact(fts).status)
6 print("T2:", is_t2(fts).status)

```

```

topoloji boyutu: 4
kompakt: true
T2: false

```

1.6 Tag Sistemi

```

1 print("Ayrik:      ", sorted(discrete_topology(0,1,2).tags))
2 print("Indiscrete:", sorted(indiscrete_topology(0,1,2).tags))
3 print("Sierpinski:", sorted(sierpinski_space().tags))

```

```

Ayrik:      ['discrete', 'finite', 'hausdorff', 'metrizable', 'normal', 'regular']
Indiscrete: ['compact', 'connected', 'finite', 'indiscrete']
Sierpinski: ['compact', 'connected', 'finite', 't0']

```

Etiketler yüklem fonksiyonlarının karar mekanizmasını hızlandırır: `is_t2(d)` açık küme taraması yerine `'hausdorff'` in `d.tags` kontrolüyle sonuç verir.

1.7 n Nokta Üzerindeki Topoloji Sayısı

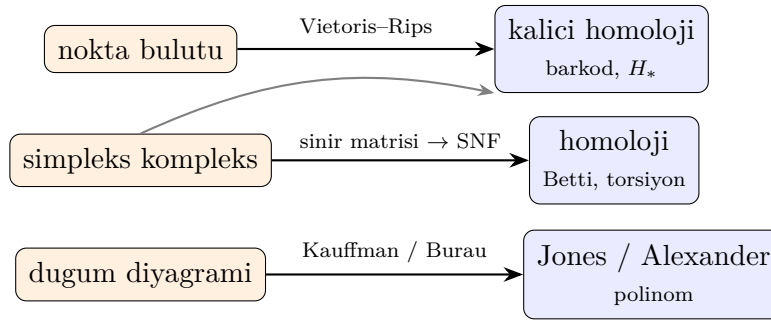
```
1 for n in range(1, 6):
2     print(f"n={n}: {count_topologies_on_n_points(n)}")
```

```
n =1: 1
n =2: 4
n =3: 29
n =4: 355
n =5: 6942
```

Topoloji sayısı hızla büyür; sonlu uzayların tüm topolojileri üzerinde kaba kuvvet taraması yalnızca küçük n için pratiktir.

1.8 pytop Ne Hesaplar? — Genel Harita

Önceki bölümler **betimsel/sonlu** katmanı gezdirdi. pytop'un asıl gücü **yapıcı çekirdek-tir**: ham bir kompleks ya da nokta bulutundan cebirsel invariantları gerçekten *hesaplar*.



Şekil 1.2: pytop hesaplama haritası: girdiden invarianta giden yollar.

1.9 Hesaplamalı Çekirdek: İlk Bakış

Örnek 9.1 — Sonlu Bir Çember Modeli ve $\pi_1 = \mathbb{Z}$

```
1 fc = finite_circle()
2 r = pi1_space(fc)
3 print("uzay adi   :", r.space_name)
4 print("grup tipi  :", r.group_type)
5 print("uretec say :", len(r.presentation.generators))
6 print("baginti say:", len(r.presentation.relators))
7 print("serbest mi :", r.is_free())
8 print("H1 betti   :", r.abelianization_betti)
```

```
uzay adi   : S^1
grup tipi  : infinite_cyclic
```

```

uretec say : 1
baginti say: 0
serbest mi : True
H1 betti   : 1

```

Yalnız 4 nokta, hiç bağıntısı olmayan tek üreteç: $\pi_1 = \langle a \rangle = \mathbb{Z}$. Sonlu bir uzay, sürekli bir çemberle aynı temel grubu taşır.

Örnek 9.2 — Üçgen Kenarından Çemberin Homolojisi

```

1 circle = simplicial_complex([[0], [1], [2], [0, 1], [1, 2], [0, 2]])
2 for k in (0, 1):
3     h = simplicial_homology(circle, k)
4     print(f"H{k}: betti={h.betti} torsion={h.torsion}")
5 print("beti_numbers:", betti_numbers(circle))

```

```

H0: betti =1 torsion =()
H1: betti =1 torsion =()
beti_numbers: (1, 1)

```

$H_0 = \mathbb{Z}$ (tek bileşen), $H_1 = \mathbb{Z}$ (bir delik) — çemberin klasik Betti imzası (1,1), ham simpleks listesinden SNF yoluyla hesaplandı.

Dikkat – sık hata

`simplicial_complex` yüz-kapalı girdi bekler: kenarları $[0,1]$ verip köşeleri $[0]$ eklemeyi unutursanız hata alırsınız. Üçgeni $[0,1,2]$ olarak eklerseniz içi dolar ve $H_1 = 0$ olur.

Örnek 9.3 — Nokta Bulutundan Kalıcı Homoloji

```

1 import math
2
3 pts = tuple(
4     (round(math.cos(2 * math.pi * k / 8), 4),
5      round(math.sin(2 * math.pi * k / 8), 4))
6     for k in range(8)
7 )
8 space = FiniteMetricSpace(carrier=pts, distance=math.dist)
9 pairs = persistent_homology(space, max_dimension=1, max_scale=1.0)
10
11 essential = [p for p in pairs if p.is_essential]
12 print("toplaml cift :", len(pairs))
13 print("kalici H0    :", sum(1 for p in essential if p.dimension == 0))
14 print("kalici H1    :", sum(1 for p in essential if p.dimension == 1))
15 print("beti        :", persistence_betti_numbers(pairs))

```

```

toplaml cift : 9
kalici H0    : 1
kalici H1    : 1
beti         : {0: 1, 1: 1}

```

Kalıcı sınıflar yine $(1,1)$ 'i verir: bir bileşen ve bir delik. Sadece sıralı noktalardan, hiçbir bağlantı bilgisi vermeden, pytop çemberin şeklini geri kurtarır.

1.10 Alıştırmalar

Kodlama

- K1. `simplicial_complex` ile içi **dolu** üçgeni ($[0,1,2]$ facet'i, tüm yüzleriyle) kurun ve `betti_numbers` çıktısını boş üçgeninkiyle karşılaştırın. H_1 neden kayboldu?
- K2. Örnek 9.3'teki nokta sayısını 8'den 4'e düşürün. Kalıcı H_1 hâlâ 1 mi? Sonucu çemberin örnekleme yoğunluğuyla ilişkilendirin.

Teori

- T1. Sonlu bir uzayın (Örnek 9.1) sürekli bir çemberle aynı π_1 'e sahip olması nasıl mümkün? McCord teoreminin ne söylediğini bir cümleyle açıklayın.

Bölüm 2

Önerme Mantığı

Matematiksel önerme (proposition), doğru ya da yanlış olabilen bir ifadedir. Bu bölüm `pytop.logic` modülünün sağladığı `Proposition`, bağlaçlar ve sonlu niceleyicileri tanıtır; ardından ayrılma aksiyomlarını bu araçlarla nasıl ifade edebileceğimizi gösterir.

2.1 Önerme ve Doğruluk Değeri

Tanım 2.1 (Önerme). Bir **önerme** (proposition), kesin bir doğruluk değerine sahip (doğru veya yanlış) matematiksel bir ifadedir.

Listing 2.1: Temel önermeler

```
1 from pytop import (  
2     Proposition,  
3     negate, conjunction, disjunction, implies, iff,  
4     for_all, there_exists, unique_exists,  
5 )  
6  
7 p = Proposition("p", True)  
8 q = Proposition("q", False)  
9  
10 print(f"p : {p.truth_value}")  
11 print(f"q : {q.truth_value}")
```

```
p : True  
q : False
```

2.2 Mantıksal Bağlaçlar

```
1 print("neg(p)  :", negate(p).truth_value)  
2 print("p and q :", conjunction(p, q).truth_value)  
3 print("p or q  :", disjunction(p, q).truth_value)  
4 print("p -> q  :", implies(p, q).truth_value)  
5 print("p <-> q :", iff(p, q).truth_value)  
6 print("p <-> p :", iff(p, p).truth_value)
```

Fonksiyon	Sembol	Anlam
negate(p)	$\neg p$	p değil
conjunction(p,q)	$p \wedge q$	p ve q
disjunction(p,q)	$p \vee q$	p veya q
implies(p,q)	$p \rightarrow q$	p ise q
iff(p,q)	$p \leftrightarrow q$	p ancak ve ancak q

Tablo 2.1: pytop.logic bağlaç fonksiyonları

```

neg(p) : False
p and q : False
p or q : True
p -> q : False
p <-> q : False
p <-> p : True

```

`implies(p, q)` yalnızca $p = \text{True}$, $q = \text{False}$ durumunda yanlış; diğer üç kombinasyonda boş doğruluk (vacuous truth) nedeniyle doğrudur.

Aşağıdaki görsel beş bağlacın dört satırlık doğruluk değerlerini bir arada özetler: yeşil hücre *doğru* (D), kırmızı hücre *yanlış* (Y).

p	q	$\neg p$	$p \wedge q$	$p \vee q$	$p \rightarrow q$	$p \leftrightarrow q$
D	D	Y	D	D	D	D
D	Y	Y	Y	D	Y	Y
Y	D	D	Y	D	D	Y
Y	Y	D	Y	Y	D	D

D = doğru
Y = yanlış

$p \rightarrow q$ yalnızca
 $p=D, q=Y$ 'de
yanlıştır

Şekil 2.1: Beş temel bağlacın doğruluk tablosu: $\neg p$, $p \wedge q$, $p \vee q$, $p \rightarrow q$, $p \leftrightarrow q$. $p \rightarrow q$ yalnızca tek satırda ($p=D$, $q=Y$) yanlıştır.

Sezgi

Bağlaçları “kapı” olarak düşünün. $\wedge$ (ve) her iki girdi de doğruyken akım geçirir; $\vee$ (veya) en az biri doğru olunca geçirir; $\rightarrow$ (ise) yalnızca “doğru bir öncülden yanlış bir sonuç” çıkarılırsa kapanır. İşte bu yüzden $p \rightarrow q$ sütununda yalnızca tek bir satır ($p=D$, $q=Y$) yanlıştır — diğer her şey, mantığın “verdiğin sözü bozmadın” kuralıdır.

Karşı-örnek

$p \rightarrow q$ ile $p \leftrightarrow q$ aynı şey değildir. $p=Y$, $q=D$ alındığında $p \rightarrow q$ **doğru** (D), ama $p \leftrightarrow q$ **yanlış** (Y) döner: içerme tek yönlüdür, çift-koşul iki yönü birden ister.

2.3 Algoritmalar

2.3.1 Doğruluk Tablosu Üretimi

Algorithm 1 Doğruluk Tablosu (İki Değişkenli)

Require: Bağlaç fonksiyonu f

Ensure: Dört satırlı doğruluk tablosu

```

1: for  $(t_p, t_q) \in \{(T, T), (T, F), (F, T), (F, F)\}$  do
2:    $p \leftarrow \text{Proposition}("p", t_p)$ 
3:    $q \leftarrow \text{Proposition}("q", t_q)$ 
4:   Yazdır  $(t_p, t_q, f(p, q).truth\_value)$ 
5: end for

```

2.3.2 Tautoloji Denetimi

Bir $\phi(p_1, \dots, p_n)$ formülü tautoloji iff tüm 2^n atama için doğrudur — $O(2^n \cdot |\phi|)$.

2.4 pytop API

Listing 2.2: logic modülü importu

```

1 from pytop import (
2     Proposition,
3     negate, conjunction, disjunction, implies, iff,
4     for_all, there_exists, unique_exists,
5 )

```

Niceleyiciler:

```

1 for_all(carrier, predicate)      # tüm x: P(x)
2 there_exists(carrier, predicate) # bazi x: P(x)
3 unique_exists(carrier, predicate) # tam bir x: P(x)

```

2.5 Örnekler

2.5.1 Doğruluk Tablosu

```

1 pairs = [(True, True), (True, False), (False, True), (False, False)]
2 print(f"{'p':>5} {'q':>5} {'neg_p':>5} {'p&q':>5} {'p|q':>5} {'p
   ->q':>6} {'p<->q':>6}")
3 for tv_p, tv_q in pairs:
4     pp = Proposition("p", tv_p)
5     qq = Proposition("q", tv_q)
6     row = (pp.truth_value, qq.truth_value,
7            negate(pp).truth_value, conjunction(pp, qq).truth_value,
8            disjunction(pp, qq).truth_value,
9            implies(pp, qq).truth_value, iff(pp, qq).truth_value)

```

10

```
print(" ".join(f"{str(v):>5}" for v in row))
```

p	q	neg_p	p&q	p q	p->q	p<->q
True	True	False	True	True	True	True
True	False	False	False	True	False	False
False	True	True	False	True	True	False
False	False	True	False	False	True	True

2.5.2 Tautolojiler — De Morgan ve Karşıt

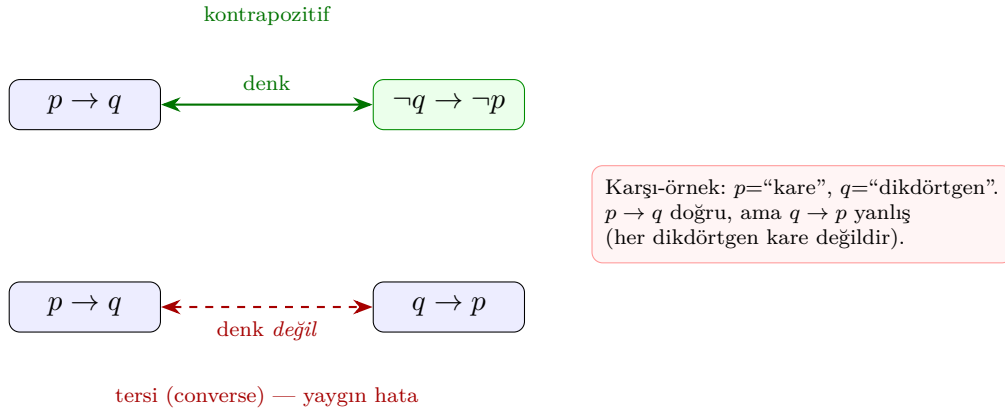
Teorem 2.2 (De Morgan Yasaları).

$$\neg(p \wedge q) \leftrightarrow \neg p \vee \neg q$$

$$\neg(p \vee q) \leftrightarrow \neg p \wedge \neg q$$

Teorem 2.3 (Karşıt (Contrapositive)). $(p \rightarrow q) \leftrightarrow (\neg q \rightarrow \neg p)$

En sık yapılan mantık hatası, bir içermeyi **tersiyle (converse)** karıştırmaktır. $p \rightarrow q$ 'nin doğru biçimde denk olduğu ifade *kontrapozitif* $\neg q \rightarrow \neg p$. Buna karşılık $q \rightarrow p$ (tersi) genellikle denk *değildir*.



Şekil 2.2: Kontrapozitif denkliği $p \rightarrow q \Leftrightarrow \neg q \rightarrow \neg p$ (yeşil) ile denk *olmayan* tersi $q \rightarrow p$ (kırmızı kesikli).

Karşı-örnek

p = “ x bir karedir”, q = “ x bir dikdörtgendir” olsun. $p \rightarrow q$ doğrudur (her kare dikdörtgendir), ama tersi $q \rightarrow p$ yanlıştır (her dikdörtgen kare değildir). Kontrapozitif $\neg q \rightarrow \neg p$ = “dikdörtgen değilse kare de değildir” ise yine doğrudur — çünkü o $p \rightarrow q$ ile denktir.

İspat eskizi (kontrapozitif). $p \rightarrow q$ ile $\neg q \rightarrow \neg p$ aynı doğruluk sütununu üretir; dört satırda da çakışır, dolayısıyla $(p \rightarrow q) \leftrightarrow (\neg q \rightarrow \neg p)$ tautolojidir.

p	q	$p \rightarrow q$	$\neg q \rightarrow \neg p$
D	D	D	D
D	Y	Y	Y
Y	D	D	D
Y	Y	D	D

İspat eskizi (De Morgan). $p \wedge q$ yalnızca her ikisi doğruyken doğrudur; öyleyse $\neg(p \wedge q)$ “en az biri yanlış” demektir. $\neg p \vee \neg q$ de tam olarak bunu söyler. İki sütun her atamada çakıştığından $\neg(p \wedge q) \leftrightarrow \neg p \vee \neg q$ tautolojidir; ikincil yasa $\neg(p \vee q) \leftrightarrow \neg p \wedge \neg q$ ile $\vee$ rolleri değiştirilerek aynı şekilde gösterilir.

p	q	$p \wedge q$	$\neg(p \wedge q)$	$\neg p \vee \neg q$
D	D	D	Y	Y
D	Y	Y	D	D
Y	D	Y	D	D
Y	Y	Y	D	D

```

1 def check_tautology(name, func):
2     ok = all(func(Proposition("p", tp), Proposition("q", tq)).
3               truth_value
4               for tp, tq in [(True, True), (True, False), (False, True), (
5                               False, False)])
6     print(f"{name}: {'tautoloji' if ok else 'DEGIL'}")
7
8 check_tautology("neg(p&q) <-> neg_p|neg_q",
9               lambda p, q: iff(negate(conjunction(p, q)), disjunction(negate(p),
10                                negate(q))))
11
12 check_tautology("(p->q) <-> (neg_q->neg_p)",
13               lambda p, q: iff(implies(p, q), implies(negate(q), negate(p))))

```

```

neg(p&q) <-> neg_p|neg_q: tautoloji
(p->q) <-> (neg_q->neg_p): tautoloji

```

2.5.3 Niceleyiciler

```

1 X = [1, 2, 3, 4, 5]
2 print("for_all(X, x>0)      :", for_all(X, lambda x: x > 0))
3 print("for_all(X, x>2)      :", for_all(X, lambda x: x > 2))
4 print("there_exists(X, x>4)  :", there_exists(X, lambda x: x > 4))
5 print("unique_exists(X, x==3):", unique_exists(X, lambda x: x == 3))

```

```

for_all(X, x>0)      : True
for_all(X, x>2)      : False
there_exists(X, x>4) : True
unique_exists(X, x==3): True

```

2.5.4 Topoloji Uygulaması — T0 ve T1

Tanım 2.4 (T0 — Kolmogorov). $\forall x \neq y \in X, \exists$ açık $U : (x \in U, y \notin U)$ veya $(y \in U, x \notin U)$.

Tanım 2.5 (T1 — Fréchet). $\forall x \neq y \in X, \exists$ açık $U : x \in U, y \notin U$.

```

1 from pytop import sierpinski_space, discrete_topology,
   indiscrete_topology
2
3 def is_t0_logic(space):
4     pts = list(space.carrier)
5     opens = list(space.topology)
6     return for_all(
7         [(x, y) for x in pts for y in pts if x != y],
8         lambda pair: there_exists(
9             opens, lambda U: (pair[0] in U) != (pair[1] in U)
10        )
11    )
12
13 def is_t1_logic(space):
14     pts = list(space.carrier)
15     opens = list(space.topology)
16     return for_all(
17         [(x, y) for x in pts for y in pts if x != y],
18         lambda pair: there_exists(
19             opens, lambda U: pair[0] in U and pair[1] not in U
20        )
21    )
22
23 for name, sp in [("Sierpinski", sierpinski_space()),
24                 ("Discrete ", discrete_topology(0, 1)),
25                 ("Indiscrete", indiscrete_topology(0, 1))]:
26     print(f"{name} T0={is_t0_logic(sp)} T1={is_t1_logic(sp)}")

```

Sierpinski	T0=True	T1=False
Discrete	T0=True	T1=True
Indiscrete	T0=False	T1=False

2.6 Alıştırmalar

1. Beş önerme değeriyle tam bir doğruluk tablosu oluşturun; *implies* kaç (p, q) çiftinde False döner?
2. `unique_exists([0,1,2,3,4], predicate)` değerini True yapan üç farklı koşul yazın.
3. T2 (Hausdorff) aksiyomunu `for_all` ve `there_exists` kullanarak kodlayın: $\forall x \neq y, \exists$ açık $U, V : x \in U, y \in V, U \cap V = \emptyset$. Sierpiński, discrete ve indiscrete için test edin.

4. (*Teori*) `implies(p, q)` neden yalnızca $p = T, q = F$ durumunda yanlıştır? Boş doğruluk (vacuous truth) kavramını açıklayın.
5. (*Teori*) De Morgan yasalarını önerme mantığı için ispatlayın: $\neg(p \wedge q) \leftrightarrow \neg p \vee \neg q$.
6. `check_tautology` yardımcı fonksiyonunu kullanarak **dağılma yasasını** doğrulayın: $p \wedge (q \vee r) \leftrightarrow (p \wedge q) \vee (p \wedge r)$. Üç değişken olduğundan sekiz (p, q, r) atamasının tümünü gezmeniz gerekir; `iff`'in her atamada `True` döndüğünü gösterin.
7. (*Teori*) Bir öğrenci " $p \rightarrow q$ doğruysa $q \rightarrow p$ de doğrudur" diyor. Bu iddianın neden yanlış olduğunu, kontrapozitif $\neg q \rightarrow \neg p$ ile tersi $q \rightarrow p$ arasındaki farkı vurgulayan somut bir karşı-örnekle açıklayın.

İpucu: $p = \text{"tam sayı 4'e bölünür"}$, $q = \text{"tam sayı çifttir"}$.

Bölüm 3

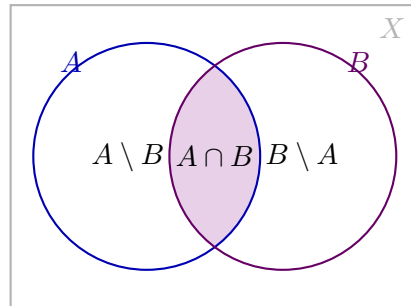
Küme Teorisi ve Fonksiyon Temelleri

pytop'un tüm modülleri küme teorisi ve fonksiyon kavramları üzerine kuruludur. Bu bölüm kılavuzun geri kalanında varsayılan ön koşulları pytop API'siyle pekiştirir.

3.1 Konu

Tanım 3.1 (Altküme ve Güç Kümesi). $A \subseteq B \iff \forall a \in A, a \in B$. **Güç kümesi:** $\mathcal{P}(A) = \{S : S \subseteq A\}$, $|\mathcal{P}(A)| = 2^{|A|}$.

İki kümenin işlemleri Venn şemasıyla görselleştirilir: $A \cup B$ üç bölgenin tamamı, $A \cap B$ ortadaki örtüşen bölge, $A \setminus B$ ise A 'nın B 'den ayrık kalan kısmıdır.



$A \cup B$ = her uc bolge
 $A \cap B$ = orta bolge $A \setminus B$ = sol bolge

Şekil 3.1: İki küme için $A \cup B$, $A \cap B$ ve $A \setminus B$ bölgeleri.

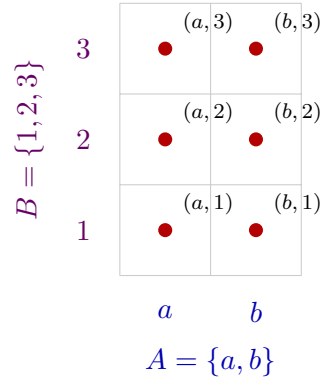
Sezgi

Tümleyen her zaman bir **evrene** (X) göre tanımlıdır; " A^c " tek başına anlamsızdır. pytop bunu `complement(subset, universe)` imzasıyla zorunlu kılar.

Karşı-örnek

" $A \cup B$ her zaman A 'dan büyüktür" yanlıştır: $B \subseteq A$ ise $A \cup B = A$. Örneğin $A = \{1, 2, 3\}$, $B = \{2\}$ için $|A \cup B| = |A| = 3$.

Tanım 3.2 (Kartezyen Çarpım). $A \times B = \{(a, b) : a \in A, b \in B\}$ ve $|A \times B| = |A| \cdot |B|$. A üzerinde bir bağıntı $R \subseteq A \times A$.



$$|A \times B| = |A| \cdot |B| = 2 \cdot 3 = 6$$

Şekil 3.2: $A \times B$ kartezyen çarpım ızgarası ($|A \times B| = 6$).

Karşı-örnek

Kartezyen çarpım değişmeli değildir: $A \times B \neq B \times A$ (genel olarak). $(a, 1) \in A \times B$ ama $(a, 1) \notin B \times A$, çünkü $B \times A$ 'nın ikilileri $(1, a)$ biçimindedir.

Tanım 3.3 (Denklik Bağıntısı). A üzerinde $R \subseteq A \times A$ bağıntısı **denklik bağıntısıdır** iff: yansımali ($\forall a, (a, a) \in R$), simetrik ($(a, b) \in R \Rightarrow (b, a) \in R$), geçişli ($(a, b), (b, c) \in R \Rightarrow (a, c) \in R$). Denklik sınıfı $[a] = \{b \in A : (a, b) \in R\}$.

Tanım 3.4 (Fonksiyon Türleri). $f : A \rightarrow B$ için:

- **Enjeksiyon:** $f(a_1) = f(a_2) \Rightarrow a_1 = a_2$
- **Sürjeksiyon:** $\forall b \in B, \exists a \in A : f(a) = b$
- **Bijeksiyon:** Enjeksiyon + Sürjeksiyon

3.2 Teoremler

Teorem 3.5 (De Morgan). $(A \cup B)^c = A^c \cap B^c$ ve $(A \cap B)^c = A^c \cup B^c$.

İspat eskizi. Çift kapsama ile: $x \in (A \cup B)^c \Leftrightarrow x \notin A \cup B \Leftrightarrow x \notin A$ ve $x \notin B \Leftrightarrow x \in A^c \cap B^c$. Her adım çift yönlü olduğundan iki küme aynı elemanlara sahiptir. İkinci eşitlik “veya”nın değillemesiyle simetrik biçimde elde edilir. $\square$

Teorem 3.6 (Güç Kümesi Boyutu). $|A| = n \Rightarrow |\mathcal{P}(A)| = 2^n$.

İspat eskizi. Her $S \subseteq A$, A 'nın n elemanının her biri için “ S içinde mi?” sorusuna evet/hayır cevabı veren bir ikili dizgiyle birebir eşleşir. n bağımsız ikili seçim olduğundan toplam 2^n . Tümevarımla: $|A| = 0$ için $|\mathcal{P}| = 1 = 2^0$; bir eleman eklendiğinde her eski alt küme hem eleman-sız hem eleman-lı sayılır, miktar ikiye katlanır. $\square$

Teorem 3.7 (Denklik Sınıfları Bölüntü Oluşturur). $\sim$ denklik bağıntısı ise $\{[a] : a \in A\}$ A 'nın bir bölüntüsüdür: sınıflar örtüşmez ve birleşimleri A 'dır.

Teorem 3.8 (Ön-Görüntü Özellikleri). $f : A \rightarrow B$ ve $S_1, S_2 \subseteq B$ için: $f^{-1}(S_1 \cup S_2) = f^{-1}(S_1) \cup f^{-1}(S_2)$ ve $f^{-1}(S_1 \cap S_2) = f^{-1}(S_1) \cap f^{-1}(S_2)$.

3.3 Algoritmalar

3.3.1 Güç Kümesi — $O(2^{|A|})$

Algorithm 2 PowerSet(A)

```

1: result  $\leftarrow \{\emptyset\}$ 
2: for each  $a \in A$  do
3:   result  $\leftarrow$  result  $\cup \{S \cup \{a\} : S \in \text{result}\}$ 
4: end for
5: return result

```

3.3.2 Denklik Bağıntısı Doğrulama — $O(|R| \cdot |A|)$

Algorithm 3 IsEquivalence(A, R)

```

1: for each  $a \in A$  do
2:   if  $(a, a) \notin R$  then return False
3:   end if
4: end for
5: for each  $(a, b) \in R$  do
6:   if  $(b, a) \notin R$  then return False
7:   end if
8: end for
9: for each  $(a, b) \in R, (b, c) \in R$  do
10:  if  $(a, c) \notin R$  then return False
11:  end if
12: end for
13: return True

```

3.4 pytop API

```

1 from pytop import (
2     make_set, power_set, empty_set,
3     set_union, set_intersection, set_difference, complement,
4     cartesian_product, indexed_union, indexed_intersection,
5     is_subset, is_proper_subset, equal_sets,
6     make_relation, is_equivalence_relation,
7     identity_relation, inverse_relation, compose_relations,
8     relation_profile,
9     equivalence_class, partition_from_equivalence,
10    total_order_from_list, is_partial_order, is_total_order,
11    normalize_finite_map_data,
12    is_injective_finite_map, is_surjective_finite_map,
13    is_bijective_finite_map,
14    image_of_subset_finite, preimage_of_subset_finite,
15 )

```



```

make_set(*elements) → frozenset
power_set(values) → set[frozenset]
complement(subset, universe) → set — evrene göre  $A^c$ 
cartesian_product(left, right) → set[tuple] —  $A \times B$ 
indexed_union(family) / indexed_intersection(family) → set
make_relation(carrier, *pairs) → set[tuple]
relation_profile(carrier, relation) → dict
normalize_finite_map_data(domain, codomain, mapping) → FiniteMapData

```

3.5 Örnekler

Örnek 5.1 — Küme İşlemleri

```

1 A = make_set(1, 2, 3)
2 B = make_set(2, 3, 4)
3 print("A union B:", sorted(set_union(A, B)))
4 print("A inter B:", sorted(set_intersection(A, B)))
5 print("A diff B:", sorted(set_difference(A, B)))
6 print("{2,3} subset A:", is_subset({2, 3}, A))

```

```

A union B: [1, 2, 3, 4]
A inter B: [2, 3]
A diff B: [1]
{2,3} subset A: True

```

Örnek 5.2 — Güç Kümesi

```

1 P = power_set([1, 2, 3])
2 print("boyut:", len(P)) # 8 = 2^3

```

```
boyut: 8
```

Örnek 5.3 — Denklik Bağıntısı

```

1 carrier = [1, 2, 3, 4]
2 rel_eq = make_relation(carrier,
3     (1,1),(2,2),(3,3),(4,4),(1,3),(3,1),(2,4),(4,2))
4 print("denklik:", is_equivalence_relation(carrier, rel_eq))
5 prof = relation_profile(carrier, rel_eq)
6 print("ıyansimal:", prof['is_reflexive'])
7 print("simetrik:", prof['is_symmetric'])
8 print("gecisli:", prof['is_transitive'])
9
10 # Gecissiz (1,2),(2,3) var ama (1,3) yok
11 rel_bad = make_relation(carrier,
12     (1,1),(2,2),(3,3),(4,4),(1,2),(2,1),(2,3),(3,2))
13 print("gecissiz denklik mi:", is_equivalence_relation(carrier,
14     rel_bad))

```

```

denklik: True
yansimalı: True
simetrik: True
gecisli: True
gecissiz denklik mi: False

```

Örnek 5.4 — Fonksiyon Türleri

```

1 f = normalize_finite_map_data(
2   [1,2,3], ['a','b','c'], {1:'a', 2:'b', 3:'c'})
3 print("bijeksiyon:", is_bijective_finite_map(f))
4
5 g = normalize_finite_map_data(
6   [1,2,3], ['x','y'], {1:'x', 2:'x', 3:'y'})
7 print("g enjeksiyon:", is_injective_finite_map(g))
8 print("g surjeksiyon:", is_surjective_finite_map(g))

```

```

bijeksiyon: True
g enjeksiyon: False
g surjeksiyon: True

```

Örnek 5.5 — Görüntü ve Ön-Görüntü

```

1 f = normalize_finite_map_data(
2   [1,2,3,4], ['a','b','c','d'], {1:'a',2:'b',3:'c',4:'d'})
3 print("f({1,2}):", sorted(image_of_subset_finite(f, [1, 2])))
4 print("f^-1({b,c}):", sorted(preimage_of_subset_finite(f, ['b','c'])))

```

```

f({1,2}): ['a', 'b']
f^-1({b,c}): [2, 3]

```

Örnek 5.6 — De Morgan Yasaları (Tümleyenle)

```

1 X = make_set(1, 2, 3, 4, 5, 6)
2 A = make_set(1, 2, 3)
3 B = make_set(3, 4, 5)
4 lhs1 = complement(set_union(A, B), X)
5 rhs1 = set_intersection(complement(A, X), complement(B, X))
6 print("(A u B)^c:", sorted(lhs1), "==", sorted(rhs1))
7 lhs2 = complement(set_intersection(A, B), X)
8 rhs2 = set_union(complement(A, X), complement(B, X))
9 print("(A n B)^c:", sorted(lhs2), "==", sorted(rhs2))

```

```

(A u B)^c: [6] == [6]
(A n B)^c: [1, 2, 4, 5, 6] == [1, 2, 4, 5, 6]

```

Örnek 5.7 — Kartezyen Çarpım

```

1 P = make_set('a', 'b')
2 Q = make_set(1, 2, 3)
3 prod = cartesian_product(P, Q)
4 print("|PxQ| =", len(prod)) # 2*3 = 6
5 print(sorted(prod, key=lambda t: (t[0], t[1])))

```

```

|PxQ| = 6
[('a', 1), ('a', 2), ('a', 3), ('b', 1), ('b', 2), ('b', 3)]

```

Örnek 5.8 — İndeksli Aile

```

1 fam = {0: [1, 2, 3], 1: [2, 3, 4], 2: [3, 4, 5]}
2 print("birlesim:", sorted(indexed_union(fam)))
3 print("kesisim :", sorted(indexed_intersection(fam)))

```

```

birlesim: [1, 2, 3, 4, 5]
kesisim : [3]

```

3.6 Alıştırmalar

Kodlama

- K1. $A = \{1, 2, 3, 4, 5\}$, $B = \{3, 4, 5, 6, 7\}$ için $A \cup B$, $A \cap B$, $A \setminus B$ 'yi hesaplayın.
- K2. $\{1, 2, 3, 4\}$ için güç kümesini üretin; $|\mathcal{P}| = 16$ olduğunu doğrulayın.
- K3. $R = \{(i, j) : 0 \leq i, j \leq 2\}$ (tam çarpım) üzerinde `relation_profile` çalıştırın ve denklik olduğunu gösterin.
- K4. $X = \{1, \dots, 8\}$, $A = \{1, 2, 3, 4\}$, $B = \{3, 4, 5, 6\}$ için `complement` ile her iki De Morgan eşitliğini doğrulayın.
- K5. $P = \{x, y, z\}$, $Q = \{0, 1\}$ için `cartesian_product` ile $P \times Q$ ve $Q \times P$ üretin; boyutların eşit ama kümelerin farklı olduğunu `equal_sets` ile gösterin.

Teori

- T1. $A \subseteq B \Leftrightarrow A \cap B = A$ olduğunu ispatlayın.
- T2. $f : A \rightarrow B$ ve $g : B \rightarrow C$ enjeksiyon ise $g \circ f$ de enjeksiyondur.

Kısım II

Nokta-Küme Topolojisi

Bölüm 4

Topolojik Uzaylar

4.1 Konu

4.1.1 Sezgisel Giriş

Topoloji, bir kümedeki noktaların birbirine “yakın” olup olmadığını, aralarındaki sürekliliği ve şekilsel özellikleri, mesafe kavramına gerek duymadan inceleyen matematiksel bir disiplindir. Bunu mümkün kılan temel araç *açık küme* kavramıdır.

Düşünelim: gerçek doğruda (a, b) aralığı “açık”tır; her noktasının çevresinde küçük bir açık aralık daha bulunur. Bu sezgiyi soyutlayarak *herhangi* bir küme üzerinde topoloji tanımlamak mümkündür.

Sezgi

Bir şehir haritasında “yakınlık”ı sokak mesafesiyle ölçebilirsiniz; ama “hangi mahalleler birbirine komşu?” sorusu mesafe olmadan da anlamlıdır. Topoloji tam bunu yapar: mesafeyi unuttur, yalnızca “hangi kümeler bir noktanın çevresini oluşturur?” bilgisini tutar. Matematiksel karşılığı: τ ailesi her noktanın “çevre” kavramını açık kümeler aracılığıyla kodlar; süreklilik ve yakınsama gibi kavramlar yalnız τ ’ya başvurularak tanımlanır.

4.1.2 Formal Tanım

Tanım 4.1 (Topolojik Uzay). Bir X kümesi ve $\tau \subseteq \mathcal{P}(X)$ ailesi verilsin. (X, τ) bir **topolojik uzay**, τ ise X üzerinde bir **topoloji** olarak adlandırılır, eğer aşağıdaki üç aksiyom sağlanıyorsa:

$$(T1) \quad \emptyset \in \tau \text{ ve } X \in \tau$$

$$(T2) \quad U, V \in \tau \Rightarrow U \cap V \in \tau$$

$$(T3) \quad \{U_\alpha\}_{\alpha \in I} \subseteq \tau \Rightarrow \bigcup_{\alpha \in I} U_\alpha \in \tau$$

τ ’nın elemanları **açık küme** olarak adlandırılır.

Aksiyomların her biri ayrı bir iş görür: (T1) uzayın tamamının ve boş kümenin her zaman “gözlemlenebilir” olmasını garanti eder; (T2) iki gözlemin kesişiminin de gözlem olmasını sağlar — sonlu kesişimle sınırlı kalması bilinçli bir tercihtir (bkz. Teorem 4.4);

(T3) keyfi birleşime izin vererek küçük çevrelerden büyük çevre kurma işlemini serbest bırakır.

Karşı-örnek

$X = \{1, 2, 3\}$ üzerinde $\sigma = \{\emptyset, \{1\}, \{2\}, X\}$ ailesi bir topoloji *değildir*: $\{1\} \cup \{2\} = \{1, 2\} \notin \sigma$ olduğundan (T3) ihlal edilir. (T1) ve (T2) sağlansa bile tek bir eksik birleşim aileyi topoloji olmaktan çıkarır. Bu ailenin `make_topology`'ye verilince ne olduğunu Örnekler bölümündeki “Dikkat” kutusunda göreceğiz.

4.1.3 Temel Örnekler

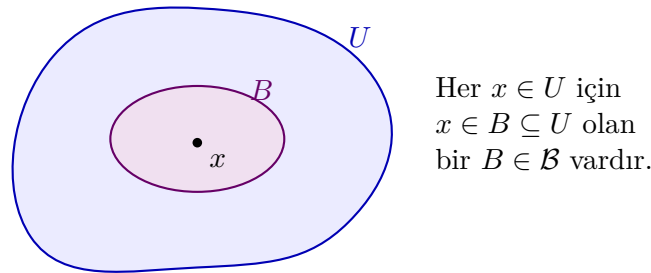
Topoloji	Tanım	Açık kümeler
Ayrık (discrete)	Her alt küme açık	$\tau = \mathcal{P}(X)$
İndirgenmiş (indiscrete)	Yalnızca $\emptyset$ ve X açık	$\tau = \{\emptyset, X\}$
Kosonlu (cofinite)	U açık $\iff U = \emptyset$ veya $X \setminus U$ sonlu	—
Sierpiński	$X = \{0, 1\}$, $\tau = \{\emptyset, \{1\}, X\}$	3 açık küme

Tablo 4.1: Standart topoloji örnekleri

4.1.4 Baz ve Alt-Baz

Tanım 4.2 (Baz). $\mathcal{B} \subseteq \tau$ ailesi, her $U \in \tau$ ve her $x \in U$ için $x \in B \subseteq U$ sağlayan bir $B \in \mathcal{B}$ varsa τ 'ya **baz** denir.

Şekil 4.1 koşulu resmeder: açık kümenin her noktası, tamamen o kümenin içinde kalan bir baz elemanıya sarılır.



Şekil 4.1: Baz koşulunun geometrisi: U açık kümesinin her x noktası için $x \in B \subseteq U$ sağlayan bir $B \in \mathcal{B}$ vardır. Topolojinin tüm açıkları bu tür “yapı taşları”nın birleşimleridir.

Neden önemli?

Baz, büyük bir topolojiyi küçük bir çekirdekle temsil etme aracıdır. Gerçek doğrunun standart topolojisi sayılamaz çoklukta açık küme içerir; ama tümü, sayılabilir $\{(a, b) : a < b, a, b \in \mathbb{Q}\}$ bazından üretilir. `pytop` tarafında `topology_from_basis` tam bu ilkeyle çalışır: yalnız baz elemanlarını verirsiniz, kapanışı kütüphane hesaplar (Algoritma 4).

Baz Koşulları:

$$(B1) \bigcup \mathcal{B} = X$$

$$(B2) B_1, B_2 \in \mathcal{B} \text{ ve } x \in B_1 \cap B_2 \text{ ise bir } B_3 \in \mathcal{B} \text{ vardır: } x \in B_3 \subseteq B_1 \cap B_2$$

Tanım 4.3 (Alt-Baz). $\mathcal{S} \subseteq \mathcal{P}(X)$ ailesi; sonlu kesişimleri baz, onların birleşimleri ise τ 'yu oluşturur.

4.1.5 Sonlu ve Sonsuz Uzaylar

pytop'ta iki temel sınıf bulunur:

- **FiniteTopologicalSpace:** Taşıyıcı (X) sonlu; topoloji açıkça frozenset'ler koleksiyonu olarak saklanır. Tüm hesaplamalar tam ve kesindir.
- **Sonsuz türevler:** Taşıyıcı simgesel ('R', 'N'); topology =None; etiket tabanlı sembolik çıkarım yapılır.

4.2 Teoremler

Teorem 4.4 (Aksiyomların Yeterliliği). (X, τ) bir topolojik uzay olsun. $T1$ – $T3$ aksiyomları, sonlu herhangi bir açık küme ailesinin kesişiminin τ 'da yer aldığını garanti eder:

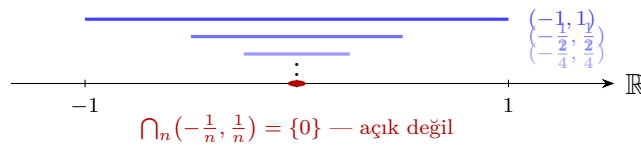
$$U_1, \dots, U_n \in \tau \Rightarrow \bigcap_{i=1}^n U_i \in \tau.$$

Not: Sonsuz kesişim gerekmez. Gerçek doğrudaki $\bigcap_{n=1}^{\infty} (-\frac{1}{n}, \frac{1}{n}) = \{0\}$ açık değildir.

Rehberli Kanıt. Strateji: n üzerinde tümevarım; tek araç (T2).

1. $n = 1$: $U_1 \in \tau$ zaten verili.
2. $n = k$ için doğru varsayalım: $V = \bigcap_{i=1}^k U_i \in \tau$.
3. $n = k + 1$: $\bigcap_{i=1}^{k+1} U_i = V \cap U_{k+1}$ olup (T2) gereği τ 'dadır.

Sonsuz kesişimde 3. adım çalışmaz: tümevarım yalnız sonlu n 'lere ulaşır. Şekil 4.2 bunun gerçek bir başarısızlık olduğunu gösterir: her sonlu kesişim açıkken limit $\{0\}$ açık değildir. $\square$



Şekil 4.2: (T2)'nin sonlu kesişimle sınırlı olmasının nedeni: iç içe $(-\frac{1}{n}, \frac{1}{n})$ açık aralıklarının sonsuz kesişimi tek noktaya çöker ve $\{0\}$ standart topolojide açık değildir.

Teorem 4.5 (Baz Teoremi). $\mathcal{B} \subseteq \mathcal{P}(X)$ ailesi (B1)–(B2) koşullarına sağlıyorsa,

$$\tau_{\mathcal{B}} = \{U \subseteq X : \forall x \in U, \exists B \in \mathcal{B}, x \in B \subseteq U\}$$

bir topoloji olur ve $\mathcal{B}$ bu topolojiye baz oluşturur.

Rehberli Kanıt. Strateji: $\tau_{\mathcal{B}}$ 'nin üç aksiyomu sağladığını, her adımda yalnız (B1)–(B2)'yi kullanarak doğrularız.

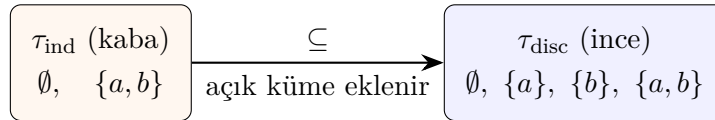
1. **(T1):** $\emptyset \in \tau_{\mathcal{B}}$, çünkü “her $x \in \emptyset$ için...” koşulu boş yere doğrudur. $X \in \tau_{\mathcal{B}}$: $x \in X$ verilsin; (B1) gereği $x \in \bigcup \mathcal{B}$, yani $x \in B \subseteq X$ olan bir $B \in \mathcal{B}$ vardır.
2. **(T2):** $U, V \in \tau_{\mathcal{B}}$ ve $x \in U \cap V$ olsun. Tanım gereği $x \in B_U \subseteq U$ ve $x \in B_V \subseteq V$ bulunur. (B2), $x \in B_3 \subseteq B_U \cap B_V \subseteq U \cap V$ veren bir B_3 sağlar; x keyfi olduğundan $U \cap V \in \tau_{\mathcal{B}}$.
3. **(T3):** $x \in \bigcup_{\alpha} U_{\alpha}$ ise $x \in U_{\alpha_0}$ olan bir indis vardır; U_{α_0} 'ın tanımından gelen B , $x \in B \subseteq U_{\alpha_0} \subseteq \bigcup_{\alpha} U_{\alpha}$ koşulunu da sağlar.
4. $\mathcal{B}$ 'nin $\tau_{\mathcal{B}}$ 'ye baz olması, $\tau_{\mathcal{B}}$ 'nin tanımının Tanım 4.2'teki koşulu birebir içermesindendir.

□

Bu teorem, `topology_from_basis`'in döndürdüğü ailenin gerçekten topoloji olduğunun matematiksel güvencesidir; fonksiyon ayrıca (B1)–(B2)'yi sağlamayan girdiyi `BasisConstructionError` ile reddeder.

Teorem 4.6 (Karşılaştırma). X üzerinde iki topoloji $\tau_1 \subseteq \tau_2$ ise τ_1 **kaba (coarser)**, τ_2 **ince (finer)** topoloji olarak adlandırılır. Ayırık topoloji en ince, indirgenmiş topoloji en kaba topolojidir.

Rehberli Kanıt. Herhangi bir τ için $\tau \subseteq \mathcal{P}(X) = \tau_{\text{disc}}$ topoloji tanımının kendisinden gelir (aksiyom gerekmez); (T1) aksiyomu da $\{\emptyset, X\} = \tau_{\text{ind}} \subseteq \tau$ verir. □



Şekil 4.3: Aynı $X = \{a, b\}$ üzerinde iki uç: indirgenmiş topoloji (kaba) ile ayırık topoloji (ince). Açık küme eklemek topolojiyi inceltir; tüm topolojiler bu iki uç arasında yaşar.

`pytop` bu uçları `tags` ile işaretler: `indiscrete_topology` nesnesi `indiscrete`, `discrete_topology` nesnesi `discrete` etiketi taşır.

4.3 Algoritmalar

4.3.1 Bazdan Topoloji Üretimi

Problem: X ve $\mathcal{B} \subseteq \mathcal{P}(X)$ verilsin; $\tau_{\mathcal{B}}$ 'yi hesapla.

Karmaşıklık:

- En kötü durum: $O(|\mathcal{B}| \cdot 2^{|\mathcal{B}|})$ (tüm alt-ailelerin birleşimi)
- Pratik (çakışmayan baz elemanları): $O(|\tau| \cdot |\mathcal{B}|)$
- Uzay: $O(|\tau|)$ açık küme saklar

Algorithm 4 Bazdan Topoloji Üretimi**Require:** X (taşıyıcı), $\mathcal{B}$ (baz ailesi)**Ensure:** τ (kapalı topoloji)

```

1:  $\tau \leftarrow \{\emptyset, X\}$ 
2: for each non-empty  $\mathcal{S} \subseteq \mathcal{B}$  do
3:    $\tau.add(\bigcup_{B \in \mathcal{S}} B)$ 
4: end for
5: CloseUnderIntersection( $\tau$ ) return  $\tau$ 

```

Algorithm 5 Kesişim Altında Kapatma

```

1:  $changed \leftarrow \text{true}$ 
2: while  $changed$  do
3:    $changed \leftarrow \text{false}$ 
4:   for each  $U, V \in \tau$  do
5:     if  $U \cap V \notin \tau$  then
6:        $\tau.add(U \cap V)$ ;  $changed \leftarrow \text{true}$ 
7:     end if
8:   end for
9: end while

```

4.3.2 İz Sürme: Küçük Bir Girdiyle Adım Adım

$X = \{1, 2, 3\}$, $\mathcal{B} = \{\{1\}, \{2, 3\}\}$ girdisiyle Algoritma 4:

4.3.3 Alt-Bazdan Topoloji Üretimi

$\mathcal{S}$ verilsin; önce $\mathcal{S}$ 'nin sonlu kesişimlerinden baz oluştur, ardından Algoritma 4'yi uygula.

Karmaşıklık: $O(|\mathcal{S}|^k \cdot 2^{|\mathcal{S}|^k})$ — k : kesişim derinliği.

4.4 pytop API**4.4.1 Sınıflar**

Listing 4.1: Temel sınıf importları

```
1 from pytop import TopologicalSpace, FiniteTopologicalSpace
```

Alanlar: `carrier` (altta yatan küme), `topology` (açık kümeler), `tags` (özellik etiketleri), `metadata` (ek bilgiler).

4.4.2 Oluşturucular

Listing 4.2: Topoloji oluşturucu importları

```

1 from pytop import (
2     make_topology, discrete_topology, indiscrete_topology,
3     cofinite_topology, sierpinski_space,
4     topology_from_basis, topology_from_subbasis,
5 )

```

Adım	$\mathcal{S}$ (alt-aile)	Eklenen birleşim	τ (o ana dek)
0	—	—	$\{\emptyset, X\}$
1	$\{\{1\}\}$	$\{1\}$	$\{\emptyset, X, \{1\}\}$
2	$\{\{2, 3\}\}$	$\{2, 3\}$	$\{\emptyset, X, \{1\}, \{2, 3\}\}$
3	$\{\{1\}, \{2, 3\}\}$	$\{1\} \cup \{2, 3\} = X$	değişmez
4	kesişim kapanışı	$\{1\} \cap \{2, 3\} = \emptyset$	değişmez

Tablo 4.2: İz sürme: $|\tau| = 4$; döngü yeni küme üretmeyince durur.

Sınıf	Taşıyıcı	Topoloji
TopologicalSpace	Any	Any (None olabilir)
FiniteTopologicalSpace	sonlu frozenset	frozenset[frozenset]

Tablo 4.3: Temel uzay sınıfları

4.4.3 Tag Sistemi

pytop nesneleri tags kümesi aracılığıyla topolojik özellikler taşır. Örnek etiketler: "finite", "discrete", "t0", "t1", "hausdorff", "compact", "connected", "metric".

4.4.4 Tag Gerekçeleri

Etiketler, kurucunun inşa anında *garanti ettiği* gerçeklerdir:

Etiketler *eksiksiz değildir*: `cofinite_topology('a','b','c')` aslında ayrıktır ve Hausdorff'tur ($2^3 = 8$ açık), ama `discrete/hausdorff` etiketi taşımaz — `is_t2` yüklemi yine `true` döner. Kesin sorgu için Bölüm 6'daki `is_*` yüklemelerini kullanın; etiket, yüklemine yerine geçmez.

4.5 Örnekler

4.5.1 Örnek 1: Sierpiński Uzayı

Listing 4.3: Sierpiński uzayı

```

1 from pytop import sierpinski_space
2
3 s = sierpinski_space()
4 print("Taşıyıcı:", s.carrier)
5 print("Topoloji:", sorted(str(t) for t in s.topology))
6 print("Etiketler:", sorted(s.tags))

```

Listing 4.4: Çıktı

```

1 Taşıyıcı: frozenset({0, 1})
2 Topoloji: ['frozenset()', 'frozenset({0, 1})', 'frozenset({1})']
3 Etiketler: ['compact', 'connected', 'finite', 't0']

```

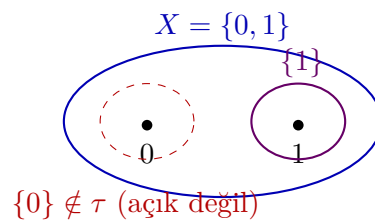
Fonksiyon	İmza	Açıklama
make_topology	(carrier, *open_sets)	Açık küme listesiyle inşa
discrete_topology	(*elements)	Ayrık topoloji
indiscrete_topology	(*elements)	İndirgenmiş topoloji
cofinite_topology	(*elements)	Kosonlu topoloji
sierpinski_space	()	Sierpiński uzayı
topology_from_basis	(carrier, basis)	Bazdan topoloji üret
topology_from_subbasis	(carrier, subbasis)	Alt-bazdan topoloji üret

Tablo 4.4: Topoloji oluşturu fonksiyonlar

Kurucu	Etiketler	Gerekçe
sierpinski_space()	compact, connected, finite, t0	Sonlu $\Rightarrow$ kompakt; $\{1\}$ tek y lü ayırır $\Rightarrow$ T0 (T1 değil); ay parçalanış yok $\Rightarrow$ bağlantılı
discrete_topology(1,2,3)	discrete, finite, hausdorff, metrizable, normal, regular	Her tekil açık $\Rightarrow$ tüm ayrı aksiyomları; 0–1 metriği ay topolojiyi üretir
indiscrete_topology(1,2,3)	compact, connected, finite, indiscrete	Açık yalnız $\emptyset$ ve X : parçala maz
cofinite_topology('a','b','c')	cofinite, compact, finite, t1	Tekiller kapalı $\Rightarrow$ T1
make_topology(...), finite_chain_space(n)	finite	Özellik çıkarımı yapılmaz; nız sonluluk işaretlenir

Tablo 4.5: Pilot bölümdeki kurucuların etiket gerekçeleri

Ne oldu? Çıktıdaki üç satırı tek tek okuyalım. Tasiyici satırı $X = \{0,1\}$ 'i verir. Topoloji satırındaki üç küme tam olarak Tanım 4.1'in istediği yapıdır: $\emptyset$ ve X (T1), tek ek açık küme $\{1\}$; $\{1\} \cap X = \{1\}$ ve $\{1\} \cup X = X$ zaten listede olduğundan (T2)–(T3) de sağlanır. Etiketler satırında t0 var ama t1 yok: 1'i içerip 0'ı dışlayan açık küme vardır ($\{1\}$), fakat 0'ı içerip 1'i dışlayan açık küme yoktur — T0 sağlanıp T1'in sağlanmamasının kaynağı budur (Bölüm 6, Örnek 1'de yüklemelerle test edilir). Şekil 4.4 açık küme yapısını gösterir.



Şekil 4.4: Sierpiński uzayının açıkları: $\emptyset$, $\{1\}$ ve X . Kesikli bölge $\{0\}$ 'ın açık *olmadığını* vurgular; bu asimetri uzayı T0 yapar ama T1 yapmaz.

4.5.2 Örnek 2: Ayrık Topoloji

Listing 4.5: Ayrık topoloji

```

1 from pytop import discrete_topology
2
3 d = discrete_topology(1, 2, 3)
4 print("|tau| =", len(d.topology))
5 print("Etiketler:", sorted(d.tags))

```

Listing 4.6: Çıktı

```

1 |tau| = 8
2 Etiketler: ['discrete', 'finite', 'hausdorff', 'metrizable', 'normal',
3            , 'regular']

```

Ne oldu? $n = 3$ eleman için $|\tau| = 2^3 = 8$: her alt küme açıktır. Her tekil küme açık olduğundan herhangi iki nokta kendi tekil komşuluklarıyla ayrılır — **hausdorff**, **normal**, **regular** etiketlerinin kaynağı budur. **metrizable**, 0–1 metriğinden gelir: $d(x, y) = 1$ ($x \neq y$) metriği tam olarak ayrık topolojiyi üretir. Listede **compact** yok; oysa uzay sonlu olduğu için kompakttır ve **is_compact** yüklemi **true** döner — fark için Tag Gereçekleri tablosuna bakın.

4.5.3 Örnek 3: make_topology ile Manuel İnşa

Listing 4.7: Manuel topoloji inşası

```

1 from pytop import make_topology
2
3 sp = make_topology({1, 2, 3}, {1}, {2, 3})
4 print("|tau| =", len(sp.topology))
5 print("Acik kumeler:", sorted(str(t) for t in sp.topology))

```

Listing 4.8: Çıktı

```

1 |tau| = 4
2 Acik kumeler: ['frozenset()', 'frozenset({1, 2, 3})',
3              'frozenset({1})', 'frozenset({2, 3})']

```

Ne oldu? **make_topology** verilen $\{1\}$ ve $\{2, 3\}$ açıklarına yalnızca $\emptyset$ ve X 'i ekledi; $|\tau| = 4$. Bu örnekte sonuç geçerli bir topolojidir, çünkü verilen iki küme ayrıktır: birleşimleri X , kesişimleri $\emptyset$ zaten ailededir. **make_topology** bu kapanışları *hesaplamaz* — aşağıdaki kutu, kapalı olmayan bir aile verildiğinde ne olduğunu gösterir.

Dikkat – sık hata

make_topology verdiğiniz aileye yalnızca $\emptyset$ ve X 'i ekler; birleşim/kesişim kapanışını **hesaplamaz** ve aksiyomları **denetlemez**. Aşağıdaki çağrıda $\{1\} \cup \{2\} = \{1, 2\}$ ailede yoktur; dönen nesne (T3)'ü ihlal eden, geçersiz bir “topoloji”dir. Kapanışın hesaplanmasını ve ailenin denetlenmesini istiyorsanız **topology_from_basis** kullanın — o, baz koşullarını sağlamayan aileyi **BasisConstructionError** ile reddeder.

Listing 4.9: Kapanış hesaplanmaz — sorumluluk kullanıcıda

```

1 from pytop import make_topology, topology_from_basis
2
3 sessiz = make_topology({1, 2, 3}, {1}, {2}) # denetlemez!
4 print("make_topology:", sorted(sorted(t) for t in sessiz.topology))
5
6 try:
7     topology_from_basis({1, 2, 3}, [{1}, {2}]) # B1 ihlali
8 except Exception as e:
9     print("topology_from_basis:", type(e).__name__)

```

Listing 4.10: Çıktı

```

make_topology: [[], [1], [1, 2, 3], [2]]
topology_from_topology: BasisConstructionError

```

4.5.4 Örnek 4: Alexandrov Zincir Uzayı

Listing 4.11: Zincir uzayı

```

1 from pytop import finite_chain_space
2
3 c = finite_chain_space(3)
4 print("Tasiyici:", c.carrier)
5 print("Topoloji:", sorted(str(t) for t in c.topology))

```

Listing 4.12: Çıktı

```

1 Tasiyici: (1, 2, 3)
2 Topoloji: ['set()', '{1, 2, 3}', '{1, 2}', '{1}']

```

Ne oldu? Çıktıdaki açıklar tam bir “önek merdiveni”dir: $\emptyset \subset \{1\} \subset \{1, 2\} \subset \{1, 2, 3\}$. Alexandrov zincir uzayında 1 her boş olmayan açıkta yer alır (“en açık” nokta), 3 yalnız X ’te görünür. Öneklerin kesişimi ve birleşimi yine önek olduğundan (T2)–(T3) otomatik sağlanır.

4.5.5 Örnek 5: Bazdan Topoloji Üretimi

Listing 4.13: Bazdan topoloji

```

1 from pytop import topology_from_basis
2
3 b = [{1}, {2, 3}, {4}]
4 ts = topology_from_basis({1, 2, 3, 4}, b)
5 print("Baz:", [set(x) for x in b])
6 print("|tau| =", len(ts.topology))
7 print("Topoloji:", sorted(str(t) for t in ts.topology))

```

Listing 4.14: Çıktı

```

1 Baz: [{1}, {2, 3}, {4}]
2 |tau| = 8
3 Topoloji: ['set()', '{1, 2, 3, 4}', '{1, 2, 3}', '{1, 4}',
4           '{1}', '{2, 3, 4}', '{2, 3}', '{4}']

```

Ne oldu? Baz $\{\{1\}, \{2, 3\}, \{4\}\}$ bir bölüntüdür: elemanları ikiye ikiye ayrılır. (B2) koşulu boş yere sağlanır (kesişimler boş olduğundan kontrol edilecek nokta yoktur) ve topoloji tüm alt-aile birleşimlerinden oluşur: $2^3 = 8$ açık küme. Çıktıdaki 8 küme, üç baz elemanının 2^3 alt kümesinin birleşimleriyle birebir eşleşir.

4.5.6 Örnek 6: Gerçek Doğru

Listing 4.15: Gerçek doğru

```

1 from pytop import real_line_metric
2
3 rl = real_line_metric()
4 print("Tasiyici:", rl.carrier)
5 print("Etiketler:", sorted(rl.tags))

```

Listing 4.16: Çıktı

```

1 Tasiyici: R
2 Etiketler: ['complete', 'connected', 'first_countable', 'hausdorff',
3            'infinite', 'lindelof', 'metric', 'not_compact',
4            'path_connected', 'second_countable', 'separable', 't0',
5            't1', 'uncountable']

```

Ne oldu? Tasiyici: R satırı taşıyıcının *simgesel* olduğunu söyler: gerçek doğrunun noktaları bellekte tutulmaz, `topology = None`'dir. Tüm topolojik bilgi etiketlerde kodlanmıştır: `connected`, `hausdorff`, `second_countable` gibi olumlu özellikler yanında `not_compact` gibi olumsuz bilgi de taşınır (Heine–Borel: $\mathbb{R}$ sınırlı değildir). Sonlu uzaylardaki “hesapla ve doğrula” yaklaşımının yerini burada “bilinen teoremleri etiketle” yaklaşımı alır; metrik tarafı Bölüm 14’te derinleşir.

4.5.7 Örnek 7: Aksiyom Denetimi ve Kapalı Kümeler

Bir açık küme ailesinin gerçekten topoloji *olup olmadığını* `is_topology` doğrudan denetler; bir topolojinin **kapalı** kümeleri (tümleyenleri açık olanlar) `closed_sets_from_topology` ile çıkarılır.

Listing 4.17: Aksiyom denetimi ve kapalı kümeler

```

1 from pytop import is_topology, make_topology,
   closed_sets_from_topology
2
3 X = {1, 2, 3}
4 gecerli = [set(), {1}, {2, 3}, {1, 2, 3}]

```

```

5 gecersiz = [set(), {1}, {2}, {1, 2, 3}]    # {1} U {2} = {1,2} eksik
      -> (T3) ihlal
6 print("gecerli aile :", is_topology(X, gecerli))
7 print("gecersiz aile :", is_topology(X, gecersiz))
8
9 sp = make_topology({1, 2, 3}, {1}, {2, 3})
10 kapali = closed_sets_from_topology(sp.carrier, sp.topology)
11 print("Acik :", sorted(sorted(t) for t in sp.topology))
12 print("Kapali:", sorted(sorted(c) for c in kapali))

```

Listing 4.18: Çıktı

```

1 gecerli aile : True
2 gecersiz aile : False
3 Acik : [[], [1], [1, 2, 3], [2, 3]]
4 Kapali: [[], [1], [1, 2, 3], [2, 3]]

```

Ne oldu? İlk iki satır, *karsiornek* kutusundaki aileyi makineyle doğrular: *gecersiz* ailede $\{1\} \cup \{2\} = \{1,2\}$ eksik olduğundan (T3) düşer ve *is_topology* *False* döner. Son iki satırda her açık kümenin tümleyeni alınır: $\emptyset$ ile X daima birbirinin tümleyenidir; $\{1\}$ ve $\{2,3\}$ de birbirinin tümleyenini olduğundan bu uzayda açık kümeler ailesi ile kapalı kümeler ailesi *çakışır*. Hem açık hem kapalı olan kümelere **clopen** denir — buradaki dört kümenin tümü clopen'dir. Kapanış/iç hesabı Bölüm 5'te bu temele oturur.

4.5.8 Örnek 8: Dışlanan-Nokta Topolojisi ve Komşuluk Karakteri

excluded_point_topology(n, p), $\{0, 1, \dots, n-1\}$ üzerinde p noktasını *dışlayan* topolojiyi kurar: bir küme açıktır $\iff$ ya p 'yi içermez ya da tüm uzaydır. *analyze_neighborhood_system* ise bir noktanın komşuluk sistemini ve **karakterini** (en küçük komşuluk tabanının boyutu) verir.

Listing 4.19: Dışlanan-nokta topolojisi ve komşuluk karakteri

```

1 from pytop import excluded_point_topology, make_topology,
      analyze_neighborhood_system
2
3 ep = excluded_point_topology(3, 0)    # X = {0,1,2}, 0 noktası
      dislanir
4 print("Tasiyici:", sorted(ep.carrier))
5 print("|tau| =", len(ep.topology))
6 print("Acik:", sorted(sorted(t) for t in ep.topology))
7 print("Etiketler:", sorted(ep.tags))
8
9 s = make_topology({1, 2, 3}, {1}, {1, 2})
10 r = analyze_neighborhood_system(list(s.carrier), list(s.topology),
      point=1)
11 print("1'in komşuluk sayisi:", r.value["neighborhood_count"])
12 print("1'in karakteri      :", r.value["character"])

```

Listing 4.20: Çıktı

```

1 Tasiyici: [0, 1, 2]
2 |tau| = 5
3 Acik: [[], [0, 1, 2], [1], [1, 2], [2]]
4 Etiketler: ['compact', 'connected', 'finite', 'first_countable',
5            'not_hausdorff', 'not_t1', 'second_countable', 'separable',
6            't0']
7 1'in komsuluk sayisi: 4
  1'in karakteri      : 3

```

Ne oldu? İlk blok dışlanan-nokta topolojisini gösterir: 0 noktası yalnız tüm uzay X 'te görüldüğünden, 0'ı içeren *tek* açık küme X 'tir; $\{1\}, \{2\}, \{1, 2\}$ açıktır ama $\{0\}$ açık değildir. Etiketlerde **t0** var ve **not_t1** var — yani uzay T_0 'dır ama T_1 değildir (Sierpiński ile aynı asimetri, üç noktaya genişlemiş hali). İkinci blok `analyze_neighborhood_system` çıktıdır: 1 noktasını içeren açık kümeler $\{1\}, \{1, 2\}, \{1, 2, 3\}$ olduğundan `neighborhood_count` = 4 ve **karakter** = 3 — bir noktanın “yerel karmaşıklığını” sayan ilk kardinal işlevdir (Bölüm 16'da derinleşir).

4.6 Alıştırmalar

Kodlama Alıştırmaları

K1. `cofinite_topology('a', 'b', 'c')` ile üç noktalı kosonlu uzayı kurun; topolojisini ve etiketlerini yazdırın, `is_t1` ve `is_t2` ile test edin. Gözlem: sonlu bir kümede kosonlu topoloji ayrık topolojiyle çakışır. T_1 olup Hausdorff olmayan bir örnek için taşıyıcının neden sonsuz olması gerektiğini bir cümleyle açıklayın (krş. Bölüm 6, Örnek 2).

İpucu: $|\tau| = 2^3$ çıkacak; anahtar, Bölüm 6'daki “Sonlu $T_1 \iff$ Ayrık” teoremidir. (çözüm)

K2. `topology_from_subbasis({1,2,3,4}, [{1,2},{3,4},{2,3}])` çağrısının ürettiği topolojinin kaç açık küme içerdiğini bulun ve açık kümeleri listeleyin.

İpucu: Önce alt-baz çiftlerinin kesişimlerini elle listeleyin ($\{2\}, \{3\}, \emptyset$); sonra birleşimleri sayın. (çözüm)

K3. `finite_chain_space(5)` ile bir Alexandrov-5 zinciri oluşturun; topolojinin kaç açık küme içerdiğini ve hangi elemanın “en açık” nokta olduğunu bulun.

İpucu: Açıklar önek yapısındadır; “en açık” nokta her boş olmayan açıktaki bulunandır. (çözüm)

K4. `count_topologies_on_n_points(n)` fonksiyonunu $n = 0, 1, 2, 3, 4$ için çalıştırın. T_1 alıştırmasındaki “ $X = \{1, 2, 3\}$ üzerinde 29 topoloji” iddiasını çıktıyla doğrulayın; $n = 4$ değerinin neden $2^{2^4} = 65536$ 'dan çok küçük olduğunu bir cümleyle açıklayın.

İpucu: Dizi 1, 1, 4, 29, 355 (OEIS A000798); aksiyom denetimi adayların ezici çoğunluğunu eler. (çözüm: ek çözümler dosyası)

Teori Alıştırımları

T1. $X = \{1, 2, 3\}$ üzerinde kaç farklı topoloji tanımlanabilir? Bu sayıyı T1–T3 aksiyomlarını doğrudan kontrol ederek hesaplamak yerine neden baz teoremini kullanmak daha pratiktir?

İpucu: Aday aile sayısı $2^{2^3} = 256$ ’dır; cevap 29. Baz teoremi adayları üretken küçük ailelere indirger. (çözüm)

T2. Ayırık topolojinin her zaman diğer topolojilerden daha ince, indirgenmiş topolojinin ise daha kaba olduğunu kanıtlayın. Kanıt için T1–T3 aksiyomlarından hangisi gereklidir?

İpucu: Ayırıklık için aksiyom gerekmez ($\tau \subseteq \mathcal{P}(X)$ tanım gereğidir); indirgenmişlik için yalnız (T1) yeter. (çözüm)

T3. `excluded_point_topology(3, 0)` uzayının `t0` taşıyıp `not_t1` taşımasını (yani T0 olup T1 olmamasını) kanıtlayın. Hangi nokta çifti hangi yönden ayrılamaz? Bu uzayı Sierpiński uzayıyla karşılaştırın.

İpucu: 0’ı içeren tek açık X ’tir; 0 ile $y \neq 0$ noktasını “0 içeren ama y içermeyen açık” yönünde ayıramazsınız, ters yön ($\{y\}$) ise işe yarar — bu tam olarak T0-ama-T1-değil tanımıdır. (çözüm: ek çözümler dosyası)

Bölüm 5

Yüklemler ve Altküme Operatörleri

5.1 Konu

5.1.1 Altküme Yüklemeleri

(X, τ) bir topolojik uzay ve $A \subseteq X$ olsun.

Yüklem	Tanım
Açık (open)	$A \in \tau$
Kapalı (closed)	$X \setminus A \in \tau$
Clopen	$A \in \tau$ ve $X \setminus A \in \tau$
Yoğun (dense)	Her boş olmayan $U \in \tau$ için $U \cap A \neq \emptyset$
Hiçbir yerde-yoğun	$\text{int}(\overline{A}) = \emptyset$

Tablo 5.1: Altküme yüklemeleri

5.1.2 Altküme Operatörleri

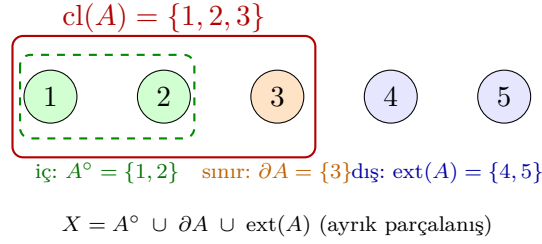
Operatör	Gösterim	Tanım
Kapanış	$\overline{A} = \text{cl}(A)$	A 'yı içeren en küçük kapalı küme
İç	$A^\circ = \text{int}(A)$	A 'ya dahil en büyük açık küme
Sınır	$\partial A = \text{bd}(A)$	$\overline{A} \setminus A^\circ$
Türev kümesi	$A' = d(A)$	A 'nın birikme noktaları kümesi

Tablo 5.2: Altküme operatörleri

Birikme noktası: $x \in A' \iff$ her $U \ni x$ açık için $U \cap (A \setminus \{x\}) \neq \emptyset$.

Sezgi

Bir A kümesini, X üzerinde duran bir “boya lekesi” gibi düşünün. **İç** A° , lekenin tamamen içine sığabildiğiniz açık bir komşuluğunuzun olduğu noktalardır — “rahatça içerideyim” bölgesi. **Kapanış** $\text{cl}(A)$, lekeye dokunan her noktayı ekler: leke artı onun “gölgesi”. **Sınır** ∂A ise tam kenardır: her açık komşuluğu hem A 'dan hem $X \setminus A$ 'dan kesen noktalar. Bu üçlü, X 'i ayrık üç parçaya böler: iç, sınır ve **dış** $\text{ext}(A) = (X \setminus A)^\circ$.



Şekil 5.1: $A = \{1, 2, 3\}$ için iç $A^\circ = \{1, 2\}$ (yeşil), sınır $\partial A = \{3\}$ (turuncu) ve dış $ext(A) = \{4, 5\}$ (mavi); kapanış $cl(A)$ kırmızı çerçevedir. X bu üç parçaya ayrık olarak bölünür.

5.1.3 Komşuluk Sistemi

Tanım 5.1 (Komşuluk Sistemi). $x \in X$ için:

$$N(x) = \{N \subseteq X : \exists U \in \tau, x \in U \subseteq N\}$$

Komşuluk Aksiyomları (N1–N4):

- (N1) $x \in N$, her $N \in N(x)$ için
- (N2) $X \in N(x)$
- (N3) $N_1, N_2 \in N(x) \Rightarrow N_1 \cap N_2 \in N(x)$
- (N4) $N \in N(x), N \subseteq M \Rightarrow M \in N(x)$

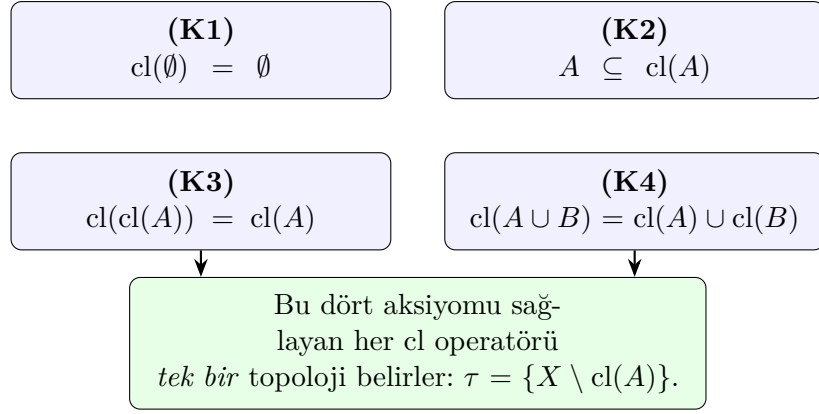
5.2 Teoremler

Teorem 5.2 (Kuratowski Kapanış Aksiyomları). $cl : \mathcal{P}(X) \rightarrow \mathcal{P}(X)$ operatörü şu dört özelliği sağlar:

- (K1) $cl(\emptyset) = \emptyset$
- (K2) $A \subseteq cl(A)$
- (K3) $cl(cl(A)) = cl(A)$
- (K4) $cl(A \cup B) = cl(A) \cup cl(B)$

Bu dört özelliği sağlayan her operatör bir topoloji tanımlar.

İspat eskizi. Kapanış, A 'yı içeren tüm kapalı kümelerin kesişimidir: $cl(A) = \bigcap \{C : C \text{ kapalı}, A \subseteq C\}$. **(K1)** $\emptyset$ zaten kapalı, en küçük kapalı üst-küme kendisidir. **(K2)** her $C \supseteq A$ içerdiğinden $A \subseteq cl(A)$. **(K3)** $cl(A)$ kapalıdır; kapalı bir kümenin kapanışı kendisidir (idempotentlik). **(K4)** $\subseteq$ yönü monotonluktan, $\supseteq$ yönü $cl(A) \cup cl(B)$ 'nin $A \cup B$ 'yi içeren kapalı bir küme olmasından gelir. Sonlu uzaylarda dördü de `kuratowski_closure_check` ile doğrulanır (Örnek 2). $\square$



Şekil 5.2: Kuratowski aksiyomları K1–K4 ve sonucu: bu dördünü sağlayan her cl operatörü tek bir topoloji belirler.

Teorem 5.3 (İç-Kapanış Dualitesi). *Her $A \subseteq X$ için:*

$$A^\circ = X \setminus \text{cl}(X \setminus A), \quad \text{cl}(A) = X \setminus \text{int}(X \setminus A).$$

İspat eskizi. A° tanım gereği A 'ya dahil en büyük açık kümedir; $\text{cl}(B)$ ise B 'yi içeren en küçük kapalı kümedir. Tümleme alma açık $\leftrightarrow$ kapalı ve “en büyük” $\leftrightarrow$ “en küçük” ile $\subseteq$ ilişkisini ters çevirir. $B = X \setminus A$ alın: $X \setminus A$ 'yı içeren en küçük kapalı kümenin tümlenyeni, A 'ya dahil en büyük açık kümedir; yani $A^\circ = X \setminus \text{cl}(X \setminus A)$. İkinci eşitlik $A \mapsto X \setminus A$ ikamesiyle simetrik olarak çıkar. $\square$

Teorem 5.4 (Komşuluk Sistemi Round-Trip). *$N1$ – $N4$ 'ü sağlayan her $\{N(x)\}_{x \in X}$ ailesi, $\tau = \{U : \forall x \in U, U \in N(x)\}$ şeklinde tek bir topolojiyi geri üretir.*

İspat eskizi. İleri yön: τ 'dan $N(x) = \{N : \exists U \in \tau, x \in U \subseteq N\}$ tanımlayın. (N1) $x \in U \subseteq N$ olduğundan $x \in N$. (N2) $X \in \tau$ her noktayı içerir. (N3) $U_1 \cap U_2 \in \tau$ açık olduğundan kesişim de komşuluktur. (N4) üst-küme almak tanımı bozmaz. Geri yön: $\tau = \{U : \forall x \in U, U \in N(x)\}$ ailesinin topoloji aksiyomlarını sağladığı ve başlangıç sistemini geri verdiği doğrulanır; teklik bu eşleşmenin tersinir olmasından gelir. $\square$

5.3 Algoritmalar

Algorithm 6 Sonlu Kapanış Hesabı

Require: $A \subseteq X, \tau$

Ensure: $\text{cl}(A)$

```

1:  $closed \leftarrow \{X \setminus U : U \in \tau\}$ 
2:  $result \leftarrow X$ 
3: for each  $C \in closed$  do
4:   if  $A \subseteq C$  and  $|C| < |result|$  then
5:      $result \leftarrow C$ 
6:   end if
7: end for return  $result$ 
```

Karmaşıklık: $O(|\tau| \cdot |X|)$.

Karmaşıklık: $O(|X| \cdot |\tau|)$.

Algorithm 7 Türev Kümesi Hesabı**Require:** $A \subseteq X, \tau$ **Ensure:** $d(A)$

```

1:  $acc \leftarrow \emptyset$ 
2: for each  $x \in X$  do
3:   if her açık  $U \ni x$  için  $U \cap (A \setminus \{x\}) \neq \emptyset$  then
4:      $acc.add(x)$ 
5:   end if
6: end for return  $acc$ 

```

5.4 pytop API

Listing 5.1: Subset operatörleri importları

```

1 from pytop import (
2     analyze_predicate,
3     closure_of_subset, interior_of_subset, boundary_of_subset,
4     derived_set_of_subset,
5     is_open_subset, is_closed_subset, is_dense_subset,
6     is_nowhere_dense_subset,
7     neighborhood_system, character_at_point,
8     analyze_neighborhood_system,
9 )

```

Fonksiyon	İmza	Açıklama
analyze_predicate	(space, predicate, subset)	Yüklem sorgula
closure_of_subset	(space, subset)	Kapanış
interior_of_subset	(space, subset)	İç
boundary_of_subset	(space, subset)	Sınır
derived_set_of_subset	(space, subset)	Türev kümesi
neighborhood_system	(carrier, topology, point)	$N(x)$
character_at_point	(carrier, topology, point)	$\chi(x)$
analyze_neighborhood_system	(carrier, topology, point =None)	Tam analiz

Tablo 5.3: Altküme operatör fonksiyonları

Not: neighborhood_system ve ilgili fonksiyonlar carrier (liste) ve topology (liste) alır. Space nesnesi için: list(space.carrier), list(space.topology).

5.5 Örnekler

5.5.1 Örnek 1: Kapanış ve İç — Sierpiński Uzayı

```

1 from pytop import sierpinski_space, closure_of_subset,
2     interior_of_subset, boundary_of_subset
3 s = sierpinski_space()

```

```

4 print("cl({0}) =", closure_of_subset(s, {0}).value)
5 print("cl({1}) =", closure_of_subset(s, {1}).value)
6 print("int({0}) =", interior_of_subset(s, {0}).value)
7 print("int({1}) =", interior_of_subset(s, {1}).value)
8 print("bd({1}) =", boundary_of_subset(s, {1}).value)

```

Listing 5.2: Çıktı

```

1 cl({0}) = frozenset({0})
2 cl({1}) = frozenset({0, 1})
3 int({0}) = frozenset()
4 int({1}) = frozenset({1})
5 bd({1}) = frozenset({0})

```

$\{0\}$ 'ın kapanışı kendisidir: $\{0\} = X \setminus \{1\}$ zaten kapalıdır. $\{1\}$ 'in kapanışı X 'tir: $\{1\}$ kapalı değildir.

5.5.2 Örnek 2: Türev Kümesi

```

1 from pytop import sierpinski_space, derived_set_of_subset,
  is_nowhere_dense_subset
2
3 s = sierpinski_space()
4 print("d({0}) =", derived_set_of_subset(s, {0}).value)
5 print("d({1}) =", derived_set_of_subset(s, {1}).value)
6 print("{0} nowhere dense? =", is_nowhere_dense_subset(s, {0}).status)

```

Listing 5.3: Çıktı

```

1 d({0}) = frozenset()
2 d({1}) = frozenset({0})
3 {0} nowhere dense? = true

```

$d(\{1\}) = \{0\}$: 0'ı içeren her $\{0, 1\}$, $\{1\} \setminus \{0\} = \{1\}$ ile kesişir.

5.5.3 Örnek 3: Komşuluk Sistemi

```

1 from pytop import sierpinski_space, neighborhood_system,
  character_at_point
2
3 s = sierpinski_space()
4 carrier = list(s.carrier)
5 topology = list(s.topology)
6 print("N(0) =", neighborhood_system(carrier, topology, 0).value)
7 print("N(1) =", neighborhood_system(carrier, topology, 1).value)
8 print("chi(0) =", character_at_point(carrier, topology, 0).value)
9 print("chi(1) =", character_at_point(carrier, topology, 1).value)

```

Listing 5.4: Çıktı

```

1 N(0) = [['0', '1']]
2 N(1) = [['1'], ['0', '1']]

```

```

3 chi(0) = 1
4 chi(1) = 2

```

$N(0) = \{X\}$: 0 yalnızca X 'te açık. $N(1) = \{\{1\}, X\}$: yerel baz büyüklükleri $\chi(0) = 1$, $\chi(1) = 2$.

5.5.4 Örnek 4: İç / Kapanış / Sınır / Dış Parçalanışı

```

1 from pytop import (make_topology, interior_of_subset,
2                     closure_of_subset,
3                     boundary_of_subset, exterior_of_subset)
4
5 sp = make_topology({1, 2, 3, 4, 5}, {1, 2}, {4, 5}, {1, 2, 4, 5})
6 A = {1, 2, 3}
7 print("int(A) =", interior_of_subset(sp, A).value)
8 print("cl(A)   =", closure_of_subset(sp, A).value)
9 print("bd(A)   =", boundary_of_subset(sp, A).value)
10 print("ext(A)  =", exterior_of_subset(sp, A).value)

```

Listing 5.5: Çıktı

```

1 int(A) = frozenset({1, 2})
2 cl(A)  = frozenset({1, 2, 3})
3 bd(A)  = frozenset({3})
4 ext(A) = frozenset({4, 5})

```

İç $\{1, 2\}$, sınır $\{3\}$ ve dış $\{4, 5\}$ birlikte X 'i ayrık olarak kaplar: $X = A^\circ \cup \partial A \cup \text{ext}(A)$ (Şekil 5.1).

5.5.5 Örnek 5: Kuratowski Aksiyomlarının Doğrulanması

```

1 from pytop import finite_chain_space, kuratowski_closure_check
2
3 s = finite_chain_space(3)
4 report = kuratowski_closure_check(list(s.carrier), list(s.topology))
5 for k in sorted(report):
6     print(f"{k:<13}= {report[k]}")

```

Listing 5.6: Çıktı

```

1 all           = True
2 empty        = True
3 extensive    = True
4 finite_union = True
5 idempotent   = True

```

$\text{empty} \rightarrow (K1)$, $\text{extensive} \rightarrow (K2)$, $\text{idempotent} \rightarrow (K3)$, $\text{finite_union} \rightarrow (K4)$; all hepsinin sağlandığını özetler (Şekil 5.2).

5.5.6 Örnek 6: Yoğun Nokta ve Birikme Kümesi

```

1 from pytop import finite_chain_space, closure_of_subset,
  is_dense_subset, derived_set_of_subset
2
3 c = finite_chain_space(4)
4 print("cl({1})      =", closure_of_subset(c, {1}).value)
5 print("{1} dense? =", is_dense_subset(c, {1}).status)
6 print("d({3})      =", derived_set_of_subset(c, {3}).value)
7 print("cl({4})      =", closure_of_subset(c, {4}).value)

```

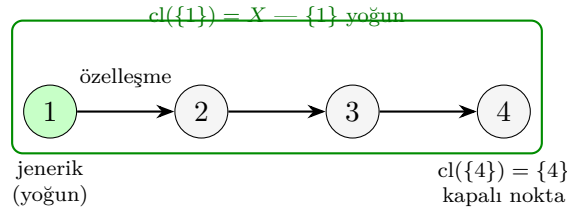
Listing 5.7: Çıktı

```

1 cl({1})      = frozenset({1, 2, 3, 4})
2 {1} dense?   = true
3 d({3})      = frozenset({4})
4 cl({4})      = frozenset({4})

```

1 jeneriktir ($\text{cl}(\{1\}) = X$, yoğun), 4 kapalı bir noktadır ($\text{cl}(\{4\}) = \{4\}$); Şekil 5.3.



Şekil 5.3: `finite_chain_space(4)` zincirinde özelleşme oku: 1 jenerik (yoğun) noktadır, $\text{cl}(\{1\}) = X$; 4 ise kapalı noktadır, $\text{cl}(\{4\}) = \{4\}$.

Karşı-örnek

“İç küme her zaman kapalıdır” yanlışlığı. Örnek 4’te $A^\circ = \{1, 2\}$ açıktır ama **kapalı değildir** ($X \setminus \{1, 2\} = \{3, 4, 5\}$ açık değil). İç ve dış daima açıktır; kapanış ve sınır daima kapalıdır — bu iki aileyi karıştırmak en sık yapılan hatadır. Ayrıca $\partial A = \emptyset$ ancak ve ancak A clopen ise olur: Örnek “Sınır Hesabı”nda açık $\{1\}$ için $\partial\{1\} = \emptyset$, fakat clopen olmayan $\{1, 2\}$ için sınır boş değildir.

5.6 Alıştırmalar

Kodlama Alıştırmaları

- K1.** İndirgenmiş iki-noktalı topolojide her noktanın komşuluk sistemini hesaplayın.
- K2.** $X = \{1, 2, 3, 4\}$, $\tau = \{\emptyset, \{1, 2\}, \{3, 4\}, X\}$ topolojisinde $\text{bd}(\{1, 2, 3\})$ ve $\text{cl}(\{1, 2, 3\})$ hesaplayın.
- K3.** `finite_chain_space(4)` zincirinde $\{1, 2\}$ ’nin iç, kapanış ve sınır kümelerini bulun.

- K4.** `make_topology({1,2,3,4,5}, {1,2}, {4,5}, {1,2,4,5})` uzayında $A = \{1, 2, 3\}$ için `interior_of_subset`, `boundary_of_subset` ve `exterior_of_subset` çalıştırıp $X = A^\circ \cup \partial A \cup \text{ext}(A)$ ayrık parçalanışını doğrulayın.
- K5.** `finite_chain_space(4)` için `kuratowski_closure_check` çağırıp `all` alanının `True` döndüğünü gösterin; ardından $\{1\}$ 'in yoğun olduğunu `is_dense_subset` ile teyit edin.

Teori Alıştırmaları

- T1.** $A^\circ = X \setminus \text{cl}(X \setminus A)$ eşitliğini Kuratowski aksiyomlarını kullanarak kanıtlayın.
- T2.** A 'nın yoğun olması için gerek ve yeter koşulun $\text{cl}(A) = X$ olduğunu gösterin.
- T3.** Her $A \subseteq X$ için $X = A^\circ \cup \partial A \cup \text{ext}(A)$ olduğunu ve üç kümenin ikişer ikişer ayrık olduğunu ispatlayın.
- İpucu:* $\text{ext}(A) = (X \setminus A)^\circ$ ve $\partial A = \text{cl}(A) \setminus A^\circ$.
- T4.** A° kümesinin daima açık olduğunu, fakat genel olarak kapalı **olmadığını** gösteren bir karşı-örnek verin.
- İpucu:* Örnek 4'teki $\{1, 2\}$.

Bölüm 6

Ayrılma Aksiyomları

6.1 Konu

Ayrılma aksiyomları, bir topolojik uzaydaki noktaların ve kapalı kümelerin birbirinden açık kümeler aracılığıyla ne ölçüde “ayrılabilirliğini” ölçer.

Sezgi

Ayrılma aksiyomlarını bir mikroskobun çözünürlük kademeleri gibi düşünün: T0’da iki noktayı *en az bir yönden* ayırt edebilirsiniz; T1’de her iki yönden; T2’de noktaları çakışmayan iki ayrı “görüş alanına” koyabilirsiniz; T3 ve T4’te artık nokta–kapalı küme ve kapalı–kapalı çiftleri bile ayırır. Matematiksel karşılığı: her aksiyom, τ ’nın “ayırma gücü” üzerine gittikçe güçlenen bir varsayımdır ve zincir boyunca her adım bir öncekini gerektirir.

Aksiyom	İsim	Koşul
T0	Kolmogorov	$\forall x \neq y: \exists U \in \tau$ ayıran
T1	Fréchet	$\forall x \neq y$: iki yönlü ayırım
T2	Hausdorff	$\forall x \neq y$: disjoint $U \ni x, V \ni y$
T2.5	Urysohn	disjoint kapanışlı komşuluklar
T3	Regüler	T1 + nokta/kapalı ayırma
T3.5	Tychonoff	T1 + sürekli fonksiyon ayırma
T4	Normal	T1 + disjoint kapalılar ayırma

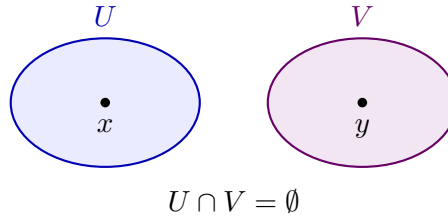
Tablo 6.1: Ayrılma aksiyomları

Sıralama: $T4 \Rightarrow T3.5 \Rightarrow T3 \Rightarrow T2.5 \Rightarrow T2 \Rightarrow T1 \Rightarrow T0$.

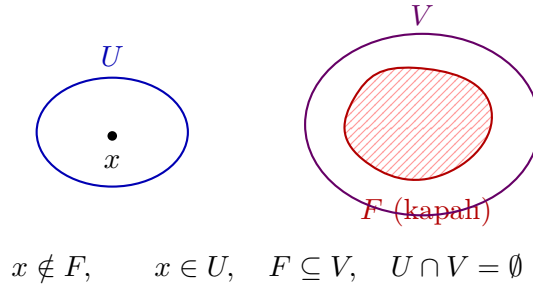
Dikkat – sık hata

T3 ve T4’ün tanımı kaynaktan kaynağa değişir: bazı kitaplar “regüler”i T1 olmadan tanımlar. Bu kılavuzda ve `pytop`’ta tablodaki konvansiyon geçerlidir: **T3 = T1 + regüler**, **T4 = T1 + normal**. Fark gerçektir: iki noktalı indirgenmiş uzay regüler *ve* normaldir (ayrılacak uygun çift yoktur), ama T1 olmadığından T3 de T4 de değildir; Örnekler bölümünde `is_regular/is_t3` ile doğrulanır.

Şekil 6.1 Hausdorff koşulunu, Şekil 6.2 regülerlik koşulunu resmeder.



Şekil 6.1: T2 (Hausdorff): farklı x, y noktaları, kesişmeyen $U \ni x$ ve $V \ni y$ açıklarıyla ayrılır.



Şekil 6.2: Regülerlik: x noktası ile onu içermeyen kapalı F kümesi, ayrık U ve V açıklarına konabilir. T3 bunun üstüne T1 ister.

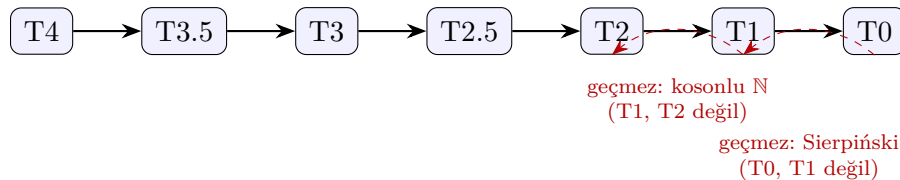
Karşı-örnek

Hiçbir ayrılma aksiyomunu sağlamayan uzay: iki noktalı indirgenmiş uzay. Açıklar yalnız $\emptyset$ ve X olduğundan iki noktayı ayıran *hiçbir* açık yoktur — uzay T0 bile değildir. Örnekler bölümünde `two_point_indiscrete_space()` ile test edilir; zincirin en altının da boş kalabileceğini gösterir.

6.2 Teoremler

Teorem 6.1 (Ayrılma Zinciri). $T_4 \Rightarrow T_{3.5} \Rightarrow T_3 \Rightarrow T_{2.5} \Rightarrow T_2 \Rightarrow T_1 \Rightarrow T_0$. Tersleri genel olarak doğru değildir.

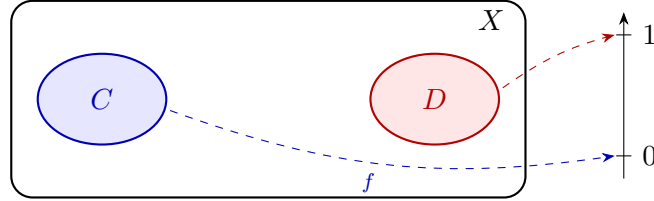
Rehberli Kanıt — model halka: $T_2 \Rightarrow T_1$. $x \neq y$ verilsin. T2 ile ayrık $U \ni x$, $V \ni y$ açıkları vardır. $U \cap V = \emptyset$ olduğundan $y \notin U$ ve $x \notin V$: hem “ x ’i içerip y ’yi dışlayan” hem “ y ’yi içerip x ’i dışlayan” açık bulundu — T1’in iki yönlü koşulu sağlandı. Kalan halkalar (T1 $\Rightarrow$ T0 dahil) aynı şablonla Alıştırma T1’de kanıtlanır. $\square$



Şekil 6.3: Ayrılma zinciri: düz oklar implikasyonları gösterir. Kesikli kırmızı oklar terslerin *geçmediği* iki kanonik karşı-örneği işaretler; her ikisi de bu kılavuzun örnekleridir (Örnek 1 ve Örnek 2).

Teorem 6.2 (Urysohn Fonksiyon Teoremi). X normal ise ve C, D disjoint kapalı kümeler ise, $f : X \rightarrow [0, 1]$ sürekli vardır: $f|_C \equiv 0$, $f|_D \equiv 1$.

İspat eskizi. Normallikle $C \subseteq U_{1/2}$, $\overline{U_{1/2}} \cap D = \emptyset$ olan açık $U_{1/2}$ seç. Aynı adımı tekrarlayıp her ikili kesir $q = k/2^n \in [0, 1]$ için, $p < q \Rightarrow \overline{U_p} \subseteq U_q$ koşulunu sağlayan iç-içe açıklar ailesi $\{U_q\}$ kur (normallik her ekleme adımını mümkün kılar). Sonra $f(x) = \inf\{q : x \in U_q\}$ (hiç içermiyorsa 1) tanımla; iç-içe geçme süreklilik verir, C üzerinde 0, D üzerinde 1 alınır. $\square$



f sürekli; $f|_C \equiv 0$, $f|_D \equiv 1$; ara bölgede 0 ile 1 arası değerler

Şekil 6.4: Urysohn fonksiyonu: normal uzayda ayırık kapalı C ve D için $f|_C \equiv 0$, $f|_D \equiv 1$ olan sürekli $f : X \rightarrow [0, 1]$ vardır; uzay, iki kapalı küme arasında “sürekli geçiş” yapacak kadar zengindir.

Neden önemli?

Urysohn teoremi, küme dilindeki bir ayırma özelliğini (normallik) *sürekli fonksiyon üretimine* çevirir. Tietze Genişleme bunun doğrudan sonucudur ve metrikleştirme teoremlerinin (Bölüm 9 sonrası ufuk) temel aracıdır: fonksiyon üretebilen uzay, metrik benzeri yapılar kurabilen uzaydır.

Teorem 6.3 (Tietze Genişleme). X normal ise her kapalı $A \subseteq X$ üzerinde $f : A \rightarrow [a, b]$ süreklisi tüm X 'e sürekli genişletilebilir.

Teorem 6.4 (Tychonoff Karakterizasyonu). X , T3.5'tir $\iff X$ bir küp $[0, 1]^I$ 'ya homeomorf gömülebilir.

İspat eskizi. $(\Leftarrow)$ $[0, 1]^I$ bir kompakt Hausdorff çarpımdır, dolayısıyla T3.5'tir; T3.5 kalıtsaldır, alt-uzaya geçer. $(\Rightarrow)$ X tam regüler ise $I = C(X, [0, 1])$ (tüm sürekli $[0, 1]$ -değerli fonksiyonlar) indeks kümesini al; değerlendirme gömmesi $e(x) = (f(x))_{f \in I}$ tanımla. Tam regülerlik, e 'nin birebir ve homeomorfik bir gömme olmasını sağlar — her nokta–kapalı çift bir f ile ayrıldığından. $\square$

Teorem 6.5 (Sonlu T1 $\iff$ Ayrık). Sonlu bir uzayda T1 $\iff$ ayrık topoloji.

Rehberli Kanıt. Strateji: T1'den tekilerin kapalılığına, oradan sonlu birleşimle her alt kümenin kapalılığına yürünür.

1. $(\Rightarrow)$ T1 gereği her $y \neq x$ için $y \in U_y$, $x \notin U_y$ olan açık U_y vardır; $X \setminus \{x\} = \bigcup_{y \neq x} U_y$ açıktır, yani $\{x\}$ kapalıdır.
2. Herhangi $A \subseteq X$, sonlu sayıda tekilin birleşimi olarak kapalıdır (kapalıların sonlu birleşimi kapalıdır).
3. Her A kapalı ise her $X \setminus A$ açıktır; A keyfi olduğundan her alt küme açıktır: topoloji ayrıktır.

4. ($\Leftarrow$) Ayrık topolojide her $\{x\}$ açıktır; $x \neq y$ çifti $\{x\}$ ve $\{y\}$ ile iki yönlü ayrılır: T1 (hatta T2).

Sonsuzlukta 2. adım çöker: sonsuz birleşim kapalılığı korumaz — kosonlu $\mathbb{N}$ tam bu nedenle T1 olup ayrık değildir. $\square$

Bu teorem, Bölüm 4 K1 alıştırmasındaki gözlemin — üç noktalı kosonlu uzayın ayrık çıkması — genel açıklamasıdır.

Teorem 6.6 (Kompakt + Hausdorff $\Rightarrow$ T4). *Her kompakt Hausdorff uzay normaldir (T4).*

İspat eskizi. Önce “kompakt Hausdorff $\Rightarrow$ regüler”i kur: $x \notin C$ (kapalı, dolayısıyla kompakt) ise, her $c \in C$ için Hausdorff ayrık $U_c \ni x$, $V_c \ni c$ verir; $\{V_c\}$ ailesi C 'yi örter, kompaktlıkla sonlu alt-örtü $V_{c_1}, \dots, V_{c_n}$ al. $U = \bigcap_i U_{c_i}$ ve $V = \bigcup_i V_{c_i}$ aradığın ayrık açıkları verir. Sonra aynı argümanı bir nokta yerine ikinci bir kapalı (kompakt) D kümesine uygula: her $d \in D$ için regülerlik ayrık $U_d \supseteq C$, $W_d \ni d$ verir; D kompakt olduğundan sonlu alt-örtüyle C ve D ayrık açıklara konur — normallik tam budur. $\square$

Bu, Kompaktlık bölümündeki “kompakt + Hausdorff $\Rightarrow$ T4” teoreminin ayrılma-aksiyomu tarafıdır; kompaktlık, sonsuz Hausdorff ayrımlarını *sonlu* sayıya indirgeyerek normalliği mümkün kılar.

6.3 Algoritmalar

Algorithm 8 Sonlu T0 Karar Prosedürü

```

1: for each pair  $(x, y)$  with  $x \neq y$  do
2:   if  $\nexists U \in \tau: (x \in U \wedge y \notin U) \text{ or } (y \in U \wedge x \notin U)$  then return false
3:   end if
4: end for return true

```

Karmaşıklık T0: $O(|X|^2 \cdot |\tau|)$. **Karmaşıklık T2:** $O(|X|^2 \cdot |\tau|^2)$.

6.3.1 İz Sürme: T0 Prosedürü Sierpiński Üzerinde

$X = \{0, 1\}$, $\tau = \{\emptyset, \{1\}, X\}$:

Çift (x, y)	Denenen U	$x \in U \wedge y \notin U$	$y \in U \wedge x \notin U$	Karar
$(0, 1)$	$\emptyset$	hayır	hayır	devam
$(0, 1)$	$\{1\}$	hayır	evet	çift ayrıldı
—	—	—	—	tüm çiftler bitti $\rightarrow$ true

Tablo 6.2: Tek çift tek açıkla ayrılır; genel sınır $O(|X|^2 \cdot |\tau|)$ 'dur.

6.4 pytop API

Listing 6.1: Ayrılma aksiyom importları

```
1 from pytop import (
2     is_t0, is_t1, is_t2, is_t2_5, is_t3, is_t4,
3     is_hausdorff, is_regular, is_normal, is_perfectly_normal,
4     separation_chain, analyze_separation,
5 )
```

Fonksiyon	İmza	Açıklama
is_t0 ...is_t4	(space)	Aksiyom kontrolü
separation_chain	(space)	Tüm zincir: dict[str, Result]
analyze_separation	(space, property = 'hausdorff')	Tek aksiyom sorgu

Tablo 6.3: Ayrılma fonksiyonları

Neden önemli?

is_* yüklemeleri ham bool değil, .status alanı true / false / unknown olabilen bir Result döndürür. Üçüncü değer dürüstlüktür: örneğin T3.5 (Tychonoff) sürekli fonksiyonlarla tanımlanır ve sonlu açık-küme taramasıyla karar verilemez; separation_chain bu durumda tychonoff: unknown raporlar. Sembolik uzaylarda da bilinmeyen özellikler unknown kalır — kütüphane bilmediğini *bilmediğini söyleyerek* belirtir.

6.5 Örnekler

6.5.1 Örnek 1: Sierpiński — T0 ✓, T1 ×

```
1 from pytop import sierpinski_space, is_t0, is_t1, is_t2
2 s = sierpinski_space()
3 print("T0:", is_t0(s).status)
4 print("T1:", is_t1(s).status)
5 print("T2:", is_t2(s).status)
```

Listing 6.2: Çıktı

```
1 T0: true
2 T1: false
3 T2: false
```

Ne oldu? T0: true — (0,1) çifti için {1} açığı 1'i içerir, 0'ı dışlar; tek çift bu olduğundan T0 tamam. T1: false — ters yön yok: 0'ı içerip 1'i dışlayan açık küme yoktur ($\{0\} \notin \tau$; bkz. Bölüm 4, Örnek 1). T1 düşünce T2 de düşer. Sierpiński, zincirin “T0’da takılan” kanonik örneğidir (Şekil 6.3).

6.5.2 Örnek 2: Kosonlu $\mathbb{N}$ — $T_1 \checkmark$, $T_2 \times$

```

1 from pytop import naturals_cofinite, is_t1, is_t2
2 nc = naturals_cofinite()
3 print("T1:", is_t1(nc).status)
4 print("T2:", is_t2(nc).status)

```

Listing 6.3: Çıktı

```

1 T1: true
2 T2: false

```

Ne oldu? $T_1: \text{true}$ — her n için $\mathbb{N} \setminus \{n\}$ kosonludur, dolayısıyla açıktır; bu, her noktayı diğerinden iki yönlü ayırır. $T_2: \text{false}$ — boş olmayan iki kosonlu açık U, V daima kesişir: $\mathbb{N} \setminus (U \cap V) = (\mathbb{N} \setminus U) \cup (\mathbb{N} \setminus V)$ iki sonlu kümenin birleşimi olarak sonludur; $\mathbb{N}$ sonsuz olduğundan $U \cap V \neq \emptyset$. Bu örnek, Bölüm 4 K1’in “neden sonsuz taşıyıcı gerekir” sorusunun cevabıdır.

6.5.3 Örnek 3: Ayrılma Zinciri — Ayrık

```

1 from pytop import discrete_topology, separation_chain
2 d = discrete_topology(1, 2, 3)
3 for prop, result in separation_chain(d).items():
4     print(f" {prop:20s}: {result.status}")

```

Listing 6.4: Çıktı

```

t0                : true
t1                : true
hausdorff         : true
urysohn           : true
t3                : true
tychonoff         : true
t4                : true
completely_normal : true
perfectly_normal  : true

```

Ne oldu? Ayrık uzayda her tekil açık olduğundan her ayırma görevi tekil komşuluklarla çözülür; dokuz yüklemün tümü true döner. tychonoff bile true ’dur: ayrık uzayda her fonksiyon süreklidir, ayırıcı fonksiyon doğrudan yazılır. separation_chain ’in anah-tar sırası zincirin mantıksal sırasıdır — bir uzayın “nerede takıldığını” yukarıdan aşağı okuyabilirsiniz (krş. Alıştırma K2).

6.5.4 Örnek 4: Regüler Ama T_3 Değil — Konvansiyon Testi

```

1 from pytop import two_point_indiscrete_space, is_regular, is_normal,
  is_t3, is_t4
2

```

```

3 tp = two_point_indiscrete_space()
4 print("regular:", is_regular(tp).status, "| normal:", is_normal(tp).
    status)
5 print("t3      :", is_t3(tp).status, "| t4      :", is_t4(tp).status)

```

Listing 6.5: Çıktı

```

regular: true | normal: true
t3      : false | t4      : false

```

Ne oldu? İndirgenmiş iki noktalı uzayda kapalı kümeler yalnız $\emptyset$ ve X 'tir; “nokta–kapalı” ve “kapalı–kapalı” ayırma koşullarının öncülü hiç gerçekleşmez, koşullar boş yere sağlanır: **regular** ve **normal** **true**. Ama uzay **T1** olmadığından (tekiler kapalı değil) **t3** ve **t4** **false** kahr — “Dikkat” kutusundaki konvansiyonun davranışsal kanıtı.

6.5.5 Örnek 5: Dışlanan-Nokta Topolojisi — İkinci Bir “T0’da Takılan”

Sierpiński, T0-olup-T1-olmayan tek örnek değildir. $X = \{0, 1, 2\}$ üzerinde **dışlanan-nokta topolojisi** (0 dışlanır): bir küme ya 0’ı içermez ya da X ’in kendisidir.

```

1 from pytop import excluded_point_topology, is_t0, is_t1,
    separation_chain
2
3 ep = excluded_point_topology(3, 0) # X = {0,1,2}, dislanan nokta 0
4 print("T0:", is_t0(ep).status, "| T1:", is_t1(ep).status)
5 for prop, result in separation_chain(ep).items():
6     print(f" {prop:20s}: {result.status}")

```

Listing 6.6: Çıktı

```

T0: true | T1: false
  t0              : true
  t1              : false
  hausdorff       : false
  urysohn         : false
  t3              : false
  tychonoff       : false
  t4              : false
  completely_normal : false
  perfectly_normal  : false

```

Ne oldu? **T0: true** — 1 ve 2, 0-içermeyen $\{1\}/\{2\}$ açıklarıyla iki yönlü ayrılır; 0 ile herhangi bir nokta arasında ise 0’ı dışlayan açık tek yönlü ayırım verir. **T1: false** — 0’ı içeren tek açık X ’tir, dolayısıyla $\{0\}$ kapalı değildir. Sierpiński’den farklı bir mekanizmayla zincir yine T0’da takılır.

6.5.6 Örnek 6: Metrik Uzaylar — Zincirin Tepesine Kadar

Her metrik uzay normaldir (T4) ve zincirin tüm üst basamaklarını sağlar; bunu $\mathbb{R}$ ve $[0, 1]$ üzerinde gözlemleyelim.

```

1 from pytop import real_line_metric, closed_unit_interval_metric
2 from pytop import is_t2, is_urysohn, is_t3, is_t4
3
4 rl = real_line_metric()
5 ui = closed_unit_interval_metric()
6 print("R          t2/urysohn/t3/t4:", is_t2(rl).status, is_urysohn(rl)
7       .status, is_t3(rl).status, is_t4(rl).status)
7 print("[0,1]      t2/urysohn/t3/t4:", is_t2(ui).status, is_urysohn(ui)
       .status, is_t3(ui).status, is_t4(ui).status)

```

Listing 6.7: Çıktı

R	t2/urysohn/t3/t4: true true true true
[0,1]	t2/urysohn/t3/t4: true true true true

Ne oldu? Metrik mesafe kendisi bir ayırıcıdır: ayırık kapalı C, D için $f(x) = d(x, C)/(d(x, C) + d(x, D))$ sürekli Urysohn fonksiyonunu doğrudan verir — yani metrik uzaylar yalnız T2 değil, T4'e kadar her aksiyonu sağlar (Teorem 2.2'nin metrik durumda elle yazılabilen tanığı). Ayrılma zincirinin sonlu/sembolik örneklerle metrik örnekler arasındaki keskin farkı budur.

6.6 Alıştırmalar

Kodlama

K1. `make_topology({1,2,3},{1},{2},{1,2})` için `separation_chain` çalıştırın; her sonucu yorumlayın.

İpucu: Önce açıkları listeleyin: $\emptyset, \{1\}, \{2\}, \{1, 2\}, X$. Her çifti ayıran açık arayın; 3'ü içerep 1'i dışlayan açık var mı? ([çözüm](#))

K2. `finite_chain_space(3)` üzerinde en yüksek sağlanan aksiyomu bulun.

İpucu: Çıktıda `true` olan en güçlü anahtarı arayın; `unknown` değerlerini “Neden önemli?” kutusuna göre yorumlayın. ([çözüm](#))

K3. `two_point_indiscrete_space()` üzerinde T0, T1, T2 test edin.

İpucu: Tek boş olmayan açık X iken herhangi bir çift nasıl ayrılabilir? ([çözüm](#))

K4. `excluded_point_topology(4, 0)` (yani $X = \{0, 1, 2, 3\}$, dışlanan nokta 0) üzerinde `separation_chain` çalıştırın; sağlanan en yüksek aksiyomu belirleyin ve neden T1'in düştüğünü açıklayın.

İpucu: 0'ı içeren tek açık X 'tir; $\{0\}$ kapalı mı? ([çözüm](#))

Teori

T1. $T2 \Rightarrow T1 \Rightarrow T0$ implikasyonlarını kanıtlayın.

İpucu: $T2 \Rightarrow T1$: ayrık $U \ni x, V \ni y$ verildiğinde U, y 'yi; V, x 'i dışlar — iki yönlü ayrım hazır. $T1 \Rightarrow T0$: iki yönlü ayrım tek yönlüyü içerir. (çözüm)

T2. Sonlu uzayda $T1 \iff$ ayrık topoloji olduğunu gösterin.

İpucu: $T1 \Rightarrow$ tekilleri kapalı $\Rightarrow$ (sonlu birleşim) her alt küme kapalı $\Rightarrow$ her alt küme açık. (çözüm)

T3. Her metrik uzayın normal ($T4$) olduğunu kanıtlayın.

İpucu: Ayrık kapalı C, D için $f(x) = d(x, C)/(d(x, C) + d(x, D))$ sürekli Urysohn fonksiyonunu yazın; $f^{-1}([0, \frac{1}{2}))$ ve $f^{-1}((\frac{1}{2}, 1])$ aradığınız ayrık açıklardır. (çözüm)

Bölüm 7

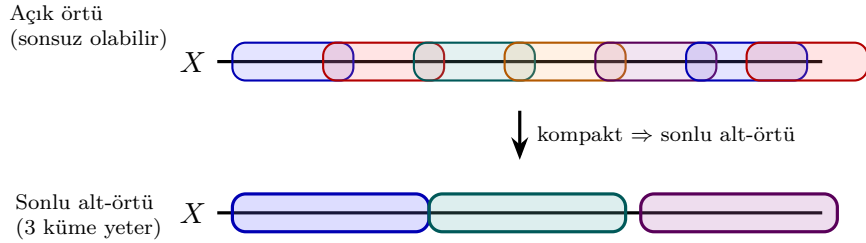
Kompaktlık

Kompaktlık, sonsuz yapılarda “sınırlılık” fikrinin topolojik soyutlamasıdır. Açık örtü tanımını temel alınarak incelenmekte; varyantlar ve lokal kompaktlık da ele alınmaktadır.

7.1 Konu

7.1.1 Sezgisel Giriş — “Gizli Sonluluk”

Kompaktlığı tek cümlede özetlemek gerekirse: *sonsuz bir uzayın, sonlu bir uzay gibi davranmasıdır*. Sonlu bir kümede her şey kolaydır; kompaktlık bu “sonluluk konforunu” sonsuz uzaylara taşıyan köprüdür: uzayı ne kadar çok açık kümeyle örtersek örtelim, **her zaman sonlu sayıda tanesi yeter**.



Şekil 7.1: Açık örtü $\rightarrow$ sonlu alt-örtü: uzayı sonsuz çoklukta açık kümeyle örtsek bile, kompaktsa sonlu sayıda tanesi onu kapatmaya yeter.

Sezgi

“Açık örtü” uzayı kaplayan açık küme ailesidir; “sonlu alt-örtü” bu aileden seçilmiş, hâlâ kaplamayı başaran sonlu bir parçadır. Kompakt olmak, bunu *her* örtü için yapabilmektir. Bu özellik, analizdeki maksimum değer teoremi, düzgün süreklilik ve Heine-Borel’in arkasındaki sessiz kahramandır.

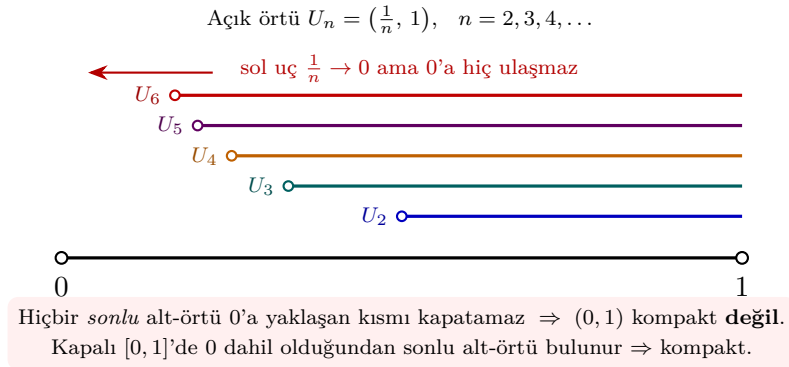
7.1.2 Kompaktlık Tanımı

Tanım 7.1 (Kompakt Uzay). (X, τ) topolojik uzayı **kompakt** ise: Her açık örtü $\{U_\alpha\}$ için sonlu alt-örtü vardır:

$$X \subseteq \bigcup_{\alpha} U_{\alpha} \implies \exists \text{ sonlu } I : X \subseteq \bigcup_{\alpha \in I} U_{\alpha}.$$

7.1.3 Neden “kapalı ve sınırlı” yetmez? — Karşı-örnekler

Reel doğrudaki kompaktlık tam olarak “kapalı + sınırlı”ya denktir (Heine-Borel). Bu denkleğin neden *hem* kapalılığa *hem* sınırlılığa ihtiyaç duyduğunu en iyi açık aralık gösterir.



Şekil 7.2: Kaçan örtü: $(0, 1)$ açık aralığını $U_n = (1/n, 1)$ ailesi örter; sol uçlar 0'a yaklaşır ama 0'a hiç ulaşmaz, bu yüzden hiçbir sonlu alt-örtü işe yaramaz.

Karşı-örnek

$(0, 1)$ **kompakt değil**. $U_n = (\frac{1}{n}, 1)$ ailesi $(0, 1)$ 'i örter. Sonlu bir alt-aile $\{U_{n_1}, \dots, U_{n_k}\}$ seçersek, en büyük indis $N = \max n_i$ için birleşim yalnızca $(\frac{1}{N}, 1)$ olur; 0'a yakın noktalar açıkta kalır. Sonlu alt-örtü yoktur. Kapalı $[0, 1]$ 'de 0 dahil olduğundan sonlu alt-örtü her zaman bulunur.

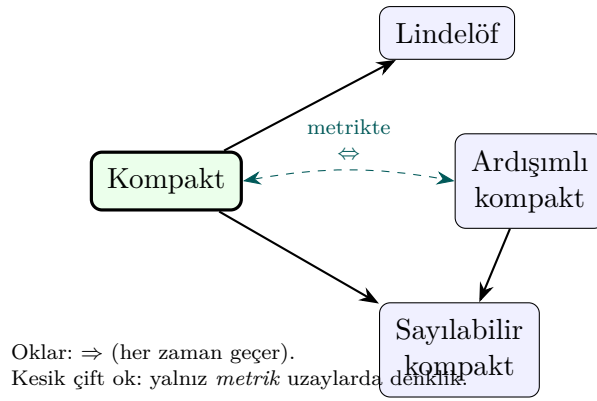
$\mathbb{R}$ **kompakt değil**. $V_n = (-n, n)$ ailesi $\mathbb{R}$ 'yi örter ama hiçbir sonlu alt-aile sınırsız $\mathbb{R}$ 'yi kaplayamaz. Burada sorun *sınırsızlık*.

7.1.4 Kompaktlık Varyantları

Varyant	Tanım
Kompakt	Her açık örtünün sonlu alt-örtüsü
Sayılabilir kompakt	Her sayılabilir örtünün sonlu alt-örtüsü
Ardışıklı kompakt	Her dizi yakınsak alt-dizi içerir
Sözde kompakt	Her sürekli $f : X \rightarrow \mathbb{R}$ sınırlıdır
Lindelöf	Her açık örtünün sayılabilir alt-örtüsü
σ -kompakt	Sayılabilir kompakt kümelerin birleşimi
Lokal kompakt	Her noktanın kompakt komşuluğu var

Tablo 7.1: Kompaktlık varyantları

Her zaman geçen implikasyonlar: Kompakt $\Rightarrow$ Sayılabilir kompakt, Kompakt $\Rightarrow$ Lindelöf, Ardışıklı kompakt $\Rightarrow$ Sayılabilir kompakt. **Metrik uzaylarda** üç kavram (kompakt, sayılabilir kompakt, ardışıklı kompakt) çakışır.



Şekil 7.3: Kompaktlık varyantları arasındaki implikasyonlar: $\text{Kompakt} \Rightarrow \text{Lindelöf}$ ve $\text{Kompakt} \Rightarrow \text{Sayılabilir kompakt}$ her zaman geçer; $\text{Kompakt} \Leftrightarrow \text{Ardışıklı kompakt}$ yalnız metrik uzaylarda.

7.2 Teoremler

Teorem 7.2 (Heine-Borel). $\mathbb{R}^n$ 'de *kompakt* $\Leftrightarrow$ *kapalı ve sınırlı*.

İspat eskizi. ($\Leftarrow$) $[a, b]$ aralığını ele alalım ve sonlu alt-örtüsü olmayan bir açık örtü bulunduğunu varsayalım. Aralığı ikiye böleriz; en az bir yarısının da sonlu alt-örtüsü yoktur. Bunu yineleyerek uzunluğu 0'a giden iç içe kapalı aralıklar dizisi elde ederiz; kesişimleri tek bir p noktasıdır. p 'yi içeren bir örtü elemanı U , yeterince küçük aralıkları tümüyle içerir — çelişki. $\mathbb{R}^n$ için Tychonoff ile $[a, b]^n$ 'e geçilir. ($\Rightarrow$) Kompakt küme Hausdorff uzayda kapalıdır; sınırlılık, $\{(-n, n)^n\}$ örtüsünün sonlu alt-örtüsünden gelir. $\square$

Teorem 7.3. *Kompakt + Hausdorff* $\Rightarrow$ *Normal* (T_4).

İspat eskizi. A, B ayrık kapalı kümeler olsun (kompakt uzayda kapalı $\Rightarrow$ kompakt). Sabit $a \in A$ için Hausdorff'luk her $b \in B$ 'yi a 'dan açık $U_b \ni a$, $V_b \ni b$ ile ayırır. $\{V_b\}$, kompakt B 'yi örter; sonlu alt-örtü $V_{b_1}, \dots, V_{b_k}$ alıp $U_a = \bigcap U_{b_i}$, $V_a = \bigcup V_{b_i}$ koyarız. Burada *sonlu* kesişimin açık kalması kritiktir — bu yüzden B 'nin kompakt olması şarttır. Sonra $\{U_a\}_{a \in A}$ kompakt A 'yı örter; yine sonlu alt-örtüden A ile B 'yi ayıran açıklar kurulur. $\square$

Teorem 7.4. *Kompakt uzaydan Hausdorff uzaya sürekli bijeksiyon* $\Rightarrow$ *homeomorfizma*.

Teorem 7.5 (Tychonoff). *Keyfi kompakt uzayların kartezyen çarpımı kompaktır*.

Teorem 7.6 (Alexandroff Tek-Nokta Kompaktifikasyonu). *Her lokal kompakt Hausdorff uzay X için $X^* = X \cup \{\infty\}$ kompakt Hausdorff'tur. (Örn. $\mathbb{R}^* \cong S^1$, $(\mathbb{R}^2)^* \cong S^2$.)*

Teorem 7.7 (Kompakt görüntü). *$f : X \rightarrow Y$ sürekli ve X kompakt ise $f(X)$ kompaktır*.

İspat eskizi. $\{V_\alpha\}$, $f(X)$ 'i örten açık aile olsun. Süreklilikten $\{f^{-1}(V_\alpha)\}$, X 'i örten açık ailedir; X kompakt olduğundan sonlu alt-örtü $f^{-1}(V_{\alpha_1}), \dots, f^{-1}(V_{\alpha_k})$ vardır. O hâlde $V_{\alpha_1}, \dots, V_{\alpha_k}$, $f(X)$ 'i örter. $\square$

7.3 Algoritmalar

7.3.1 Sonlu Uzayda Kompaktlık

Sonlu uzay her zaman kompaktır: her örtü zaten sonlu. Algoritma: $O(1)$.

7.3.2 analyze_compactness_variants — Tag Tabanlı Çıkarım

Algorithm 9 AnalyzeVariants(X, τ)

```

1: if  $X$  sonlu then
2:   tüm varyantlar  $\leftarrow$  True; return VariantProfile(...)
3: end if
4: if 'compact'  $\in$  tags then
5:   countably_compact, sequentially_compact, lindelof  $\leftarrow$  True
6: end if
7: if 'metric'  $\wedge$  'compact'  $\in$  tags then
8:   sequentially_compact  $\leftarrow$  True
9: end if

```

7.4 pytop API

```

1 from pytop import (
2     is_compact,
3     is_lindelof,
4     is_locally_compact,
5     naturals_cofinite,
6     one_point_compactification_of_reals,
7     one_point_compactification_of_N,
8 )
9 from pytop.compactness_variants import (
10     is_countably_compact,
11     is_sequentially_compact,
12     analyze_compactness_variants,
13 )

```

analyze_compactness_variants(space) $\rightarrow$ Result; .value bir dict içerir.

7.5 Örnekler

Örnek 5.1 — Sonlu Uzaylar: Otomatik Kompakt

```

1 from pytop import sierpinski_space, discrete_topology,
   finite_chain_space, is_compact
2
3 s = sierpinski_space()
4 print("Sierpinski compact?", is_compact(s).status)
5 d = discrete_topology(1, 2, 3)
6 print("Discrete(3) compact?", is_compact(d).status)
7 c = finite_chain_space(3)
8 print("Chain(3) compact?", is_compact(c).status)

```

Sierpinski compact? true

```
Chain(3) compact?      true
Discrete(3) compact?   true
```

Örnek 5.2 — Gerçek Doğru $\mathbb{R}$: Kompakt Değil

```
1 rl = real_line_metric()
2 print("compact?", is_compact(rl).status)
3 print("lindelof?", is_lindelof(rl).status)
4 print("locally_compact?", is_locally_compact(rl).status)
```

```
compact?      false
lindelof?     true
locally_compact? conditional
```

Örnek 5.3 — analyze_compactness_variants: Sierpiński

```
1 r = analyze_compactness_variants(s)
2 for key, val in r.value.items():
3     if hasattr(val, 'status'):
4         print(f" {key:<25}: {val.status}")
```

```
representation      : finite
countably_compact    : true
sequentially_compact : true
pseudocompact        : true
lindelof             : true
```

Örnek 5.4 — Sonsuz Ama Kompakt: Kosonlu Topoloji

Kompakthlık sınırlılığa değil, *örtü davranışına* bağlıdır. Sonsuz bir küme üzerindeki kosonlu topoloji buna örnektir: bir açık örtüde tek bir eleman seçtiğimizde dışarıda yalnız sonlu çoklukta nokta kalır; onları da sonlu sayıda elemanla kapatırız.

```
1 from pytop import naturals_cofinite, is_compact, is_lindelof
2 from pytop.compactness_variants import is_countably_compact
3
4 nc = naturals_cofinite()
5 print("compact?", is_compact(nc).status)
6 print("lindelof?", is_lindelof(nc).status)
7 print("countably?", is_countably_compact(nc).status)
```

```
compact?  true
lindelof? true
countably? true
```

Sonsuz olmasına karşın kompakt — “kompakt = sonlu” sezgisinin neden yalnız *metrik* uzaylarda işlediğini gösterir.

Örnek 5.5 — Tek-Nokta Kompaktifikasyon (Alexandroff)

Lokal kompakt ama kompakt olmayan bir uzaya tek bir ∞ noktası ekleyerek onu kompakt hâle getirebiliriz. $\mathbb{R}$ için sonuç çembere (S^1) denktir.

```

1 from pytop import (
2     one_point_compactification_of_reals,
3     one_point_compactification_of_N,
4     is_compact,
5 )
6
7 r_star = one_point_compactification_of_reals() # R u {inf} ~= S^1
8 n_star = one_point_compactification_of_N()
9 print("R* compact?", is_compact(r_star).status)
10 print("N* compact?", is_compact(n_star).status)

```

R* compact? true

N* compact? true

$\mathbb{R}$ kompakt değildi (Örnek 5.2); tek bir nokta eklemek onu kompakt yaptı.

7.6 Alıştırmalar

Kodlama

- K1. `finite_chain_space(5)` ve `discrete_topology(1,2,3,4,5)` için `is_compact` karşılaştırın.
- K2. `naturals_cofinite()` için `is_compact`, `is_lindelof`, `is_countably_compact` hesaplayın.
- K3. `closed_unit_interval_metric()` ile `real_line_metric()` için `analyze_compactness_variant` çalıştırıp karşılaştırın.
- K4. `real_line_metric()` kompakt değildi. `one_point_compactification_of_reals()` çıktısının `is_compact` durumunu kontrol edip tek noktanın kompaktlığı nasıl kurtardığını açıklayın.

Teori

- T1. Kompakt + sürekli $\Rightarrow$ görüntü kompakt olduğunu ispatlayın.
- T2. Heine-Borel teoreminin neden geçerli olduğunu: $[0, 1]$ neden kompakt, $(0, 1)$ neden kompakt değil? (İpucu: kaçan örtü $U_n = (1/n, 1)$.)
- T3. Kompakt + Hausdorff $\Rightarrow$ Normal ispatında, B 'yi örten $\{V_b\}$ ailesinden sonlu alt-örtü çekme adımının neden kompaktlığa ihtiyaç duyduğunu açıklayın.

Bölüm 8

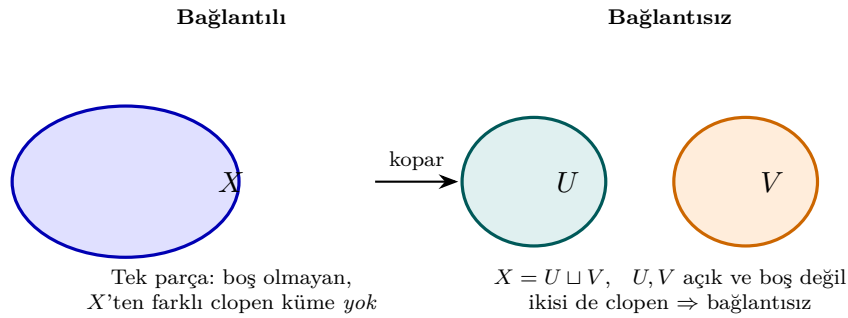
Bağlantılılık

Bağlantılılık, bir topolojik uzayın iki ayrı açık parçaya bölünememesi özelliğidir. Yol-bağlantılılık daha güçlü bir kavramdır ve sürekli yolların varlığına dayanır.

8.1 Konu

8.1.1 Sezgisel Giriş — “Tek Parça mı, Kopuk mu?”

Bağlantılılığı tek cümlede özetlemek gerekirse: *bir uzayın iki ayrı açık parçaya kesilememesidir*. Bir kâğıt parçasını düşünün; onu hiçbir yeri yırtmadan iki ayrı açık parçaya bölemiyorsanız o kâğıt bağlantılıdır. Baştan iki ayrı kâğıdınız varsa uzay bağlantısızdır.



Şekil 8.1: Bağlantılı (tek parça) ile bağlantısız ($X = U \sqcup V$, iki ayrı clopen parça) arasındaki fark.

Bu “kesilememe” fikrinin teknik karşılığı **clopen ayrılma**dır: uzay bağlantısızdır ancak ve ancak boş olmayan, uzayın tamamından farklı, hem açık hem kapalı (*clopen*) bir küme varsa.

Sezgi

“Bağlantılı” uzayın tek parça olmasıdır; “yol-bağlantılı” ise herhangi iki noktayı sürekli bir yol ile birleştirebilmektir. İkinci kavram birincisinden *daha güçlü*: yol kurabiliyorsanız zaten tek parçasınızdır, ama tek parça olmak her zaman yol kurabildiğiniz anlamına gelmez (topolojist sinüs eğrisi).

8.1.2 Bağlantılılık Kavramları

Tanım 8.1 (Bağlantılı Uzay). (X, τ) uzayı **bağlantılı** ise: $X = U \cup V$, $U \cap V = \emptyset$, U, V açık $\Rightarrow U = \emptyset$ veya $V = \emptyset$.

Tanım 8.2 (Yol-Bağlantılı). $\forall x, y \in X: \exists$ sürekli $f : [0, 1] \rightarrow X$, $f(0) = x$, $f(1) = y$.

Kavram	Özellik
Bağlantılı	Clopen bölme yok
Yol-bağlantılı	Her iki nokta arasında sürekli yol var
Ark-bağlantılı	Yol-bağlantılı; yollar enjektif
Lokal bağlantılı	Her noktanın bağlantılı komşuluğu
Tamamen bağlantısız	Her bağlantılı alt küme tek noktadır

Tablo 8.1: Bağlantılılık hiyerarşisi

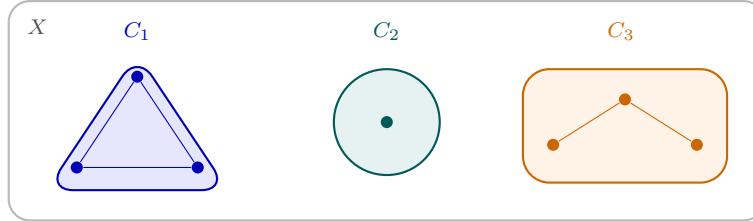
Sıralama: Ark-bağlantılı $\Rightarrow$ Yol-bağlantılı $\Rightarrow$ Bağlantılı.

8.1.3 Clopen Ayrılma

X bağlantısız $\Leftrightarrow$ boş olmayan trivial olmayan bir clopen $A \subsetneq X$ vardır.

8.1.4 Bağlantılı Bileşenler

Bir uzayı, içerme bakımından **en büyük** bağlantılı alt kümelerine ayırabiliriz; bunlara *bağlantılı bileşenler* denir. Bileşenler X 'i örten, ayık parçalardır. Uzay bağlantılıdır $\Leftrightarrow$ tek bir bileşeni vardır.

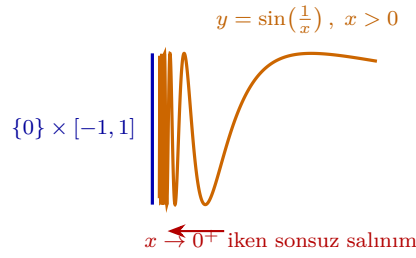


Bağlantılı *bileşenler*: X 'i örten, en büyük bağlantılı, ayık parçalar.
 $X = C_1 \sqcup C_2 \sqcup C_3$ — X bağlantılı $\Leftrightarrow$ tek bileşen ($k = 1$).

Şekil 8.2: Bağlantılı bileşenler: $X = C_1 \sqcup C_2 \sqcup C_3$; her parça en büyük bağlantılı alt kümedir. X bağlantılı $\Leftrightarrow$ tek bileşen.

Karşı-örnek

Topolojist sinüs eğrisi — bağlantılı ama yol-bağlantılı değil. $S = \{(x, \sin \frac{1}{x}) : 0 < x \leq 1\} \cup (\{0\} \times [-1, 1])$ kümesini ele alın. Sağdaki eğri $x \rightarrow 0^+$ giderken sonsuz hızla salınır ve soldaki dikey segmente keyfi yakın olur; bu yüzden S **bağlantılıdır**. Ama dikey segmentteki bir noktayı eğri üzerindeki bir noktaya bağlayan *sürekli* bir yol **yoktur** — yol $x = 0$ 'a yaklaşırken sinüs salınımı yakınsamayı engeller. Demek ki S bağlantılı, fakat yol-bağlantılı değildir.



Bağlantılı ama yol-bağlantılı değil: sağdaki eğri ile soldaki dikey segmenti birleştiren *sürekli* bir yol yoktur (salınım yakınsamayı engeller).

Şekil 8.3: Topolojist sinüs eğrisi: $x \rightarrow 0^+$ iken sonsuz salınım. Eğri ile dikey limit segmenti arasında sürekli yol kurulamaz.

8.2 Teoremler

Teorem 8.3. *Bağlantılı küme süreklilik altında bağlantılıdır: $f : X \rightarrow Y$ sürekli, X bağlantılı $\Rightarrow f(X)$ bağlantılı.*

İspat eskizi. $f(X)$ bağlantısız olsaydı, $f(X) = U \sqcup V$ biçiminde boş olmayan, ayrık, açık iki kümeye ayrılırdı. Süreklilikten $f^{-1}(U)$ ve $f^{-1}(V)$ açıktır; ayrıktır; birleşimleri X 'tir ve ikisi de boş değildir. Bu, X 'in bir clopen ayrılmasıdır — X 'in bağlantılılığıyla çelişir. Demek ki $f(X)$ bağlantılıdır. $\square$

Teorem 8.4. *Yol-bağlantılı $\Rightarrow$ bağlantılı. Tersini genel olarak yanlış. (Karşı örnek: Topolojist sinüs eğrisi.)*

İspat eskizi. X yol-bağlantılı ama bağlantısız olsaydı, $X = U \sqcup V$ clopen ayrılması olurdu. $x \in U$, $y \in V$ alıp $f : [0, 1] \rightarrow X$, $f(0) = x$, $f(1) = y$ sürekli yolunu kuralım. $[0, 1]$ bağlantılı olduğundan görüntü $f([0, 1])$ de bağlantılıdır; ama bu görüntü U ve V tarafından gerçek bir clopen ayrılmaya uğrar — çelişki. Ters yön yanlıştır: topolojist sinüs eğrisi bağlantılı, ama yol-bağlantılı değildir. $\square$

Teorem 8.5 (Ara Değer Teoremi). *$f : X \rightarrow \mathbb{R}$ sürekli, X bağlantılı $\Rightarrow f(X)$ bir aralıktır.*

İspat eskizi. $f(X) \subseteq \mathbb{R}$ bağlantılıdır (önceki teorem). $\mathbb{R}$ 'de bağlantılı kümeler tam olarak aralıklardır; demek ki $f(X)$ bir aralıktır. $a, b \in f(X)$ ise aralık özelliğinden $[a, b] \subseteq f(X)$; yani her $c \in [a, b]$ için $f^{-1}(c) \neq \emptyset$. $\square$

Teorem 8.6. *Gerçek doğru $\mathbb{R}$ 'de bağlantılı alt kümeler tam olarak aralıklardır.*

İspat eskizi. (Aralık $\Rightarrow$ bağlantılı) Bir I aralığı $U \sqcup V$ clopen ayrılmasına uğrasaydı, $a \in U$, $b \in V$ ($a < b$) alıp $c = \sup\{x \in U : x \leq b\}$ tanımlanır; c 'nin hangi parçada olduğu hem açıklık hem kapalılıkla çelişir. (Bağlantılı $\Rightarrow$ aralık) S aralık değilse $a < c < b$, $a, b \in S$, $c \notin S$ olan bir c vardır; o zaman $(-\infty, c) \cap S$ ve $(c, \infty) \cap S$ bir clopen ayrılma verir. $\square$

8.3 Algoritmalar

8.3.1 Sonlu Bağlantılılık — $O(|\tau|^2)$

Karmaşıklık: $O(|\tau|^2)$ — clopen küme taraması.

Algorithm 10 $\text{BagliMi}(X, \tau)$

```

1: for each non-empty  $A \subsetneq X$  do
2:   if  $A \in \tau$  and  $X \setminus A \in \tau$  then
3:     return False
4:   end if
5: end for
6: return True

```

▷ Clopen bölme bulundu

8.4 pytop API

```

1 from pytop import (
2     is_connected,
3     is_path_connected,
4     is_locally_connected,
5     is_totally_disconnected,
6 )

```

8.5 Örnekler

Örnek 5.1 — Sierpiński: Bağlantılı

```

1 s = sierpinski_space()
2 print("connected?", is_connected(s).status)
3 print("path_connected?", is_path_connected(s).status)

```

```

connected?      true
path_connected? unknown

```

Örnek 5.2 — Ayırık Topoloji: Tamamen Bağlantısız

```

1 d = discrete_topology(1, 2, 3)
2 print("connected?", is_connected(d).status)
3 print("totally_disconn?", is_totally_disconnected(d).status)

```

```

connected?      false
totally_disconn? true

```

Örnek 5.3 — Gerçek Doğru ve $[0,1]$

```

1 rl = real_line_metric()
2 ui = closed_unit_interval_metric()
3 print("R connected?", is_connected(rl).status)
4 print("[0,1] path_connected?", is_path_connected(ui).status)

```

```

R connected?      true
[0,1] path_connected? true

```

Örnek 5.4 — Ark / Yol / Bağlantı Hiyerarşisi

Hiyerarşi **Ark** $\Rightarrow$ **Yol** $\Rightarrow$ **Bağlantılı** idi. pytop bu üç düzeyi ayrı ayrı sorgular; bazı uzaylarda bir düzey `true`, diğeri `unknown` döner — pytop yalnızca *kanıtlayabildiğini* bildirir.

```
1 from pytop import (indiscrete_topology, real_line_metric,
2                     discrete_topology,
3                     is_arc_connected, is_path_connected, is_connected)
4 for name, sp in [("Indiscrete(2)", indiscrete_topology(1, 2)),
5                  ("R", real_line_metric()),
6                  ("Discrete(3)", discrete_topology(1, 2, 3))]:
7     print(f"{name:14s} connected={is_connected(sp).status:6s} "
8           f"path={is_path_connected(sp).status:8s} arc={
9               is_arc_connected(sp).status}")
```

Indiscrete(2)	connected =true	path =unknown	arc =true
R	connected =true	path =true	arc =unknown
Discrete(3)	connected =false	path =unknown	arc =false

Örnek 5.5 — Tek Çağrıyla Çoklu Sorgu: `analyze_connectedness`

`analyze_connectedness(space, property_name)` tek bir dağıtıcı sunar; geçerli adlar: `connected`, `path_connected`, `locally_connected`, `totally_disconnected`, `arc_connected`.

```
1 from pytop import analyze_connectedness, sierpinski_space
2
3 s = sierpinski_space()
4 for prop in ["connected", "path_connected", "locally_connected",
5              "totally_disconnected", "arc_connected"]:
6     print(f" {prop:22s}: {analyze_connectedness(s, prop).status}")
```

connected	: true
path_connected	: unknown
locally_connected	: unknown
totally_disconnected	: false
arc_connected	: false

Sierpiński bağlantılıdır ve tamamen bağlantısız *değildir* — tutarlı, çünkü (tek noktadan büyük) bağlantılı bir uzay tamamen bağlantısız olamaz.

Örnek 5.6 — Clopen Ayrılmayı Elle Kurmak

$X = \{1, 2, 3, 4\}$ üzerinde $U = \{1, 2\}$, $V = \{3, 4\}$ taban alınırsa ikisi de clopen olur; $X = U \sqcup V$ bir ayrılımadır. İç içe açıklar ise bölme vermez.

```
1 from pytop import make_topology, is_connected,
2                     is_totally_disconnected
3
4 X = make_topology({1, 2, 3, 4}, {1, 2}, {3, 4}) # clopen bölme
5 print("split connected?", is_connected(X).status)
```

```

5 print("split totally_disconn?", is_totally_disconnected(X).status)
6
7 Y = make_topology({1, 2, 3}, {1}, {1, 2}, {1, 2, 3}) # ic ice, bölme
   yok
8 print("nested connected?", is_connected(Y).status)

```

```

split connected?      false
split totally_disconn? false
nested connected?     true

```

İlk uzay bağlantısız (clopen bölme var) ama *tamamen* bağlantısız değil: $\{1, 2\}$ içinde clopen bölme yoktur. İkinci uzay iç içe açıklardan oluşur; gerçek clopen bölme kurulamaz, bu yüzden bağlantılıdır.

8.6 Alıştırmalar

Kodlama

- K1. `make_topology({1,2,3},{1},{2,3})` için `is_connected` hesaplayın.
 - K2. `finite_chain_space(4)` zincirinin bağlantılı olup olmadığını kontrol edin.
 - K3. İki noktalı ayrık ve indiscrete uzaylar üzerinde `is_connected`, `is_path_connected` karşılaştırın.
 - K4. `indiscrete_topology(1,2)` ve `real_line_metric()` için `is_arc_connected` ve `is_path_connected` çıktılarını karşılaştırın. Hangi uzayda hangi düzey `unknown` döner?
- İpucu: Örnek 5.4.*
- K5. `discrete_topology(1,2,3)` için `analyze_connectedness(d, "connected")` ve `analyze_connectedness(d, "totally_disconnected")` çağırın; iki sonucun neden tutarlı olduğunu açıklayın.
- İpucu: Örnek 5.5.*

Teori

- T1. Bağlantılı + sürekli $\Rightarrow$ görüntü bağlantılı teoremini ispatlayın.
- T2. Yol-bağlantılı $\Rightarrow$ bağlantılı implicasyonunu ispatlayın.
- T3. Topolojist sinüs eğrisinin neden bağlantılı *ama* yol-bağlantılı olmadığını açıklayın: dikey segmentten eğriye sürekli bir yol varsayıp çelişkiye ulaşın.

İpucu: §1 karşı-örnek kutusu + sinüs eğrisi figürü.

Bölüm 9

Sayılabilirlik Aksiyomları

Sayılabilirlik aksiyomları, bir topolojik uzaydaki “bilgi”nin sayılabilir büyüklükte yapılarla tanımlanıp tanımlanamayacağını araştırır.

9.1 Konu

9.1.1 Sayılabilirlik Aksiyomları

Aksiyom	Tanım
Birinci sayılabilir	Her $x \in X$ için sayılabilir yerel baz
İkinci sayılabilir	Tüm topoloji için sayılabilir global baz
Ayrılabilir	Sayılabilir yoğun alt küme: $\overline{D} = X$
Lindelöf	Her açık örtünün sayılabilir alt-örtüsü

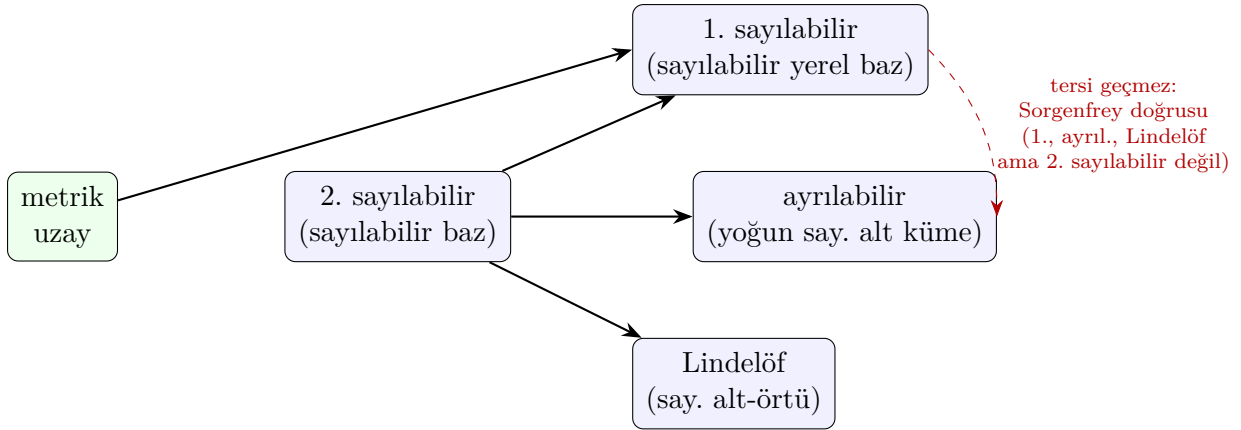
Tablo 9.1: Sayılabilirlik aksiyomları

Sıralamalar:

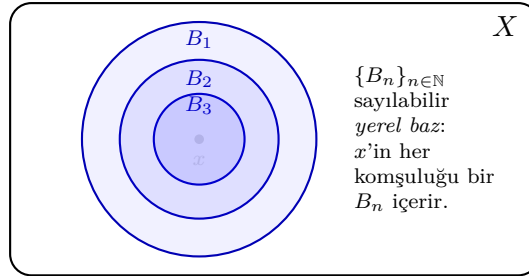
- 2^{nd} sayılabilir $\Rightarrow$ 1^{st} sayılabilir
- 2^{nd} sayılabilir $\Rightarrow$ ayrılabilir $\wedge$ Lindelöf
- Metrik $\Rightarrow$ 1^{st} sayılabilir
- Ayrılabilir metrik $\Rightarrow$ 2^{nd} sayılabilir

Sezgi

Sayılabilirlik aksiyomları, bir uzayı “sayılabilir bir sözlükle” anlatabilme sorusudur. 2^{nd} sayılabilirlik uzayın *tüm* açıklarını sayılabilir bir baz listesinden kurar — tek bir küresel sözlük. 1^{st} sayılabilirlik ise yalnızca *her nokta için ayrı* sayılabilir bir komşuluk listesi ister — yerel sözlükler. Küresel liste her noktaya yerel liste verdiğinden $2^{\text{nd}} \Rightarrow 1^{\text{st}}$; tersi her zaman doğru değildir.



Şekil 9.1: Sayılabilirlik aksiyomları arasındaki implikasyonlar: düz oklar 2nd sayılabilirden 1st sayılabilir, ayrılabilir ve Lindelöf'e gider; metrik uzaylar 1st sayılabilirdir. Kesikli kırmızı ok terslerin geçmediği kanonik karşı-örneği (Sorgenfrey doğrusu) işaretler.



Şekil 9.2: Bir x noktası çevresinde iç içe küçülen $B_1 \supseteq B_2 \supseteq B_3$ komşulukları: sayılabilir bir *yerel baz*. x 'in her komşuluğu bir B_n içerir.

Karşı-örnek

Sorgenfrey doğrusu. $[a, b)$ biçimli yarı-açık aralıklarla üretilen Sorgenfrey doğrusu 1st sayılabilir, ayrılabilir ve Lindelöf'tür ama 2nd sayılabilir *değildir*. Her x için $\{[x, x + 1/n) : n \in \mathbb{N}\}$ sayılabilir yerel baz verir; $\mathbb{Q}$ yoğundur. Ama herhangi bir baz, her x için tam x 'te başlayan bir eleman içermek zorundadır; farklı x 'ler farklı eleman gerektirir, dolayısıyla baz en az continuum büyüklükte — sayılamaz. Yani “2nd $\Leftarrow$ 1st $\wedge$ ayrılabilir $\wedge$ Lindelöf” çıkarımı geçersizdir. `sorgenfrey_line_like()` ile test edilir.

9.2 Teoremler

Teorem 9.1. $2^{\text{nd}} \text{ sayılabilir} \Rightarrow \text{ayrılabilir} \wedge \text{Lindelöf}$.

İspat eskizi. $\mathcal{B} = \{B_n\}$ sayılabilir baz olsun. **Ayrılabilir:** her boş olmayan B_n 'den bir d_n seç; $D = \{d_n\}$ sayılabilirdir ve her boş olmayan açık U bir $B_n \subseteq U$ içerdiğinden $d_n \in U \cap D$ — yani $\bar{D} = X$. **Lindelöf:** keyfi açık örtü $\{U_\alpha\}$ 'da kullanılan her noktayı içeren bir B_n 'i, onu kapsayan bir U_α ile eşle; kullanılan B_n 'ler sayılabilir olduğundan seçilen U_α 'lar sayılabilir bir alt-örtü verir. $\square$

Teorem 9.2. *Metrik uzay $\Rightarrow$ 1st sayılabilir.*

Kanıt İskeleti. Her x için $\{B(x, 1/n) : n \in \mathbb{N}\}$ sayılabilir yerel bazdır. $\square$

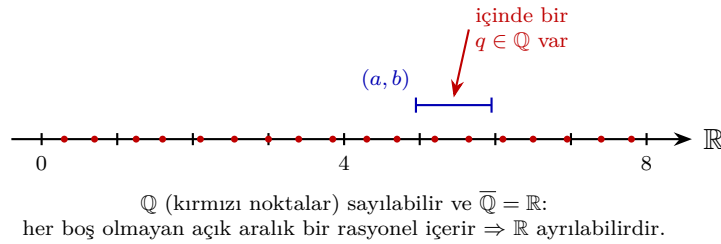
Teorem 9.3. *Ayrılabilir + metrik $\Rightarrow 2^{\text{nd}}$ sayılabilir.*

Kanıt İskeleti. $D \subseteq X$ yoğun sayılabilir; $\{B(d, 1/n) : d \in D, n \in \mathbb{N}\}$ sayılabilir bazdır. $\square$

Teorem 9.4 (Sorgenfrey Karşı Örneği). *Sorgenfrey doğrusu ayrılabilir ve Lindelöf'tür ama 2^{nd} sayılabilir değildir. (Yukarıdaki karşı-örnek kutusuna bakın.)*

Teorem 9.5 (Sayılamaz Ayrık Uzay — İkinci Karşı Örnek). *Sayılamaz taşıyıcı üzerindeki ayrık topoloji 1^{st} sayılabilirdir ama ayrılabilir, Lindelöf ve 2^{nd} sayılabilir değildir.*

İspat eskizi. Her $\{x\}$ açık olduğundan $\{\{x\}\}$ tek elemanlı yerel bazdır (1^{st} sayılabilir). Yoğun bir küme her $\{x\}$ 'i kesmek zorunda olduğundan taşıyıcının tamamını içermelidir; taşıyıcı sayılamaz olduğundan ayrılabilir değildir. $\{\{x\}\}_{x \in X}$ örtüsünün sayılabilir alt-örtüsü yoktur (Lindelöf değil) ve her baz tüm $\{x\}$ 'leri içermek zorunda olduğundan sayılamaz (2^{nd} sayılabilir değil). `uncountable_discrete_space()` ile doğrulanır. $\square$



Şekil 9.3: Ayrılabilirlik: $\mathbb{Q}$ (kırmızı noktalar) sayılabilirdir ve $\overline{\mathbb{Q}} = \mathbb{R}$. Her boş olmayan açık aralık (a, b) bir rasyonel içerdiğinden $\mathbb{R}$ ayrılabilirdir.

9.3 Algoritmalar

9.3.1 Sonlu Uzayda Sayılabilirlik

Sonlu uzayda her aksiom trivially sağlanır. $O(1)$.

9.3.2 Sonsuz Uzayda: Tag-Tabanlı Çıkarım

pytop sembolik uzaylarda tag koridorlarından sayılabilirlik çıkarır:

- `'second_countable' ∈ tags` $\Rightarrow$ tüm aksiyomlar
- `'metric' ∧ 'separable'  $\Rightarrow$  'second_countable'`

9.4 pytop API

```

1 from pytop import (
2     is_first_countable,
3     is_second_countable,
4     is_separable,
5     is_lindelof,
6 )
7 # Yardimcilar:
8 from pytop import countability_report, is_dense_subset

```

`countability_report(space)` dört aksiyomu bir dict içinde toplar; `is_dense_subset(space, subset)` ayrılabilirliği somut bir tanıkla gösterir.

9.5 Örnekler

Örnek 5.1 — Gerçek Doğru: Tüm 4 Aksiyom

```

1 rl = real_line_metric()
2 print("1st countable?", is_first_countable(rl).status)
3 print("2nd countable?", is_second_countable(rl).status)
4 print("separable?", is_separable(rl).status)
5 print("lindelof?", is_lindelof(rl).status)

```

```

1st countable?  true
2nd countable?  true
separable?      true
lindelof?      true

```

Örnek 5.5 — Sorgenfrey Doğrusu: 2nd Countable $\times$

```

1 from pytop import sorgenfrey_line_like
2 sl = sorgenfrey_line_like()
3 print("1st countable?", is_first_countable(sl).status)
4 print("2nd countable?", is_second_countable(sl).status)
5 print("separable?", is_separable(sl).status)

```

```

1st countable?  true
2nd countable?  false
separable?      true

```

Örnek 5.6 — Toplu Rapor + Yoğunluk Tanığı

```

1 from pytop import countability_report, is_dense_subset
2 from pytop import sorgenfrey_line_like, discrete_topology
3
4 rep = countability_report(sorgenfrey_line_like())

```

```

5 for key in ['first_countable', 'second_countable', 'separable', '
    lindelof']:
6     print(f" {key:<17}: {rep[key].status}")
7
8 d = discrete_topology(1, 2, 3)
9 print("dense {1,2,3}?", is_dense_subset(d, {1, 2, 3}).status)
10 print("dense {1,2}? ", is_dense_subset(d, {1, 2}).status)

```

```

first_countable : true
second_countable : false
separable       : true
lindelof        : true
dense {1,2,3}? true
dense {1,2}?    false

```

Örnek 5.7 — Sayılamaz Ayırık Uzay: İkinci Karşı Örnek

```

1 from pytop import uncountable_discrete_space
2 from pytop import (is_first_countable, is_second_countable,
3                     is_separable, is_lindelof)
4
5 u = uncountable_discrete_space()
6 print("1st countable?", is_first_countable(u).status)
7 print("2nd countable?", is_second_countable(u).status)
8 print("separable?     ", is_separable(u).status)
9 print("lindelof?      ", is_lindelof(u).status)

```

```

1st countable? true
2nd countable? false
separable?     false
lindelof?      false

```

Her tekil $\{x\}$ açık olduğundan yerel baz tek elemanlıdır (1^{st} sayılabilir), ama yoğun sayılabilir alt küme ve sayılabilir alt-örtü yoktur.

9.6 Alıştırmalar

Kodlama

- K1. `finite_chain_space(5)` üzerinde `is_first_countable` ve `is_second_countable` hesaplayın.
- K2. İki noktalı ayırık ve indiscrete uzaylar üzerinde `is_separable` karşılaştırın.
- K3. `integers_discrete()` için 2^{nd} sayılabilir mi?
- K4. `uncountable_discrete_space()` üzerinde dört aksiyomu `countability_report` ile hesaplayıp `integers_discrete()` ile karşılaştırın. Hangi aksiyom(lar)da ayrışıyorlar?

İpucu: Fark, taşıyıcının sayılabilir olup olmamasındadır.

Teori

- T1. 2^{nd} sayılabilir $\Rightarrow$ ayrılabilir olduğunu ispatlayın.
- T2. Sorgenfrey doğrusunun 2^{nd} sayılabilir olmadığını gösterin.
- T3. “ 1^{st} sayılabilir $\Rightarrow$ ayrılabilir” çıkarımının yanlış olduğunu bir karşı-örnekle gösterin.

İpucu: Sayılamaz ayrık uzay; yoğun küme her $\{x\}$ ’i kesmek zorundadır.

Bölüm 10

Süreklili Fonksiyonlar ve Homeomorfizmalar

Topolojik uzaylar arasındaki fonksiyonlarda süreklilik, açık kümelerin geri çekiminin açık olması koşuluna dayanır.

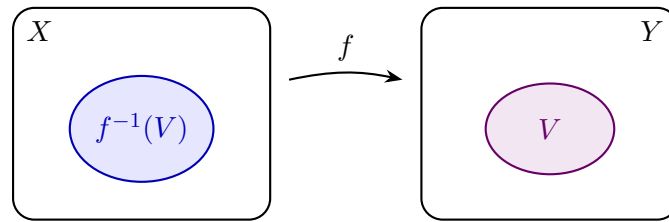
10.1 Konu

Sezgi

Sürekliliği “açıklığın geri taşınması” gibi düşünün: hedef uzayda hangi açık kümeye bakarsanız bakın, onun *geri çekimi* (preimage) kaynak uzayda yine açık olmalı. Analizden bildiğiniz ε - δ tanımı bunun metrik özeli; topolojide ε - δ yerine doğrudan açık kümelerin dilini kullanırız. Homeomorfizma ise bu taşımının *iki yönde* de bozulmadan işlemesi — yani iki uzayın topolojik olarak “aynı” olmasıdır.

Tanım 10.1 (Süreklili Fonksiyon). $f : (X, \tau_X) \rightarrow (Y, \tau_Y)$ fonksiyonu **süreklili** ise:

$$\forall V \in \tau_Y : f^{-1}(V) \in \tau_X.$$

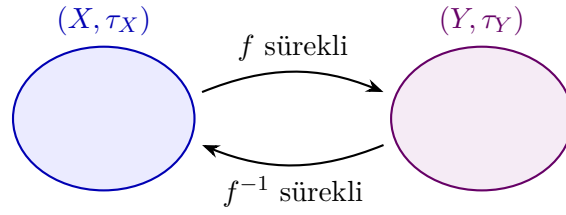


V açık $\Rightarrow f^{-1}(V)$ açık olmalı (her açık V için)

Şekil 10.1: Süreklilik: hedef uzaydaki her açık V için geri çekim $f^{-1}(V)$ kaynak uzayda açık olmalıdır.

Eşdeğer koşullar:

- Her kapalı $F \subseteq Y$ için $f^{-1}(F)$ kapalıdır.
- Her $x \in X$ ve $f(x)$ 'in her komşuluğu W için $f^{-1}(W)$, x 'in komşuluğudur.



bijektif, iki yön de süreklili $\Rightarrow (X, \tau_X) \cong (Y, \tau_Y)$

Şekil 10.2: Homeomorfizma: bijektif bir eşleme hem f hem f^{-1} yönünde sürekliliyse iki uzay topolojik olarak özdeştir, $(X, \tau_X) \cong (Y, \tau_Y)$.

Tanım 10.2 (Homeomorfizma). $f : X \rightarrow Y$ **homeomorfizma** ise: bijektif, f süreklili, f^{-1} süreklili. $(X, \tau_X) \cong (Y, \tau_Y)$ ile gösterilir.

Karşı-örnek

Süreklilik *yöne duyarlıdır*. Aynı taşıyıcı küme $\{0, 1\}$ üzerinde özdeşlik fonksiyonunu düşünün: ayrık topolojiden (ince) Sierpiński'ye (kaba) **süreklili**, ama Sierpiński'den ayrığa **süreklili değil** — çünkü ayrıktaki açık $\{0\}$ 'ın geri çekimi Sierpiński'de açık değildir. Yani “kaba $\rightarrow$ ince” yönünde özdeşlik genelde süreklilik kaybeder (Örnek 5.8'de `is_continuous_finite_map` ile doğrulanır).

Tür	Koşul
Sabit fonksiyon	$f(x) = c$; her zaman süreklili
Özdeşlik	$f(x) = x$; her zaman süreklili
Bileşke	f, g süreklili $\Rightarrow g \circ f$ süreklili
Homeomorfizma	Bijektif; f ve f^{-1} süreklili

Tablo 10.1: Süreklilik hiyerarşisi

10.2 Teoremler

Teorem 10.3. Her sabit fonksiyon süreklidir.

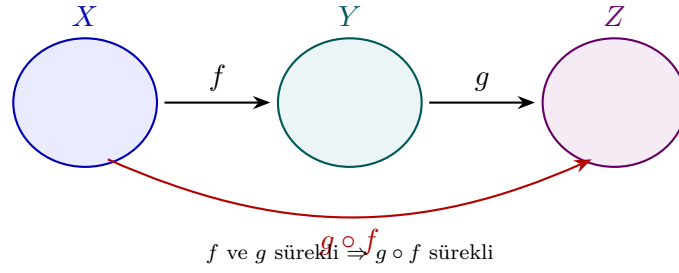
Kanıt İskeleti. $f^{-1}(V) = X$ eğer $c \in V$, $\emptyset$ eğer $c \notin V$; her ikisi de açıktır. □

Teorem 10.4. $f : X \rightarrow Y$ ve $g : Y \rightarrow Z$ süreklili $\Rightarrow g \circ f$ süreklidir.

İspat eskizi. $W \subseteq Z$ açık olsun. g süreklili olduğundan $g^{-1}(W) \subseteq Y$ açıktır. f süreklili olduğundan $f^{-1}(g^{-1}(W)) \subseteq X$ açıktır. $(g \circ f)^{-1}(W) = f^{-1}(g^{-1}(W))$ olduğundan $g \circ f$ 'in her açık kümenin geri çekimi açıktır; yani $g \circ f$ süreklidir. □

Teorem 10.5. $f : X \rightarrow Y$ süreklili, X kompakt $\Rightarrow f(X)$ kompattır.

İspat eskizi. $f(X)$ 'in bir açık örtüsü $\{V_\alpha\}$ verilsin. f süreklili olduğundan her $f^{-1}(V_\alpha)$ X 'te açıktır ve bunlar X 'i örter. X kompakt olduğundan sonlu bir alt-örtü $f^{-1}(V_{\alpha_1}), \dots, f^{-1}(V_{\alpha_n})$ vardır. O zaman $V_{\alpha_1}, \dots, V_{\alpha_n}$ $f(X)$ 'i örter: süreklilik açık örtüyü geri çeker, kompaktlık sonlu alt-örtü verir, görüntüyü geri itilir. Demek ki $f(X)$ kompattır. □



Şekil 10.3: Bileşke süreklilik: f ve g sürekliyse $g \circ f$ de sürekli; geri çekim zincirlenir, $(g \circ f)^{-1}(W) = f^{-1}(g^{-1}(W))$.

Teorem 10.6. $f : X \rightarrow Y$ sürekli, X bağlantılı $\Rightarrow f(X)$ bağlantılıdır.

Teorem 10.7. Kompakt X 'ten Hausdorff Y 'ye sürekli bijeksiyon $\Rightarrow$ homeomorfizma.

10.3 Algoritmalar

10.3.1 is_continuous_finite_map — $O(|\tau_Y| \cdot |X|)$

Algorithm 11 IsContinuous(X, τ_X, Y, τ_Y, f)

```

1: for each  $V \in \tau_Y$  do
2:   preimage  $\leftarrow \{x \in X : f(x) \in V\}$ 
3:   if preimage  $\notin \tau_X$  then
4:     return False
5:   end if
6: end for
7: return True

```

10.4 pytop API

```

1 from pytop import (
2     is_continuous_finite_map,
3     finite_homeomorphism_result,
4     sierpinski_space,
5     discrete_topology,
6     indiscrete_topology,
7     make_topology,
8     make_set,
9     empty_set,
10 )

```

`is_continuous_finite_map(domain, domain_topology, codomain, codomain_topology, mapping) → bool`

`finite_homeomorphism_result(left, right) → Result`

10.5 Örnekler

Örnek 5.2 — Ayırık Topoloji: Her Fonksiyon Sürekli

```

1 d = discrete_topology(0, 1)
2 d_pts = list(d.carrier)
3 d_topo = [set(u) for u in d.topology]
4
5 f_id = {0: 0, 1: 1}
6 f_swap = {0: 1, 1: 0}
7 print("ozdeslik:", is_continuous_finite_map(d_pts, d_topo, d_pts,
8       d_topo, f_id))
9 print("swap:", is_continuous_finite_map(d_pts, d_topo, d_pts,
10      d_topo, f_swap))

```

```

ozdeslik continuous: True
swap          continuous: True

```

Örnek 5.3 — Sierpiński $\rightarrow$ Ayırık: Sürekli Değil

```

1 s = sierpinski_space()
2 s_pts = list(s.carrier)
3 s_topo = [set(u) for u in s.topology]
4 print("id: S->D:", is_continuous_finite_map(s_pts, s_topo, d_pts,
5       d_topo, f_id))
6 print("id: D->S:", is_continuous_finite_map(d_pts, d_topo, s_pts,
7       s_topo, f_id))

```

```

id: Sierpinski -> Disc continuous: False
id: Disc -> Sierpinski continuous: True

```

Sierpiński $\tau = \{\emptyset, \{0, 1\}, \{1\}\}$: $f^{-1}(\{0\}) = \{0\}$ Sierpiński’de açık değil.

Örnek 5.5 — Homeomorfizma Kontrolü

```

1 d2a = discrete_topology(1, 2)
2 d2b = discrete_topology('a', 'b')
3 ind2 = indiscrete_topology(1, 2)
4 print("D(1,2) ~ D(a,b):", finite_homeomorphism_result(d2a, d2b).
5       status)
6 print("D(1,2) ~ S(0,1):", finite_homeomorphism_result(d2a, s).status)
7 print("D(1,2) ~ Ind(1,2):", finite_homeomorphism_result(d2a, ind2).
8       status)

```

```

D(1,2) ~ D(a,b): true
D(1,2) ~ S(0,1): false
D(1,2) ~ Ind(1,2): false

```


Örnek 5.7 — Bileşke Süreklilik: $g \circ f$

```

1 TX = make_topology(make_set(1, 2, 3), make_set(1), make_set(1, 2))
2 TY = make_topology(make_set('a', 'b', 'c'), make_set('a'), make_set('a', 'b'))
3 TZ = make_topology(make_set('p', 'q', 'r'), make_set('p'), make_set('p', 'q'))
4 X_pts, X_topo = list(TX.carrier), list(TX.topology)
5 Y_pts, Y_topo = list(TY.carrier), list(TY.topology)
6 Z_pts, Z_topo = list(TZ.carrier), list(TZ.topology)
7
8 f = {1: 'a', 2: 'b', 3: 'c'}
9 g = {'a': 'p', 'b': 'q', 'c': 'r'}
10 gof = {x: g[f[x]] for x in X_pts}
11 print("f continuous: ", is_continuous_finite_map(X_pts, X_topo,
12           Y_pts, Y_topo, f))
12 print("g continuous: ", is_continuous_finite_map(Y_pts, Y_topo,
13           Z_pts, Z_topo, g))
13 print("g of f continuous:", is_continuous_finite_map(X_pts, X_topo,
14           Z_pts, Z_topo, gof))

```

```

f continuous:      True
g continuous:      True
g of f continuous: True

```

İki sürekli fonksiyonun geri çekimleri zincirlenir: $(g \circ f)^{-1}(W) = f^{-1}(g^{-1}(W))$ (Teorem 2.2'nin sayısal doğrulaması).

Örnek 5.8 — Süreklilik Yöne Duyarlı

```

1 fine = discrete_topology(0, 1)
2 coarse = sierpinski_space()
3 fine_pts, fine_topo = list(fine.carrier), list(fine.topology)
4 co_pts, co_topo = list(coarse.carrier), list(coarse.topology)
5 idmap = {0: 0, 1: 1}
6 print("id: discrete -> sierpinski:", is_continuous_finite_map(
7     fine_pts, fine_topo, co_pts, co_topo, idmap))
7 print("id: sierpinski -> discrete:", is_continuous_finite_map(co_pts,
8     co_topo, fine_pts, fine_topo, idmap))

```

```

id: discrete -> sierpinski: True
id: sierpinski -> discrete: False

```

Ayrıktaki açık $\{0\}$ 'ın geri çekimi yine $\{0\}$ 'dır; bu Sierpiński $\tau = \{\emptyset, \{1\}, \{0, 1\}\}$ içinde açık olmadığından “kaba $\rightarrow$ ince” yönü süreklilik kaybeder.

10.6 Alıştırırmalar

Kodlama

- K1. Özel bir topolojide $f(0) = 0, f(1) = 0, f(2) = 1$ fonksiyonunun sürekli olup olmadığını kontrol edin.
- K2. Sierpiński uzayından kendisine giden dört fonksiyonu test edin.
- K3. `finite_homeomorphism_result` ile iki ayrık uzayın homeomorf olduğunu doğrulayın.
- K4. `make_topology` ile $\{1, 2, 3\}$ üzerinde $\{\alpha\}, \{\alpha, \beta\}$ açıklarıyla zincir topolojileri kurun; zinciri koruyan f ve g tanımlayıp f, g ve $g \circ f$ 'in üçünün de sürekli olduğunu `is_continuous_finite_map` ile doğrulayın.

İpucu: Teorem 2.2'nin sayısal kontrolü.

Teori

- T1. Her sabit fonksiyonun sürekli olduğunu ispatlayın.
- T2. $f \circ g$ bileşkesinin sürekli olduğunu ispatlayın.
- T3. $f : X \rightarrow Y$ sürekli ve X kompakt ise $f(X)$ 'in kompakt olduğunu ispatlayın.

İpucu: $f(X)$ 'in açık örtüsünü f ile geri çekin, X 'in kompaktlığıyla sonlu alt-örtü alın, görüntüye geri itin.

Bölüm 11

Alt Uzay ve Çarpım Topolojisi

Alt uzay topolojisi bir uzayın alt kümesine miras kalan topolojiyi tanımlar. Çarpım topolojisi ise iki uzayı birleştirerek yeni bir uzay kurar.

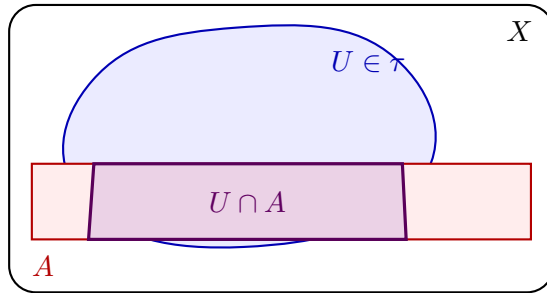
11.1 Konu

Tanım 11.1 (Alt Uzay Topolojisi). (X, τ) uzayı ve $A \subseteq X$ verilsin. **Alt uzay topolojisi:**

$$\tau_A = \{U \cap A : U \in \tau\}.$$

Sezgi

A 'yı X 'ten kesilen bir “pencere” gibi düşünün. A üzerindeki açıkları sıfırdan tanımlamayız; X 'in hazır açıklarını A ile keser, ne kadarı A 'ya düşüyorsa onu açık sayarız. Bu yüzden τ_A , τ 'dan *miras* alınır.



Alt uzay topolojisi:
 $\tau_A = \{U \cap A : U \in \tau\}$.
 A 'nın açıkları, X 'in
açıklarını A ile kesmekle doğar.

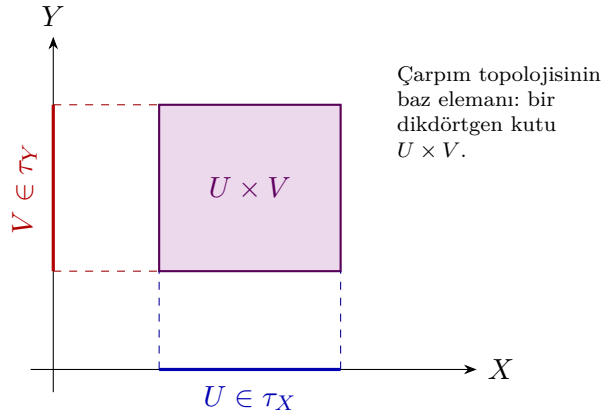
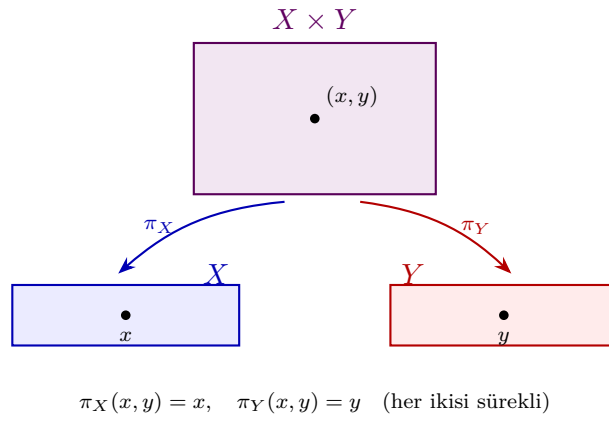
Şekil 11.1: Alt uzay topolojisi: $A \subseteq X$ 'te açıklar $U \cap A$ biçimindedir.

Tanım 11.2 (Çarpım Topolojisi). (X, τ_X) ve (Y, τ_Y) verilsin. Çarpım topolojisi $X \times Y$ üzerinde $\{U \times V : U \in \tau_X, V \in \tau_Y\}$ bazından üretilir.

Projeksiyon $\pi_X : X \times Y \rightarrow X$ ve $\pi_Y : X \times Y \rightarrow Y$ süreklidir.

Sezgi

Çarpım topolojisinin bazı, düzlemdeki açık dikdörtgenlere benzer: bir “yatay” açık U ile bir “dikey” açık V 'yi çarparak $U \times V$ kutusunu kurarız. Bu kutular tüm açıkların *bazıdır* — açıklar bu kutuların birleşimleridir.

Şekil 11.2: Çarpım topolojisinin baz elemanı: $U \times V$ dikdörtgen kutusu.Şekil 11.3: π_X ve π_Y projeksiyonları (x, y) noktasını bileşenlerine düşürür.**Karşı-örnek**

“ $U \times V$ biçiminde olmayan açık yok” demek yanlıştır. Birim diskte $D(0, 1) \times D(0, 1)$ içindeki dairesel bir bölge, hiçbir tek $U \times V$ kutusuna eşit olmasa da açıktır; çünkü açıkler kutuların *birleşimidir*. $\{U \times V\}$ yalnızca **baz**’dır, topolojinin tamamı değil.

Özellik	Alt uzay	Çarpım
Hausdorff	✓	✓
Kompaktlık	Yalnız kapalı alt uzay	✓ (Tychonoff)
Bağılantılılık	Hayır (genel)	✓
T0/T1/T2	✓	✓

Tablo 11.1: Topolojik özelliklerin kalıtımı

11.2 Teoremler

Teorem 11.3. *Hausdorff uzayın her alt uzayı Hausdorff’tur.*

İspat eskizi. $x \neq y$ noktaları $A \subseteq X$ içinde olsun. X Hausdorff olduğundan X 'te ayrık açıklar $U \ni x$, $V \ni y$ vardır. O hâlde $U \cap A$ ve $V \cap A$, A 'da açık, x ile y 'yi ayıran komşuluklardır ve $(U \cap A) \cap (V \cap A) = (U \cap V) \cap A = \emptyset$. $\square$

Teorem 11.4. *Kompakt uzayın kapalı alt uzayı kompaktır.*

İspat eskizi. $A \subseteq X$ kapalı, X kompakt olsun. A 'nın bir açık örtüsü $\{U_\alpha \cap A\}$ verilsin. Her U_α 'yı açık olan $X \setminus A$ ile birlikte alırsak X 'in açık örtüsünü elde ederiz; X kompakt olduğundan sonlu alt-örtü vardır. $X \setminus A$ 'yı atınca A 'yı örten sonlu sayıda $U_\alpha \cap A$ kalır. $\square$

Teorem 11.5. X ve Y bağlantılı $\Rightarrow X \times Y$ bağlantılıdır.

Kanıt İskeleti. $\{x_0\} \times Y$ ve $X \times \{y_0\}$ bağlantılı dilimler; kesişimleri üzerinden birleşerek tüm çarpımı kapsar. $\square$

Teorem 11.6 (Tychonoff, $n = 2$). X ve Y kompakt $\Rightarrow X \times Y$ kompaktır.

Teorem 11.7. X ve Y Hausdorff $\Rightarrow X \times Y$ Hausdorff'tur.

İspat eskizi. $(x_1, y_1) \neq (x_2, y_2)$ ise $x_1 \neq x_2$ veya $y_1 \neq y_2$. Diyelim $x_1 \neq x_2$; X 'te ayrık $U_1 \ni x_1$, $U_2 \ni x_2$ seçilir. O zaman $U_1 \times Y$ ve $U_2 \times Y$ çarpımda açık, ayrık ve iki noktayı ayırır. $\square$

Teorem 11.8 (Evrensel Özellik — Çarpım). $f : Z \rightarrow X \times Y$ sürekli $\Leftrightarrow \pi_X \circ f$ ve $\pi_Y \circ f$ süreklidir.

İspat eskizi. $(\Rightarrow)$ Projeksiyonlar sürekli, sürekli fonksiyonların bileşkesi süreklidir. $(\Leftarrow)$ Bir baz elemanı $U \times V$ 'nin ön-görüntüsü $f^{-1}(U \times V) = (\pi_X \circ f)^{-1}(U) \cap (\pi_Y \circ f)^{-1}(V)$ olup iki açık ön-görüntünün kesişimi olarak Z 'de açıktır; baz üzerinde süreklilik tüm topolojide sürekliliği verir. $\square$

11.3 Algoritmalar

11.3.1 finite_subspace — $O(|\tau| \cdot |A|)$

Algorithm 12 Subspace(X, τ, A)

```

1:  $\tau_A \leftarrow \{\}$ 
2: for each  $U \in \tau$  do
3:    $\tau_A.add(U \cap A)$ 
4: end for
5: return  $(A, \tau_A)$ 
```

11.3.2 binary_product — $O(|\tau_X| \cdot |\tau_Y|)$

Algorithm 13 Product(X, τ_X, Y, τ_Y)

```

1:  $basis \leftarrow \{U \times V : U \in \tau_X, V \in \tau_Y\}$ 
2: return topology_from_basis( $X \times Y, basis$ )
```

11.4 pytop API

```

1 from pytop import (
2     finite_subspace,
3     binary_product,
4     sierpinski_space,
5     discrete_topology,
6     indiscrete_topology,
7     make_topology,
8     make_set,
9     empty_set,
10    is_compact,
11    is_connected,
12    is_t2,
13    separation_inherited_by_subspace,
14 )

```

finite_subspace(space, subset) → FiniteTopologicalSpace
 binary_product(left, right) → FiniteTopologicalSpace (carrier: çift-demetler)
 separation_inherited_by_subspace(subspace_obj, feature = "hausdorff") → Result

11.5 Örnekler

Örnek 5.1 — Alt Uzay: Ayrık Topoloji

```

1 d4 = discrete_topology(1, 2, 3, 4)
2 sub12 = finite_subspace(d4, [1, 2])
3 print("carrier:", sub12.carrier)
4 print("topology:", sorted([sorted(list(u)) for u in sub12.topology],
5                             key=lambda x: (len(x), x)))

```

carrier: (1, 2)
 topology: [[], [1], [2], [1, 2]]

Örnek 5.4 — Çarpım: $D(0, 1) \times D(0, 1)$

```

1 d2 = discrete_topology(0, 1)
2 prod_dd = binary_product(d2, d2)
3 print("carrier:", sorted(prod_dd.carrier))
4 print("topology size:", len(list(prod_dd.topology)))
5 print("compact:", is_compact(prod_dd).status)
6 print("connected:", is_connected(prod_dd).status)

```

carrier: [(0, 0), (0, 1), (1, 0), (1, 1)]
 topology size: 16
 compact: true
 connected: false

Örnek 5.5 — Çarpım: Sierpiński $\times$ Sierpiński

```

1 ss = binary_product(s, s)
2 print("topology size:", len(list(ss.topology)))
3 print("compact:", is_compact(ss).status)
4 print("connected:", is_connected(ss).status)

```

```

topology size: 6
compact: true
connected: true

```

Örnek 5.7 — İndiscrete'in Alt Uzayı İndiscrete'tir

```

1 ind4 = indiscrete_topology(1, 2, 3, 4)
2 sub_ind = finite_subspace(ind4, [1, 2])
3 print("carrier:", sub_ind.carrier)
4 print("topology:", sorted([sorted(list(u)) for u in sub_ind.topology
5                             ],
6                             key=lambda x: (len(x), x)))
7 print("connected:", is_connected(sub_ind).status)

```

```

carrier: (1, 2)
topology: [[], [1, 2]]
connected: true

```

Örnek 5.8 — Hausdorff Kalıtımı ve Çarpıma Etkisi

```

1 d4 = discrete_topology(1, 2, 3, 4)
2 sub_d = finite_subspace(d4, [1, 2])
3 inh = separation_inherited_by_subspace(sub_d, "hausdorff")
4 print("subspace Hausdorff inherited:", inh.status)
5
6 prod_T2 = binary_product(d2, d2)
7 print("D(0,1) x D(0,1) T2:", is_t2(prod_T2).status)
8 prod_notT2 = binary_product(d2, indiscrete_topology(0, 1))
9 print("D(0,1) x Ind(0,1) T2:", is_t2(prod_notT2).status)

```

```

subspace Hausdorff inherited: true
D(0,1) x D(0,1) T2: true
D(0,1) x Ind(0,1) T2: false

```

11.6 Alıştırmalar**Kodlama**

- K1. Sierpiński uzayının $\{0\}$ ve $\{1\}$ alt uzaylarını oluşturun ve hangi topolojiyi taşıdığını belirleyin.

- K2. 4-noktalı özel bir uzayda $\{2, 3\}$ alt uzayını bulun.
- K3. `binary_product(sierpinski_space(), discrete_topology(0,1))` için `is_compact` ve `is_connected` kontrol edin.
- K4. `indiscrete_topology(1,2,3,4)` uzayının $[1,2]$ alt uzayını oluşturup açık küme sayısı ile `is_connected` çıktısını bulun; Örnek 5.7 ile karşılaştırın.
- K5. `separation_inherited_by_subspace` ile `discrete_topology(1,2,3)`'ten alınan $[1,2]$ alt uzayının Hausdorff özelliğini miras alıp almadığını kontrol edin.

Teori

- T1. Alt uzay topolojisinde dahil etme $i : A \rightarrow X$ 'in sürekli olduğunu ispatlayın.
- T2. X ve Y bağlantılı $\Rightarrow X \times Y$ bağlantılı olduğunu ispatlayın.
- T3. Çarpımın evrensel özelliğini ispatlayın: $f : Z \rightarrow X \times Y$ sürekli $\Leftrightarrow \pi_X \circ f$ ve $\pi_Y \circ f$ süreklidir.

Bölüm 12

Bölüm Topolojisi

Bölüm topolojisi, bir topolojik uzayın noktalarını denklik sınıflarına göre *yapıştırarak* yeni bir uzay kurar. Alt uzay ve çarpım topolojisiyle birlikte temel üç inşaat yönteminden birini oluşturur.

12.1 Konu

Tanım 12.1 (Bölüm Kümesi). (X, τ) topolojik uzayı ve X üzerinde bir denklik bağıntısı $\sim$ verilsin. Denklik sınıfı $[x] = \{y \in X : y \sim x\}$. **Bölüm kümesi:** $X/\sim = \{[x] : x \in X\}$.

Tanım 12.2 (Bölüm Topolojisi). Bölüm haritası $q : X \rightarrow X/\sim$, $q(x) = [x]$. **Bölüm topolojisi** $\tau_{X/\sim}$ üzerinde:

$$U \subseteq X/\sim \text{ açıktır} \iff q^{-1}(U) \in \tau.$$

Bu, q 'yu sürekli kılan en ince topolojidir.

Sezgi

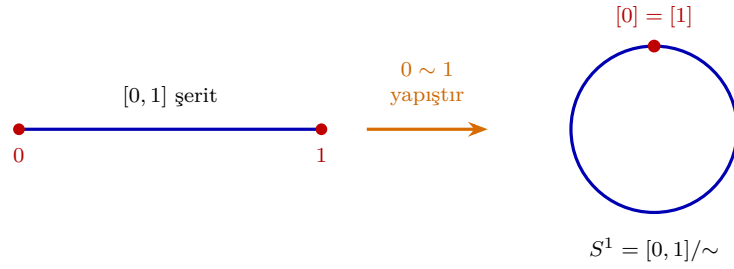
Bölüm topolojisini bir “katlama” gibi düşünün. Elinizde bir kâğıt şerit $([0, 1])$ varsa ve iki ucunu birbirine yapıştırırsanız bir çember (S^1) elde edersiniz. Yapıştırma kuralı bir denklik bağıntısıdır $(0 \sim 1)$; bölüm topolojisi ise “kâğıdı yırtmadan” yapılan bu katlamanın doğal topolojisidir. q haritası her noktayı yapıştırma sonrası bulunduğu konuma gönderir; bir kümenin yapıştırılmış uzayda açık olması, ön-görüntüsünün orijinal kâğıtta açık olmasına bağlıdır.

Tanım 12.3 (Doymuş Küme). Bir $A \subseteq X$ kümesi **doymuştur** (saturated), eğer her $x \in A$ için x 'in tüm denklik sınıfı $[x]$ yine A içinde kalıyorsa, yani $A = q^{-1}(q(A))$ ise. Bölüm topolojisinin açık kümeleri tam olarak X 'in doymuş açık kümelerine karşılık gelir:

$$q(A) \text{ } X/\sim \text{ içinde açık} \iff A \text{ doymuş ve } X \text{ içinde açık.}$$

Karşı-örnek

Hausdorff özelliği bölüm altında korunmaz. $\mathbb{R}$ üzerinde “ $x \sim y \iff x - y \in \mathbb{Q}$ ” bağıntısını alın. Bölüm uzayı $\mathbb{R}/\mathbb{Q}$ kaba (indiscrete) bir uzaydır: tek açık kümeler $\emptyset$ ve tüm uzaydır, çünkü $\mathbb{Q}$ ile öteleme altında doymuş tek açık küme boş küme ve $\mathbb{R}$ 'dir. Dolayısıyla iki farklı nokta hiçbir zaman ayrık komşuluklarla ayrılamaz — $\mathbb{R}$



Bölüm haritası q her noktayı sınıfına gönderir; *tek* fark, iki ucun tek bir noktada ($[0] = [1]$) çakışmasıdır.

Şekil 12.1: $[0, 1]$ şeridinin uçları yapıştırılarak çember S^1 elde edilir; q haritası her noktayı denklik sınıfına gönderir.

Özellik	Bölüm uzayı için durum
Kompaktlık	X kompakt $\Rightarrow X/\sim$ kompakt
Bağlantılılık	X bağlantılı $\Rightarrow X/\sim$ bağlantılı
Hausdorff	Genel olarak korunmaz
T1	$\sim$ kapalı bağıntı ise korunur

Tablo 12.1: Topolojik özelliklerin bölüm altında korunumu

Hausdorff olmasına rağmen $\mathbb{R}/\mathbb{Q}$ T1 bile değildir. Sonlu modelde de aynı tuzak geçerlidir: doymamış açık kümeler yapıştırma sonrası “kaybolur” ve ayırma aksiyomlarını bozabilir.

12.2 Teoremler

Teorem 12.4 (Bölüm haritası sürekliliği). $q : X \rightarrow X/\sim$ her zaman süreklidir.

Teorem 12.5 (Evrensel Özellik). $g : X/\sim \rightarrow Z$ olsun. g süreklidir $\iff g \circ q : X \rightarrow Z$ süreklidir.

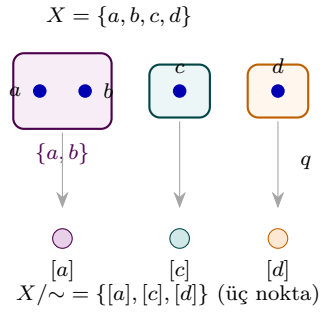
İspat eskizi. $(\Rightarrow)$ g sürekli ve q sürekli (Teorem 1) olduğundan, iki sürekli haritanın bileşkesi $g \circ q$ süreklidir. $(\Leftarrow)$ $g \circ q$ 'nin sürekli olduğunu varsayalım. Z 'de bir V açık kümesi alın; $g^{-1}(V) \subseteq X/\sim$ kümesinin bölüm topolojisinde açık olduğunu göstermeliyiz. Tanım gereği bu, $q^{-1}(g^{-1}(V))$ kümesinin X 'te açık olmasına denktir. Fakat $q^{-1}(g^{-1}(V)) = (g \circ q)^{-1}(V)$ ve $g \circ q$ sürekli olduğundan bu küme X 'te açıktır. Dolayısıyla $g^{-1}(V)$ açıktır ve g süreklidir. Bu özellik, bölüm uzayından çıkan sürekli haritaların kontrolünü X üzerindeki haritaların kontrolüne indirger ve bölüm topolojisini evrensel bir nesne olarak karakterize eder. $\square$

Teorem 12.6 (Kompaktlık Korunumu). X kompakt $\Rightarrow X/\sim$ kompakttır.

İspat eskizi. q sürekli ve örtendir; kompakt bir kümenin sürekli görüntüsü kompakt olduğundan $X/\sim = q(X)$ kompakttır. $\square$

Teorem 12.7 (Bağlantılılık Korunumu). X bağlantılı $\Rightarrow X/\sim$ bağlantılıdır.

Teorem 12.8 (Hausdorff Kriteri). $X/\sim$ Hausdorff $\iff \sim$ grafiği $X \times X$ 'te kapalıdır.



Şekil 12.2: Denklik sınıfları bölüm uzayında birer noktaya çöker: dört nokta $\{a, b\}$, $\{c\}$, $\{d\}$ sınıflarıyla üç noktaya iner.

12.3 Algoritmalar

12.3.1 quotient_set — $O(|X|^2 \cdot \alpha(|X|))$

Algorithm 14 QuotientSet($X, \sim$)

- 1: Her x için sınıf $\leftarrow \{x\}$ (union-find başlat)
 - 2: **for** each $(x, y) \in \sim$ **do**
 - 3: **Union**(x, y)
 - 4: **end for**
 - 5: **return** kök temsilcilere göre sınıfları tuple olarak
-

12.3.2 finite_quotient_contract — $O(|X| + |P|)$

Algorithm 15 FiniteQuotientContract(X, P)

- 1: // P : X 'in örtüşmeyen bölüntüsü
 - 2: Doğrula: P her x 'i kapsar ve bloklar ayrık
 - 3: **return** FiniteConstructionContract(status =true, carrier_size = $|X|$,
 block_count = $|P|$)
-

12.4 pytop API

```

1 from pytop import (
2     quotient_set,           # denklik bagintisindan bolum kumesi
3     finite_quotient_contract, # boluntuden infaat sozlesmesi
4     finite_quotient_summary, # kısa ozet dizesi
5     make_quotient_map,      # QuotientMap nesnesi
6     is_quotient_map,        # Result: harita bolum haritasi mi?
7     equivalence_class,      # bir noktanin denklik sinifi
8     partition_from_equivalence, # denklik bagintisi -> boluntu
9     is_equivalence_relation, # baginti denklik bagintisi mi?
10    equivalence_from_partition, # boluntu -> denklik bagintisi

```

```

11     equivalence_from_classes,      # sınıf bloklarından denklik
        bagintisi
12     canonical_projection_from_equivalence,  # q haritasini sozluk
        olarak
13     discrete_topology,            # ayrik topoloji
14     sierpinski_space,            # Sierpinski uzayi
15     make_topology,                # tasiyici + aciklardan
        FiniteTopologicalSpace
16 )

```

```

quotient_set(carrier, relation) → tuple[frozenset, ...]
finite_quotient_contract(carrier, partition) → FiniteConstructionContract
finite_quotient_summary(carrier, partition) → str
make_quotient_map(domain, codomain) → QuotientMap
is_quotient_map(map_obj) → Result
equivalence_from_partition(universe, partition) → set[tuple]
partition_from_equivalence(carrier, relation) → set[frozenset]
equivalence_class(carrier, relation, point) → set
canonical_projection_from_equivalence(carrier, relation) → dict

```

12.5 Örnekler

Örnek 5.1 — İki Noktayı Özdeşleştirme

```

1 d4 = discrete_topology(0, 1, 2, 3)
2 partition_1 = [[0, 1], [2], [3]]
3 qs1 = quotient_set(
4     [0, 1, 2, 3],
5     [(0,0),(1,1),(2,2),(3,3),(0,1),(1,0)]
6 )
7 print("Bolum kumesi:", qs1)
8 contract1 = finite_quotient_contract([0, 1, 2, 3], partition_1)
9 print("Blok sayisi:", contract1.block_count)
10 print("Durum:", contract1.status)

```

```

Bolum kumesi: (frozenset({2}), frozenset({3}), frozenset({0, 1}))
Blok sayisi: 3
Durum: true

```

Örnek 5.2 — Tüm Noktaları Tek Noktaya Çökertme

```

1 carrier = [0, 1, 2, 3]
2 qs2 = quotient_set(
3     carrier,
4     [(i, j) for i in carrier for j in carrier]
5 )
6 print("Tek blok:", qs2)
7 contract2 = finite_quotient_contract(carrier, [carrier])
8 print("Blok sayisi:", contract2.block_count)

```

```

9 r2 = contract2.to_result()
10 print("Metadata:", r2.metadata['block_sizes'])

```

Tek blok: (frozenset({0, 1, 2, 3}),)
 Blok sayısı: 1
 Metadata: [4]

Örnek 5.3 — Özel Topolojide Bölüm Sözleşmesi

```

1 X5 = [1, 2, 3, 4, 5]
2 tau5 = [set(), {1,2}, {3,4}, {1,2,3,4}, {1,2,3,4,5}]
3 fts5 = make_topology(X5, *tau5)
4 partition5 = [[1, 2], [3, 4], [5]]
5 contract5 = finite_quotient_contract(X5, partition5)
6 r5 = contract5.to_result()
7 print("Status:", r5.status)
8 print("Blokler:", r5.metadata['block_count'])
9 print("Blok boyutlari:", r5.metadata['block_sizes'])

```

Status: true
 Bloklar: 3
 Blok boyutlari: [2, 2, 1]

Örnek 5.4 — Bölüm Haritası Nesnesi

```

1 d3 = discrete_topology(0, 1, 2)
2 d2 = discrete_topology('a', 'b')
3 qmap = make_quotient_map(d3, d2)
4 print("Etiketler:", sorted(qmap.tags))
5 r_qmap = is_quotient_map(qmap)
6 print("is_quotient_map status:", r_qmap.status)
7 print("Justification:", r_qmap.justification[0])

```

Etiketler: ['continuous', 'quotient', 'surjective']
 is_quotient_map status: true
 Justification: Map tag 'quotient' is explicitly present.

Örnek 5.5 — $[0,1]$ Uçlarını Yapıştırarak Çember S^1

$[0,1]$ aralığını 5 noktayla modelliyoruz. İki ucu (0 ve 4) özdeşleştirmek, şeridi bir çembere yapıştırmaya karşılık gelir.

```

1 ring = [0, 1, 2, 3, 4]
2 ring_classes = [[0, 4], [1], [2], [3]]
3 ring_rel = equivalence_from_partition(ring, ring_classes)
4 print("Denklik mi?", is_equivalence_relation(ring, ring_rel))
5 print("Bolum kumesi boyutu:", len(quotient_set(ring, ring_rel)))
6 print("Ozet:", finite_quotient_summary(ring, ring_classes))
7 print("0'in sinifi:", sorted(equivalence_class(ring, ring_rel, 0)))
8 print("4'un sinifi:", sorted(equivalence_class(ring, ring_rel, 4)))

```

```

Denklik mi? True
Bolum kumesi boyutu: 4
Ozet: quotient: status =true, carrier =5, blocks =4
0'in sinifi: [0, 4]
4'un sinifi: [0, 4]

```

Uçlar tek bir sınıfta birleşti: 5 noktalı şerit, 4 noktalı çembere indi.

Örnek 5.6 — Doymuş Kümeler ve Kanonik İzdüşüm

Kanonik izdüşüm q her noktayı denklik sınıfına gönderir. Bir kümenin **doymuş** olması, içerdiği her noktanın sınıfını da tamamen içermesi demektir.

```

1 sat_carrier = ['a', 'b', 'c', 'd']
2 sat_part = [['a', 'b'], ['c'], ['d']]
3 sat_rel = equivalence_from_partition(sat_carrier, sat_part)
4 proj = canonical_projection_from_equivalence(sat_carrier, sat_rel)
5 for pt in sat_carrier:
6     print(f"q({pt}) =", sorted(proj[pt]))
7 cls_a = equivalence_class(sat_carrier, sat_rel, 'a')
8 A = {'a', 'b'}
9 print("{a,b} doymus mu?",
10       all(equivalence_class(sat_carrier, sat_rel, p) <= A for p in A)
11       )
11 print("{a} doymus mu?", cls_a <= {'a'})

```

```

q(a) = ['a', 'b']
q(b) = ['a', 'b']
q(c) = ['c']
q(d) = ['d']
{a,b} doymus mu? True
{a} doymus mu? False

```

$\{a, b\}$ doymuştur, ama $\{a\}$ doymuş değildir: sınıf arkadaşı b 'yi içermez. Bölüm topolojisinin açık kümeleri ancak doymuş açık kümelerden gelir.

Örnek 5.7 — Sınıflardan Noktalara: Round-Trip

`equivalence_from_classes` sınıf bloklarından denklik bağıntısı kurar; `partition_from_equivalence` bağıntıyı tekrar bölüntüye çevirir.

```

1 rt_blocks = equivalence_from_classes([1, 2], [3], [4, 5])
2 print("Denklik mi?", is_equivalence_relation([1, 2, 3, 4, 5],
3       rt_blocks))
4 rt_part = partition_from_equivalence([1, 2, 3, 4, 5], rt_blocks)
5 print("Blok sayısı:", len(rt_part))
6 print("Bloklar:", sorted(sorted(b) for b in rt_part))

```

```

Denklik mi? True
Blok sayısı: 3
Bloklar: [[1, 2], [3], [4, 5]]

```

Sınıflardan kurulan bağıntı, bölüntüye çevrilip geri okunduğunda aynı üç bloğu verir: bölüm uzayının 3 noktası, tam olarak 3 denklik sınıfına karşılık gelir.

12.6 Alıştırmalar

Kodlama

- K1. `discrete_topology(0,1,2,3,4)` üzerinde `[[0,4],[1,3],[2]]` bölüntüsünü kullanarak `finite_quotient_contract` çağırın; blok sayısını ve boyutlarını yazdırın.
- K2. `sierpinski_space()` üzerinde `trivial` bölüntü ile `finite_quotient_summary` çağırın ve çıktıyı yorumlayın.
- K3. `make_topology([1,2,3,4], set(), {1,2}, {3,4}, {1,2,3,4})` üzerinde `[[1,2],[3,4]]` ve `[[1],[2],[3],[4]]` bölüntülerini karşılaştırın.
- K4. `[0,1,2,3,4,5]` taşıyıcısı üzerinde `[[0,5],[1],[2],[3],[4]]` bölüntüsünü `equivalence_from_p` ile denklik bağıntısına çevirin; `is_equivalence_relation` ile doğrulayın ve 0 ile 5'in `equivalence_class` çıktılarının aynı olduğunu gösterin.
- K5. `['x','y','z']` taşıyıcısı ve `[['x','y'],['z']]` bölüntüsü için `canonical_projection_from_eq` çağırıp her noktanın görüntüsünü yazdırın; `['x','y']` kümesinin doymuş, `['x']` kümesinin doymuş olmadığını gösterin.

Teori

- T1. $q : X \rightarrow X/\sim$ bölüm haritasının $\tau_{X/\sim}$ tanımı gereği her zaman sürekli olduğunu ispatlayın.
- T2. X bağlantılı $\Rightarrow X/\sim$ bağlantılı olduğunu ispatlayın. (q sürekli ve surjektif; bağlantılı kümenin sürekli görüntüsü bağlantılıdır.)
- T3. Bölüm haritasının **evrensel özelliğini** ispatlayın: $g : X/\sim \rightarrow Z$ için g sürekli $\iff g \circ q$ sürekli.
İpucu: $(g \circ q)^{-1}(V) = q^{-1}(g^{-1}(V))$ eşitliğini bölüm topolojisinin tanımıyla birleştirin.
- T4. $A \subseteq X$ doymuş ($A = q^{-1}(q(A))$) ise $q(A)$ 'nın bölüm uzayında açık olması için A 'nın X 'te açık olmasının yeterli ve gerekli olduğunu gösterin. Doymamış bir A için bu yazışmanın neden bozulduğunu açıklayın.

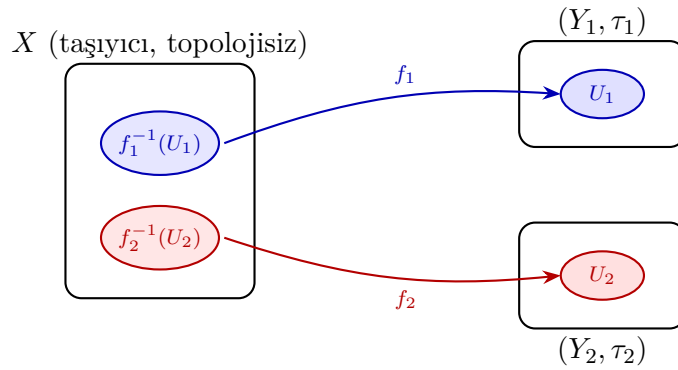
Bölüm 13

Başlangıç ve Son Topoloji

Başlangıç topolojisi (initial topology), verilen haritaları sürekli kılan en kaba (coarsest) topolojidir. **Son topoloji** (final topology), verilen haritaları sürekli kılan en ince (finest) topolojidir. Altuzay ve çarpım topolojileri başlangıç topolojisinin; bölüm topolojisi ise son topolojisinin özel halleridir.

Sezgi

Başlangıç topolojisini, X 'e “tam yeterince” açık küme koyan bir ayar düğmesi gibi düşünün. Çok az açık koyarsanız f_α haritaları sürekli olmaz; çok fazla koyarsanız gereksizdir. Geri çekme $f_\alpha^{-1}(U)$ ile üretilen alt-baz, haritaları sürekli kılan *en küçük* (en kaba) açık-küme koleksiyonudur. Son topoloji ise tam tersi yönde çalışır: Y 'ye en çok açık kümeyi koyar, ama her g_i 'yi sürekli bozmadan.



Alt-baz $\mathcal{S} = \{f_\alpha^{-1}(U)\}$ geri çekme ile üretilir;
 $\tau_{\text{ini}} = \langle \mathcal{S} \rangle$ her f_α 'yı sürekli kılan *en kaba* topoloji.

Şekil 13.1: Başlangıç topolojisi: bir harita ailesini sürekli kılan en kaba topoloji, geri çekme $f_\alpha^{-1}(U)$ ile üretilen alt-bazdan doğar.

13.1 Başlangıç Topolojisi

X kümesi ve haritalar ailesi $\{f_\alpha : X \rightarrow (Y_\alpha, \tau_\alpha)\}_{\alpha \in A}$ verilsin.

Tanım 13.1 (Başlangıç Topolojisi). **Başlangıç topolojisi** τ_{ini} , her f_α 'yı $(X, \tau_{\text{ini}}) \rightarrow (Y_\alpha, \tau_\alpha)$ sürekli kılan topolojilerin kesişimidir. Alt-baz:

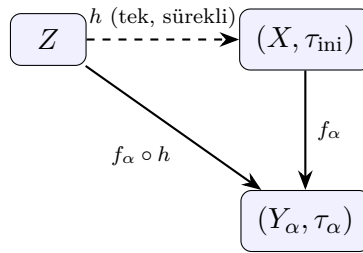
$$\mathcal{S} = \{f_\alpha^{-1}(U) \mid U \in \tau_\alpha, \alpha \in A\}, \quad \tau_{\text{ini}} = \langle \mathcal{S} \rangle.$$

Teorem 13.2 (Varlık ve Teklik). τ_{ini} *tekil ve vardır*; her f_α 'yı sürekli kılan tüm topolojilerin en kaba olanıdır.

Kanıt İskeleti. $\mathcal{S}$ 'nin ürettiği topoloji $\langle \mathcal{S} \rangle$ her f_α 'yı sürekli kılar (çünkü $f_\alpha^{-1}(U) \in \langle \mathcal{S} \rangle$ her $U \in \tau_\alpha$ için). Eğer τ' de her f_α 'yı sürekli kılıyorsa, $\mathcal{S} \subseteq \tau'$ olduğundan $\langle \mathcal{S} \rangle \subseteq \tau'$; yani τ_{ini} en kaba. $\square$

Teorem 13.3 (Evrensel Özellik). $h : Z \rightarrow (X, \tau_{\text{ini}})$ *sürekli* ancak ve ancak $f_\alpha \circ h$ her α için sürekli olduğunda.

İspat eskizi. $(\Rightarrow)$ h sürekli ve f_α sürekli ise bileşke $f_\alpha \circ h$ sürekli. $(\Leftarrow)$ τ_{ini} 'nin bir alt-baz elemanı $f_\alpha^{-1}(U)$ biçimindedir ve ön-görüntüsü $h^{-1}(f_\alpha^{-1}(U)) = (f_\alpha \circ h)^{-1}(U)$, ki bu $f_\alpha \circ h$ sürekli olduğundan Z 'de açıktır. Süreklilik alt-bazda denetlenebildiğinden h sürekli. $\square$



Evrensel özellik: h sürekli $\iff$ her $f_\alpha \circ h$ sürekli.
 τ_{ini} bu faktörizasyonu mümkün kılan *tek* topolojidir.

Şekil 13.2: Evrensel özellik: h sürekli $\iff$ her $f_\alpha \circ h$ sürekli; τ_{ini} bu faktörizasyonu mümkün kılan tek topolojidir.

Teorem 13.4 (Özel Haller). 1. *Altuzay topolojisi, ekleme haritası $i : A \hookrightarrow X$ 'in başlangıç topolojisidir.*

2. *Çarpım topolojisi, projeksiyon haritaları $\{\pi_\alpha\}$ 'nın başlangıç topolojisidir.*

13.2 Algoritmalar

13.2.1 Başlangıç Topolojisi İnşası

Karmaşıklık. Ön-görüntü hesabı $O(|\tau_\alpha| \cdot |X|)$; topoloji üretimi $O(|\mathcal{S}| \cdot 2^{|X|})$ en kötü durumda.

13.2.2 Son Topoloji İnşası

Karmaşıklık. $O(2^{|Y|} \cdot \sum_i |\sigma_i|)$.

Algorithm 16 Başlangıç Topolojisi (Sonlu Durum)**Require:** Sonlu taşıyıcı X , haritalar $\{(f_\alpha, \tau_\alpha)\}$ **Ensure:** τ_{ini} — en kaba süreklilik topolojisi

```

1:  $\mathcal{S} \leftarrow \emptyset$ 
2: for her  $f_\alpha$  do
3:   for her  $U \in \tau_\alpha$  do
4:      $\mathcal{S} \leftarrow \mathcal{S} \cup \{f_\alpha^{-1}(U)\}$ 
5:     //  $f_\alpha^{-1}(U) = \{x \in X \mid f_\alpha(x) \in U\}$ 
6:   end for
7: end for
8: return  $\langle \mathcal{S} \rangle$  // alt-bazdan topoloji üret

```

Algorithm 17 Son Topoloji (Sonlu Durum)**Require:** Sonlu Y , kaynak uzaylar $\{(X_i, \sigma_i)\}$, haritalar $\{g_i : X_i \rightarrow Y\}$ **Ensure:** τ_{fin} — en ince süreklilik topolojisi

```

1:  $\tau_{\text{fin}} \leftarrow \emptyset$ 
2: for her  $V \subseteq Y$  do
3:   if  $g_i^{-1}(V) \in \sigma_i$  her  $i$  için then
4:      $\tau_{\text{fin}} \leftarrow \tau_{\text{fin}} \cup \{V\}$ 
5:   end if
6: end for
7: return  $\tau_{\text{fin}}$ 

```

13.3 pytop API

Listing 13.1: Başlangıç Topolojisi API

```

1 from pytop.finite_map_analysis import FiniteMap
2 from pytop import (
3     initial_topology_from_maps,
4     generic_quotient_space_from_map,
5     discrete_topology,
6     indiscrete_topology,
7     make_topology,
8     sierpinski_space,
9     finite_subspace,
10    binary_product,
11    is_continuous_finite_map,
12    is_t0,
13    is_t2,
14 )
15
16 # FiniteMap doğrudan veri sınıfı olarak kullanılır (make_function
17   # değil)
18 # tau_ini = initial_topology_from_maps(carrier, [f1, f2, ...])
19 # quot = generic_quotient_space_from_map(q_map) # son topoloji,
20   # bolum özel hali

```

Önemli: `pytop.quotient_space_from_map` sembolik motoru kullanır; sonlu-duyarlı

sürüm `generic_quotient_space_from_map`'tir.

13.4 Örnekler

13.4.1 Tek Harita ile Başlangıç Topolojisi

```

1 X_carrier = [0, 1, 2, 3]
2 Y_d = discrete_topology(0, 1)
3 X_ph = make_topology(X_carrier, set(), set(X_carrier))
4
5 f = FiniteMap(domain=X_ph, codomain=Y_d, name="f",
6             mapping={0: 0, 1: 0, 2: 1, 3: 1})
7 tau_f = initial_topology_from_maps(X_carrier, [f])
8
9 print("Baslangic topolojisi:")
10 for u in sorted([sorted(u) for u in tau_f.topology], key=lambda x: (
11     len(x), x)):
12     print("    ", u if u else "empty")
13 print("Eleman sayisi:", len(tau_f.topology))

```

```

Baslangic topolojisi:
    empty
    [0, 1]
    [2, 3]
    [0, 1, 2, 3]
Eleman sayisi: 4

```

$f^{-1}(\{0\}) = \{0, 1\}$, $f^{-1}(\{1\}) = \{2, 3\}$ — alt-baz $\{\{0, 1\}, \{2, 3\}\}$, üretilen topoloji 4 elemanlı.

13.4.2 İki Harita — Topoloji İncelir

```

1 S_sier = sierpinski_space()
2 g = FiniteMap(domain=X_ph, codomain=S_sier, name="g",
3             mapping={0: 0, 1: 1, 2: 1, 3: 1})
4 tau_fg = initial_topology_from_maps(X_carrier, [f, g])
5
6 print("f tek basina:", len(tau_f.topology), "eleman | f+g:", len(
7     tau_fg.topology), "eleman")

```

```
f tek basina: 4 eleman | f+g: 6 eleman
```

$g^{-1}(\{1\}) = \{1, 2, 3\}$ yeni bir alt-baz elemanı ekler; kesişim $\{0, 1\} \cap \{1, 2, 3\} = \{1\}$ yeni bir açık küme üretir.

13.4.3 Altuzay = Başlangıç

```

1 fts_X = make_topology([0, 1, 2, 3], set(), {0, 1}, {2, 3}, {0, 1, 2,
2     3})

```

```

2 sub_A = finite_subspace(fts_X, [1, 2])
3
4 A_dom = discrete_topology(1, 2)
5 inc = FiniteMap(domain=A_dom, codomain=fts_X, name="i", mapping
6         ={1: 1, 2: 2})
7
8 init_A = initial_topology_from_maps([1, 2], [inc])
9
10 sub_tau = sorted([sorted(u) for u in sub_A.topology], key=lambda x:
11                 (len(x), x))
12 init_tau = sorted([sorted(u) for u in init_A.topology], key=lambda x:
13                 (len(x), x))
14 print("Esit mi:", sub_tau == init_tau)

```

```
Esit mi: True
```

13.4.4 Çarpım = Başlangıç

```

1 d2 = discrete_topology(0, 1)
2 prod = binary_product(d2, d2)
3
4 pi_x = FiniteMap(domain=prod, codomain=d2, name="pi_x",
5                 mapping={(0,0):0, (0,1):0, (1,0):1, (1,1):1})
6 pi_y = FiniteMap(domain=prod, codomain=d2, name="pi_y",
7                 mapping={(0,0):0, (0,1):1, (1,0):0, (1,1):1})
8
9 init_prod = initial_topology_from_maps(list(prod.carrier), [pi_x,
10                 pi_y])
11 prod_fs = {frozenset(u) for u in prod.topology}
12 init_fs = {frozenset(u) for u in init_prod.topology}
13 print("Carpim == Baslangic:", prod_fs == init_fs)

```

```
Carpim == Baslangic: True
```

13.4.5 Son Topoloji — Manuel Hesap

```

1 X1 = make_topology([0, 1, 2], set(), {0, 1}, {0, 1, 2})
2 Y_car = [0, 1]
3 g1_map = {0: 0, 1: 0, 2: 1}
4 tau_X1 = {frozenset(u) for u in X1.topology}
5
6 open_Y = []
7 for mask in range(1 << len(Y_car)):
8     V = frozenset(Y_car[i] for i in range(len(Y_car)) if mask
9                 & (1 << i))
10    preimage = frozenset(x for x in X1.carrier if g1_map[x] in V)
11    if preimage in tau_X1:
12        open_Y.append(set(V))
13
14 final_Y = make_topology(Y_car, *open_Y)

```

```

14 final_tau = sorted([sorted(u) for u in final_Y.topology], key=lambda
    x: (len(x), x))
15 print("Son topoloji:", final_tau)

```

```
Son topoloji: [[], [0], [0, 1]]
```

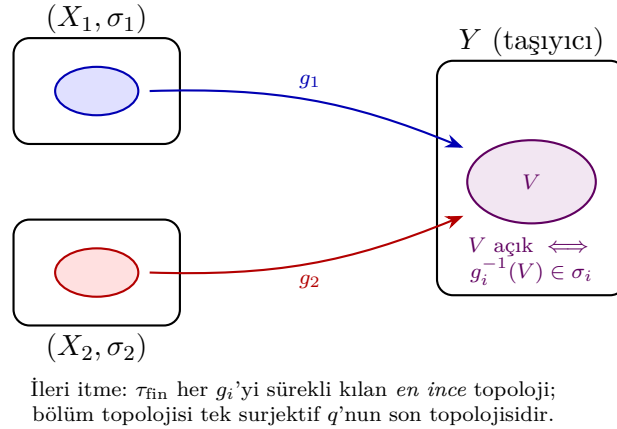
13.4.6 Bölüm = Son Topoloji

```

1 Y_ind = indiscrete_topology(0, 1)
2 q = FiniteMap(domain=X1, codomain=Y_ind, name="q",
3             mapping={0: 0, 1: 0, 2: 1})
4 quot_Y = generic_quotient_space_from_map(q)
5 quot_tau = sorted([sorted(u) for u in quot_Y.topology], key=lambda x:
6                 (len(x), x))
7 print("Bolum topolojisi:", quot_tau)
8 print("Son topoloji ile esit:", quot_tau == final_tau)

```

```
Bolum topolojisi: [[], [0], [0, 1]]
Son topoloji ile esit: True
```



Şekil 13.3: Son topoloji: kaynak uzayları ortak bir hedefe iten en ince topoloji; bölüm topolojisi tek surjektif haritanın son topolojisidir.

13.4.7 Yeni Örnek — “En Kaba” Olmanın Doğrudan Doğrulaması

Başlangıç topolojisinin tanımı *en kaba* olmaktır: f' 'yi sürekli kılar, ama ondan daha kaba hiçbir topoloji kılamaz. Bunu süreklilik yüklemiyle doğrulayalım.

```

1 ini = [set(u) for u in tau_f.topology]
2 codom = [set(u) for u in Y_d.topology]
3 coarser = [set(), set(X_carrier)] # indiscrete: tau_ini'den daha
    kaba
4

```

```

5 print("f, tau_ini'ye gore surekli mi:",
6       is_continuous_finite_map(X_carrier, ini, list(Y_d.carrier),
7                                codom, f.mapping))
8 print("f, daha kaba (indiscrete) topolojiye gore surekli mi:",
9       is_continuous_finite_map(X_carrier, coarser, list(Y_d.carrier),
                                codom, f.mapping))
10 print("tau_ini eleman sayisi:", len(tau_f.topology))

```

```

f, tau_ini'ye gore surekli mi: True
f, daha kaba (indiscrete) topolojiye gore surekli mi: False
tau_ini eleman sayisi: 4

```

τ_{ini} ile f sürekli; aynı f indiscrete topolojiyle sürekli **değil** — çünkü $f^{-1}(\{0\}) = \{0, 1\}$ orada açık değildir. τ_{ini} tam da bu açık kümeyi ekleyecek kadar incedir, fazlası değil.

Karşı-örnek

“Daha çok açık küme her zaman daha iyidir” sanmayın. Discrete topoloji de f ’yi sürekli kılar (16 açık küme), ama *en kaba* değildir — başlangıç topolojisi yalnızca süreklilik için gereken 4 açık kümeyi koyar. Gereğinden ince bir topoloji evrensel özelliği bozar: τ_{ini} ’den daha ince bir τ' seçerseniz, $f_\alpha \circ h$ sürekli olduğu hâlde h sürekli olmayan bir h bulunabilir.

13.4.8 Yeni Örnek — Evrensel Özelliğin Doğrulanması

İki harita $f, g : X \rightarrow \text{discrete}(\{0, 1\})$ ile $X = \{a, b\}$ üzerinde başlangıç topolojisini kurar, ardından $h : Z \rightarrow X$ sürekliliğinin her bileşenin sürekliliğine denk olduğunu gösteririz.

```

1 Xc = ["a", "b"]
2 Xph = make_topology(Xc, set(), set(Xc))
3 Yd2 = discrete_topology(0, 1)
4 fA = FiniteMap(domain=Xph, codomain=Yd2, name="fA", mapping={"a":
5                      0, "b": 1})
6 gA = FiniteMap(domain=Xph, codomain=Yd2, name="gA", mapping={"a":
7                      1, "b": 0})
8
9 tau_X = initial_topology_from_maps(Xc, [fA, gA])
10 Xopens = [set(u) for u in tau_X.topology]
11 Yopens = [set(u) for u in Yd2.topology]
12
13 Zc, Zopens = ["p", "q"], [set(), {"p"}, {"q"}, {"p", "q"}]
14 h = {"p": "a", "q": "b"}
15 fh = {z: fA.mapping[h[z]] for z in Zc} # fA . h
16 gh = {z: gA.mapping[h[z]] for z in Zc} # gA . h
17
18 c_fh = is_continuous_finite_map(Zc, Zopens, list(Yd2.carrier), Yopens,
19                                  fh)
20 c_gh = is_continuous_finite_map(Zc, Zopens, list(Yd2.carrier), Yopens,
21                                  gh)
22 c_h = is_continuous_finite_map(Zc, Zopens, Xc, Xopens, h)
23
24 print("tau_ini(X) eleman sayisi:", len(tau_X.topology))

```

```

21 print("fA.h sürekliliği:", c_fh, "| gA.h sürekliliği:", c_gh, "| h sürekliliği:",
    c_h)
22 print("Evrensel özellik sağlanıyor mu:", c_h == (c_fh and c_gh))

```

```

tau_ini(X) eleman sayısı: 4
fA.h sürekliliği: True | gA.h sürekliliği: True | h sürekliliği: True
Evrensel özellik sağlanıyor mu: True

```

$\tau_{\text{ini}}(X)$ discrete($\{a, b\}$)'ye eşittir; h sürekli ancak ve ancak hem $f_A \circ h$ hem $g_A \circ h$ sürekli olduğunda. Yükleme bu denkliği `c_h == (c_fh and c_gh)` ile doğrular.

13.5 Alıştırılmalar

1. $f : \{a, b, c, d\} \rightarrow \text{discrete}(\{0, 1\})$, $f(a) = f(b) = 0$, $f(c) = f(d) = 1$ için `initial_topology_from_` kullanın ve sonucu `finite_subspace` ile karşılaştırın.
2. Çarpım özel halinde yalnızca π_x kullanıldığında başlangıç topolojisi ne olur? Discrete'den daha kaba mı?
3. $X = \{0, 1, 2, 3, 4\}$, $\sigma_X = \text{discrete}$, $g : X \rightarrow \{0, 1, 2\}$ haritası için son topolojiyi manuel hesaplayın.
4. (Teori) $h : Z \rightarrow (X, \tau_{\text{ini}})$ sürekliliğinin $f_\alpha \circ h$ sürekliliğine denk olduğunu ispatlayın.
5. (Teori) τ_{fin} 'den daha ince hiçbir topoloji her g_i 'yi sürekli kılamaz; bunu tanımdaki en-ince koşulundan türetin.
6. `initial_topology_from_maps` ile bir $f : \{0, 1, 2, 3\} \rightarrow \text{discrete}(\{0, 1\})$, $f(\{0, 1\}) = 0$, $f(\{2, 3\}) = 1$ haritası için τ_{ini} kurun. Sonra `is_continuous_finite_map` ile f 'nin τ_{ini} 'ye göre sürekli ama indiscrete topolojiye göre sürekli olmadığını doğrulayın. Aynı f discrete topolojiyle de sürekli midir? Bu, "en kaba" iddiasını nasıl destekler?
7. (Evrensel özellik) $X = \{a, b\}$ üzerinde $f(a) = 0, f(b) = 1$ ve $g(a) = 1, g(b) = 0$ (kodomen `discrete(\{0, 1\})`) için τ_{ini} 'yi hesaplayın. $Z = \text{discrete}(\{p, q\})$, $h(p) = a$, $h(q) = b$ alın. `is_continuous_finite_map` ile $f \circ h$, $g \circ h$ ve h sürekliliklerini hesaplayıp h 'nin sürekliliğinin bileşenlerin sürekliliğine denk olduğunu (`c_h == (c_fh and c_gh)`) gösterin.

Kısım III

Metrik Uzaylar

Bölüm 14

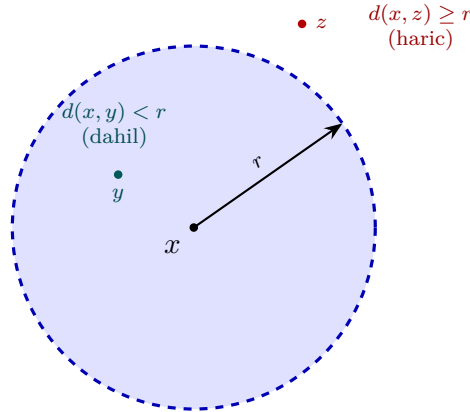
Metrik Uzaylar

Metrik uzaylar, mesafe fonksiyonuna dayanan topolojik yapılardır. Her metrik uzay doğal olarak bir topolojik uzay oluşturur; bu yapı “indüklenen topoloji” olarak adlandırılır.

14.1 Konu

14.1.1 Sezgisel Giriş — “Mesafe Topolojiyi Doğurur”

Metrik uzayı tek cümlede özetlemek gerekirse: *noktalar arasındaki mesafe fikrinden bir topoloji üretmektir*. Elimizde yalnız “iki nokta ne kadar uzak?” sorusunu yanıtlayan bir $d(x, y)$ fonksiyonu vardır; bundan “bir noktanın yakınında olmak” kavramını — yani açık kümeleri — kurarız. Köprü tek bir nesnedir: **açık yuvar**.



$B(x, r) = \{y \in M : d(x, y) < r\}$ — kesikli sınır açık yuvara **dahil değildir**.

Şekil 14.1: Açık yuvar $B(x, r)$: merkez x ’ten r ’den daha yakın olan tüm noktalar; kesikli sınır yuvara dahil değildir.

Bu yuvarları temel taş yaparak indüklenen topolojiyi kurarız: bir küme açıktır $\Leftrightarrow$ her noktasının etrafına tümüyle içerde kalan bir açık yuvar sığar.

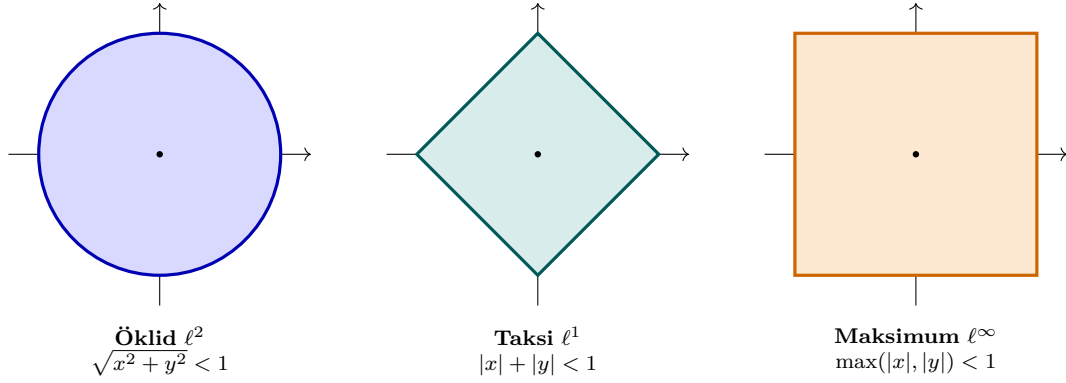
Sezgi

Metrik, sayısal bir “uzaklık cetveli”dir; topoloji ise yalnız “yakın mı, uzak mı?” sorusunu önemser. Aynı topolojiyi farklı cetveller (metrikler) üretebilir — önemli olan

hangi noktaların hangi yuvarın *içinde* kaldığıdır, mesafenin tam sayısal değeri değil.

Aynı düzlem üzerinde farklı metrikler farklı biçimde yuvarlar üretir ama çoğu zaman **aynı topolojiyi** indükler:

Aynı düzlemde üç farklı metriğin birim yuvarı $B(0, 1)$: biçimleri farklı, indükledikleri *topoloji* aynıdır.



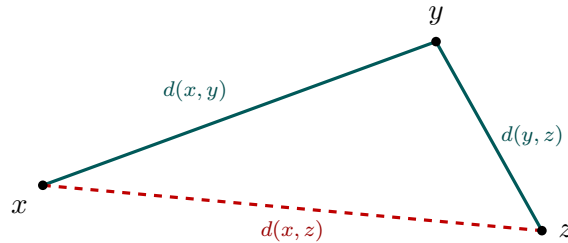
Şekil 14.2: Üç farklı metriğin birim yuvarı $B(0, 1)$: Öklid çember, taksi (Manhattan) elması, maksimum metrik karesi — biçimleri farklı, indükledikleri topoloji aynı.

14.1.2 Metrik Aksiyomları

Tanım 14.1 (Metrik Uzay). M kümesi ve $d : M \times M \rightarrow [0, \infty)$ verilsin. (M, d) bir **metrik uzay**, d ise **metrik** ise:

- (M1) **Özdeşlik**: $d(x, y) = 0 \Leftrightarrow x = y$
- (M2) **Simetri**: $d(x, y) = d(y, x)$
- (M3) **Üçgen**: $d(x, z) \leq d(x, y) + d(y, z)$

Üçgen eşitsizliği, üç aksiyom içinde en derin olanıdır: “ x ’ten z ’ye doğrudan gitmek, y ’ye uğrayıp gitmekten kısadır” der.



Üçgen eşitsizliği: $d(x, z) \leq d(x, y) + d(y, z)$
Doğrudan yol (kesikli kırmızı), y üzerinden geçen dolaylı yoldan kısadır.

Şekil 14.3: Üçgen eşitsizliği: x ’ten z ’ye doğrudan mesafe, y üzerinden geçen dolaylı yoldan kısadır.

Karşı-örnek

Üçgen eşitsizliği şart. $\{A, B, C\}$ üzerinde $d(A, B) = 1$, $d(B, C) = 1$ ama $d(A, C) = 5$ koyalım (özdeşlik ve simetri korunsun). M1 ve M2 sağlanır, ama $d(A, C) = 5 > d(A, B) + d(B, C) = 2$ olduğundan M3 ihlal edilir. Bu fonksiyon bir *metrik değildir*; ürettiği “yuvarlar” tutarlı bir topoloji vermez. `validate_metric` bu durumu yakalar (Alıştırma K4).

14.1.3 Temel Kavramlar

- **Açık top:** $B(x, r) = \{y \in M : d(x, y) < r\}$
- **Kapalı top:** $\bar{B}(x, r) = \{y \in M : d(x, y) \leq r\}$
- **İndüklenen topoloji:** $\tau_d = \{U : \forall x \in U, \exists \varepsilon > 0, B(x, \varepsilon) \subseteq U\}$
- **Çap:** $\text{diam}(A) = \sup\{d(x, y) : x, y \in A\}$

14.1.4 Standart Metrikler

Metrik	Tanım	Uzay
Öklid	$d(x, y) = x - y $	$\mathbb{R}$
Ayrık	$d(x, y) = 0$ veya 1	Herhangi X
ℓ^∞ (max)	$\max_i d_i(x_i, y_i)$	Çarpım uzayı
Sınırlı (capped)	$d'(x, y) = \min(d(x, y), 1)$	Eşdeğer topoloji

Tablo 14.1: Standart metrikler

14.2 Teoremler

Teorem 14.2. Her metrik uzay Hausdorff (T_2) ve 1^{st} sayılabilirdir.

İspat eskizi. (T_2) $x \neq y$ olsun; o hâlde $r = d(x, y) > 0$ (M1). $B(x, r/2)$ ve $B(y, r/2)$ yuvarları ayrıktır: ortak bir z noktası olsaydı, üçgen eşitsizliğinden (M3) $r = d(x, y) \leq d(x, z) + d(z, y) < r/2 + r/2 = r$ çelişkisi çıkardı. Demek ki x ve y açık komşuluklarla ayrılır. (1. sayılabilir) Her x için sayılabilir $\{B(x, 1/n) : n \in \mathbb{N}\}$ ailesi yerel bazdır: $x \in U$ açık ise $B(x, \varepsilon) \subseteq U$ olacak $\varepsilon > 0$ vardır; $1/n < \varepsilon$ seçince $B(x, 1/n) \subseteq U$. $\square$

Teorem 14.3 (Sınırlı Metrik Eşdeğerliği). $d'(x, y) = \min(d(x, y), 1)$ ile d aynı topolojiyi indükler.

İspat eskizi. İki metrik aynı topolojiyi indükler $\Leftrightarrow$ her d -açık yuvarın içine bir d' -açık yuvar sığar ve tersi. Yarıçapı $r \leq 1$ olan yuvarlarda $B_d(x, r) = B_{d'}(x, r)$ tam çakışır, çünkü d ile d' yalnız mesafesi ≥ 1 olan çiftlerde ayrışır. Topoloji “yeterince küçük yuvarlar” tarafından belirlendiğinden büyük yarıçaplı farklılık topolojiyi değiştirmez; $\tau_d = \tau_{d'}$. $\square$

Teorem 14.4. $\mathbb{R}^n$ üzerindeki *max*, *sum* ve Öklid metrikleri topolojik olarak eşdeğerdir.

İspat eskizi. Üç metrik karşılıklı sabit çarpanlarla sınırlanır: $d_\infty \leq d_2 \leq d_1 \leq n d_\infty$ ($\max \leq$ Öklid $\leq$ taksi $\leq n \cdot \max$). Böyle bir “sandviç” her metriğin yuvarının içine ötekinin yuvarını sığdırır (ör. $B_\infty(x, r/n) \subseteq B_1(x, r) \subseteq B_\infty(x, r)$). Karşılıklı içerme $\Rightarrow$ aynı açık kümeler $\Rightarrow$ aynı topoloji. Bölüm 1.0’daki birim-yuvar figürü bu üç biçimi yan yana gösterir. $\square$

14.3 Algoritmalar

14.3.1 `validate_metric` — $O(|X|^3)$

Algorithm 18 `ValidateMetric( $X, d$ )`

```

1: for each  $x \in X$  do
2:   if  $d(x, x) \neq 0$  then return False
3:   end if
4:   for each  $y \neq x$  do
5:     if  $d(x, y) = 0$  then return False
6:     end if
7:   end for
8: end for
9: for each  $x, y \in X$  do
10:  if  $d(x, y) \neq d(y, x)$  then return False
11:  end if
12: end for
13: for each  $x, y, z \in X$  do
14:  if  $d(x, z) > d(x, y) + d(y, z)$  then return False
15:  end if
16: end for
17: return True

```

Karmaşıklık: $O(|X|^3)$ — üçgen eşitsizliği dominates.

14.3.2 `open_ball` — $O(|X|)$

Algorithm 19 `OpenBall( $x, r, X, d$ )`

```

1: return  $\{y \in X : d(x, y) < r\}$ 

```

14.4 `pytop` API

```

1 from pytop.metric_spaces import (
2     FiniteMetricSpace,
3     SymbolicMetricSpace,
4     validate_metric,
5 )
6 from pytop import (
7     open_ball,

```

```

8     closed_ball,
9     distance_to_subset,
10    diameter_of_subset,
11    is_bounded_subset,
12    capped_metric,
13 )

```

`FiniteMetricSpace(carrier=tuple, distance=dict)` — mesafe sözlüğü (a,b): float.

`open_ball(fms, point, radius)` → set döner (Result değil).

`capped_metric(distance_fn, cap=1.0)` → Callable döner.

Metrik → TDA köprüsü: Bir `FiniteMetricSpace` (carrier + distance) doğrudan `pytop.persistent_homology(space, max_dimension=, max_scale=)` fonksiyonuna beslenebilir; bu, ham nokta bulutunu Vietoris-Rips filtrasyonuna çevirip kalıcı homolojiyi hesaplar. Dönen `PersistencePair` alanları: `dimension`, `birth`, `death`, `persistence`, `is_essential` (Örnek 5.8).

14.5 Örnekler

Örnek 5.1 — Ayrik Metrik Uzay

```

1 points = [1, 2, 3, 4]
2 dist_discrete = {(a, b): (0 if a == b else 1)
3                   for a in points for b in points}
4 fms = FiniteMetricSpace(carrier=tuple(points), distance=dist_discrete
5 )
6 print("carrier:", fms.carrier)
7 print("d(1,2):", fms.distance[(1,2)])

```

carrier: (1, 2, 3, 4)

d(1,2): 1

Örnek 5.3 — open_ball

```

1 print("B(1, 0.5) =", open_ball(fms, 1, 0.5))
2 print("B(1, 1.5) =", open_ball(fms, 1, 1.5))

```

B(1, 0.5) = {1}

B(1, 1.5) = {1, 2, 3, 4}

Örnek 5.6 — capped_metric

```

1 cap_fn = capped_metric(fms_line.distance, cap=1.0)
2 print("d(0,3) capped:", cap_fn(0, 3))
3 print("d(0,1) capped:", cap_fn(0, 1))

```

d(0,3) capped: 1.0

d(0,1) capped: 1.0

Örnek 5.7 — Üç Metrik, Üç Yuvar, Aynı Geçerlilik

Düzlemdeki aynı beş nokta üzerinde Öklid, taksi (ℓ^1) ve maksimum (ℓ^∞) metriklerini kurarız: üçü de geçerli metriktir ama aynı çift için farklı mesafe verir (Bölüm 1.0 birim-yuvar figürünün sayısal karşılığı).

```

1 import math
2 from pytop.metric_spaces import FiniteMetricSpace, validate_metric
3
4 grid = [(0, 0), (1, 0), (0, 1), (1, 1), (2, 1)]
5 carrier = tuple(range(len(grid)))
6
7 def euclid(i, j):     return math.dist(grid[i], grid[j])
8 def taxicab(i, j):    return abs(grid[i][0]-grid[j][0]) + abs(grid[i]
9                        ][1]-grid[j][1])
10 def chebyshev(i, j): return max(abs(grid[i][0]-grid[j][0]), abs(grid[
11                                i][1]-grid[j][1]))
12
13 for name, d in [("Oklid", euclid), ("Taksi", taxicab), ("Maksimum",
14                                chebyshev)]:
15     dist = {(i, j): d(i, j) for i in carrier for j in carrier}
16     fms = FiniteMetricSpace(carrier=carrier, distance=dist)
17     print(name, "d(0,4)=", round(dist[(0,4)], 4), "metrik?",
18           validate_metric(fms).status)

```

Oklid d(0,4) = 2.2361 metrik? true

Taksi d(0,4) = 3.0 metrik? true

Maksimum d(0,4) = 2.0 metrik? true

Mesafeler farklı (Öklid $\sqrt{5}$, taksi 3, maksimum 2), ama üçü de aynı topolojiyi indükler (Teorem 2.3).

Örnek 5.8 — Metrik $\rightarrow$ Kalıcı Homoloji (TDA Köprüsü)

Metrik uzayın hesaplama gücüne köprü: bir nokta bulutundan `FiniteMetricSpace` kurup doğrudan `persistent_homology`'ye veririz. Çember üzerinde örneklenmiş 8 nokta — “delik” (1-boyutlu döngü) topolojik imza olarak ortaya çıkmalıdır.

```

1 import math, pytop
2 from pytop.metric_spaces import FiniteMetricSpace
3
4 n = 8
5 pts = [(math.cos(2*math.pi*k/n), math.sin(2*math.pi*k/n)) for k in
6         range(n)]
7 carrier = tuple(range(n))
8 dist = {(i, j): math.dist(pts[i], pts[j]) for i in carrier for j in
9         carrier}
10 circle = FiniteMetricSpace(carrier=carrier, distance=dist)
11
12 pairs = pytop.persistent_homology(circle, max_dimension=2, max_scale
13                                   =2.5)

```

```

11 components = [p for p in pairs if p.dimension == 0 and p.is_essential
12 ]
13 loops = [p for p in pairs if p.dimension == 1]
14 loop = loops[0]
15 print("bilesen (H0):", len(components), " dongu (H1):", len(loops))
16 print("dongu: dogum=", round(loop.birth,4), "olum=", round(loop.death
,4))

```

```

bilesen (H0): 1  dongu (H1): 1
dongu: dogum = 0.7654 olum = 1.8478

```

Tam beklenen sonuç: çember bağlantılı olduğundan tek kalıcı bileşen (H_0), bir deliği olduğundan tek kalıcı döngü (H_1). Döngü, komşu noktalar bağlandığında doğar (≈ 0.765) ve yarıçap deliği dolduracak kadar büyüyünce ölür (≈ 1.848). Ham koordinatlardan (`math.dist`) pytop topolojik bir özelliği — deliğin varlığını — sayısal olarak çıkardı.

14.6 Alıştırmalar

Kodlama

- K1. $\{A, B, C, D\}$ üzerinde bir metrik tanımlayın ve `validate_metric` ile doğrulayın. Üçgen eşitsizliğini ihlal eden bir örnek de deneyin.
- K2. Öklid metriklili $\{0, \dots, 9\}$ üzerinde $B(5, 3.0)$ ve $\bar{B}(5, 3.0)$ hesaplayın.
- K3. `normalized_metric` kullanarak bir metriği $[0, 1]$ 'e normalleştirin.
- K4. §1.0'daki karşı-örneği koda dökün: $\{A, B, C, D\}$ üzerinde $d(A, C) = 5$, $d(A, B) = d(B, C) = 1$ olan bir mesafe sözlüğü kurun (geri kalanını ayırık metrikle doldurun) ve `validate_metric` ile üçgen eşitsizliğinin ihlal edildiğini gösterin; `justification` alanını yazdırın.
- K5. (*TDA köprüsü*) Birim kareyi örnekleyen **dolu** bir 3×3 ızgara (9 nokta) üzerinde Öklid `FiniteMetricSpace` kurun ve `persistent_homology(space, max_dimension =2, max_scale =2.5)` çalıştırın. H_1 döngülerinin `persistence` değerlerine bakın: Örnek 5.8'deki çemberin uzun ömürlü (≈ 1.08) döngüsüne benzer **kalıcı** bir döngü var mı? Dolu karenin neden gerçek bir deliği yoktur?

Teori

- T1. Her metrik uzayın Hausdorff olduğunu ispatlayın.
- T2. $d' = \min(d, 1)$ ve d aynı topolojiyi indüklediğini gösterin.
- T3. Üçgen eşitsizliğinin (M3) Teorem 2.1'deki Hausdorff ispatında tam olarak nerede kullanıldığını açıklayın. (İpucu: $B(x, r/2)$ ve $B(y, r/2)$ 'nin ayırık olduğunu gösteren adım hangisidir?)

Bölüm 15

Metrik Tamlık ve Kompaktlık

Metrik tamlık (completeness), bir metrik uzayda Cauchy dizilerinin yakınsayıp yakınsamadığını araştıran temel bir kavramdır.

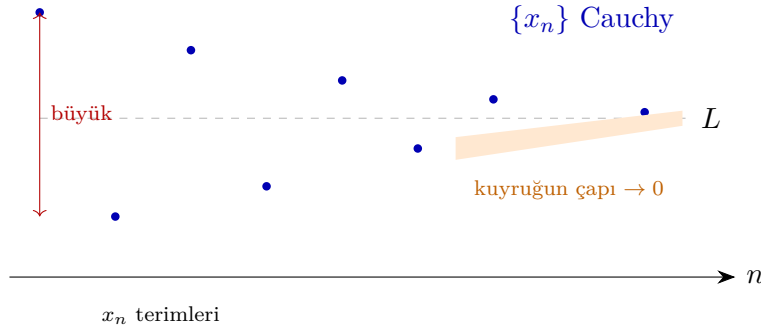
15.1 Konu

15.1.1 Cauchy Dizisi ve Tamlık

Tanım 15.1 (Cauchy Dizisi). (M, d) metrik uzayında $\{x_n\}$ dizisi **Cauchy** ise:

$$\forall \varepsilon > 0, \exists N \in \mathbb{N} : m, n \geq N \Rightarrow d(x_m, x_n) < \varepsilon.$$

Tanım 15.2 (Tam Metrik Uzay). (M, d) **tam** ise: Her Cauchy dizisi M 'de bir limite yakınsır.



Şekil 15.1: Cauchy dizisinin terimleri birbirine yaklaşır: kuyruğun çapı sıfıra gider.

Sezgi

Bir Cauchy dizisini, ilerledikçe birbirine sokulan bir terim kalabalığı gibi düşünün. İlk terimler birbirinden uzaktayken, yeterince ileri gidildiğinde tüm terimler bir noktada toplanır: “kuyruğun çapı” ($\sup_{m,n \geq N} d(x_m, x_n)$) sıfıra iner. **Tam** uzayda bu kalabalığın sıkıştığı yerde gerçekten bir nokta (limit) durur; tam olmayan uzayda ise o yerde bir *boşluk* olabilir.

15.1.2 Tamamen Sınırlılık

$A \subseteq M$ **tamamen sınırlı** ise: her $\varepsilon > 0$ için A 'nın sonlu bir ε -net kümesi vardır.

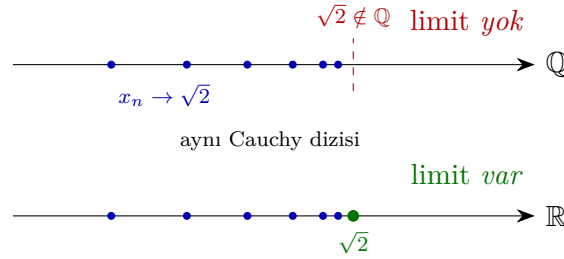
$$S = \{s_1, \dots, s_n\} \subseteq M, \quad \forall x \in A \exists i : d(x, s_i) < \varepsilon.$$

15.1.3 Metrik Kompaktlık Karakterizasyonu

(M, d) tam $\wedge$ tamamen sınırlı $\iff (M, d)$ kompakt.

Uzay	Tam?	Tam. sınırlı?	Kompakt?
$\mathbb{R}$	✓	×	×
$[0, 1]$	✓	✓	✓
$(0, 1)$	×	✓	×
$\mathbb{Q}$	×	×	×
Sonlu metrik	✓	✓	✓

Tablo 15.1: Tamlık ve kompaktlık örnekleri



Şekil 15.2: $\mathbb{Q}$ 'da $\sqrt{2}$ 'ye yakınsayan Cauchy dizisinin limiti yoktur; aynı dizi $\mathbb{R}$ 'de yakınsar.

Karşı-örnek

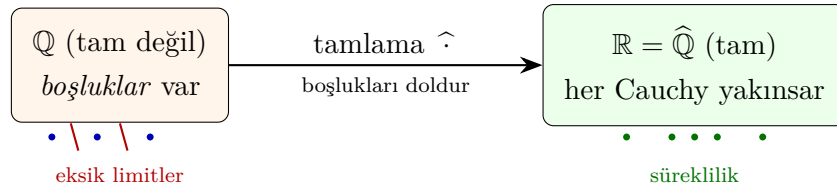
Rasyonel sayılar $\mathbb{Q}$, Öklid metriğiyle **tam değildir**. $x_1 = 1$, $x_2 = 1.4$, $x_3 = 1.41$, $x_4 = 1.414$, ... dizisi $\sqrt{2}$ 'nin ondalık açılımıdır: her terim rasyoneldir ve dizi Cauchy'dir ($d(x_m, x_n) \rightarrow 0$). Ama limit $\sqrt{2} \notin \mathbb{Q}$ olduğundan dizi $\mathbb{Q}$ içinde *hiçbir* noktaya yakınsamaz. Aynı dizi $\mathbb{R}$ 'de $\sqrt{2}$ 'ye yakınsar — fark uzayda “boşluk” olup olmamasıdır.

Tamlama (completion). Her metrik uzay (M, d) bir **tam** uzayın $(\widehat{M}, \widehat{d})$ yoğun alt-uzayı olarak gömülebilir; $\widehat{M}$ 'ye M 'nin *tamlaması* denir. Sezgisel olarak tamlama, eksik limitleri (“boşlukları”) ekleyerek uzayı kapatır: $\mathbb{R} = \widehat{\mathbb{Q}}$ bu inşanın klasik örneğidir.

15.2 Teoremler

Teorem 15.3 (Heine-Borel Metrik Genelleştirmesi). *Metrik uzayda kompaktlık $\iff$ tamlık $\wedge$ tamamen sınırlılık.*

Teorem 15.4 (Banach Sabit-Nokta Teoremi). *(M, d) tam metrik uzay; $T : M \rightarrow M$ büzülme ($K < 1$) $\Rightarrow T$ 'nin eşsiz sabit noktası x^* vardır: $T(x^*) = x^*$.*



Şekil 15.3: Tamlama: $\mathbb{Q}$ 'nın boşlukları doldurularak tam uzay $\mathbb{R}$ elde edilir.

İspat eskizi. $K < 1$ büzülme sabiti olsun. Herhangi bir x_0 alıp $x_{n+1} = T(x_n)$ iterasyonunu kurun. Büzülmeden $d(x_{n+1}, x_n) \leq K d(x_n, x_{n-1}) \leq K^n d(x_1, x_0)$. Üçgen eşitsizliği ve geometrik seri ile $d(x_{n+m}, x_n) \leq \frac{K^n}{1-K} d(x_1, x_0)$; $K < 1$ olduğundan bu ifade $n \rightarrow \infty$ iken sıfıra gider, yani $\{x_n\}$ Cauchy'dir. **Tamlık** kullanılarak dizi bir x^* 'ye yakınsar. T sürekli (Lipschitz) olduğundan $T(x^*) = T(\lim x_n) = \lim x_{n+1} = x^*$: x^* sabit noktadır. *Eşsizlik:* $T(p) = p$, $T(q) = q$ olsaydı $d(p, q) = d(Tp, Tq) \leq K d(p, q)$, $K < 1$ ile bu ancak $d(p, q) = 0$, yani $p = q$ ise mümkündür. $\square$

Teorem 15.5 (Tam Uzayın Kapalı Alt-Uzayı Tamdır). (M, d) tam ve $A \subseteq M$ kapalı ise (A, d) de tamdır.

İspat eskizi. $\{a_n\} \subseteq A$ bir Cauchy dizisi olsun. $\{a_n\}$ aynı zamanda M içinde Cauchy'dir ve M tam olduğundan bir $a \in M$ limitine yakınsar. A **kapalı** olduğundan kendi limit noktalarını içerir; $a_n \rightarrow a$ ve $a_n \in A$ olması $a \in \bar{A} = A$ demektir. Böylece her Cauchy dizisi A içinde bir limite yakınsar: A tamdır. $\square$

Teorem 15.6 (Baire Kategorisi Teoremi). Tam metrik uzay 1. kategoriden değildir.

15.3 Algoritmalar

15.3.1 Sonlu Metrikte is_complete ve is_totally_bounded

Sonlu metrik uzayda her Cauchy dizisi eninde sonunda sabit: trivially tam. Taşıyıcı kendisi ε -net: trivially tamamen sınırlı. Her ikisi: $O(1)$.

15.3.2 metric_compactness_check

Algorithm 20 MetricCompactnessCheck(M, d)

- 1: complete $\leftarrow$ is_complete(M, d)
 - 2: totally_bounded $\leftarrow$ is_totally_bounded(M, d)
 - 3: **return** complete $\wedge$ totally_bounded
-

15.4 pytop API

```

1 from pytop.metric_completeness import (
2     is_complete,
3     is_totally_bounded,
4     metric_compactness_check,

```

```

5     analyze_metric_completeness,
6 )

```

15.5 Örnekler

Örnek 5.1 — Sonlu Metrik: Tam ve Kompakt

```

1 points = ['A', 'B', 'C', 'D']
2 dist = {(a, b): (0 if a == b else 1)
3         for a in points for b in points}
4 fms = FiniteMetricSpace(carrier=tuple(points), distance=dist)
5 print("is_complete:", is_complete(fms).status)
6 print("is_totally_bounded:", is_totally_bounded(fms).status)
7 mc = metric_compactness_check(fms)
8 print("metric_compactness:", mc.status, mc.value)

```

```

is_complete: true
is_totally_bounded: true
metric_compactness: true True

```

Örnek 5.4 — analyze_metric_completeness

```

1 r = analyze_metric_completeness(fms)
2 print("status:", r.status)
3 print("value:", r.value)

```

```

status: true
value: {'is_complete': True, 'is_totally_bounded': True, 'metric_compact': True}

```

Örnek 5.6 — Rasyoneller $\mathbb{Q}$: Tam Olmayan Uzay (Karşı-Örnek)

`rationals_metric()` sembolik bir uzaydır; `is_complete` ondalık taramayla karara bağlanamadığı için `unknown` döner — fakat uzayın **etiketleri** (tags) $\mathbb{Q}$ 'nun tam olmadığını kayıt altına alır ('not_complete').

```

1 from pytop import rationals_metric
2 from pytop.metric_completeness import is_complete
3
4 qm = rationals_metric()
5 res = is_complete(qm)
6 print("is_complete:", res.status)
7 print("'not_complete' tag:", 'not_complete' in qm.tags)
8 print("justification:", res.justification[0])

```

```

is_complete: unknown
'not_complete' tag: True
justification: Completeness has exact support only for explicit finite metric spaces.

```

Örnek 5.7 — Banach ve Sabit-Nokta Profilleri

`get_fixed_point_profiles()` sabit-nokta teorisinin küratörlü profillerini verir; `fixed_point_stability` bunları kararlılığa göre gruplar. Banach büzülme teoremi **çekici (attracting)** sabit nokta profiline karşılık gelir.

```

1 from pytop import get_fixed_point_profiles,
   fixed_point_stability_summary
2
3 profiles = get_fixed_point_profiles()
4 print("profil sayısı:", len(profiles))
5 for p in profiles:
6     print(f"  {p.key:24s}: {p.stability}")
7
8 summary = fixed_point_stability_summary()
9 print("stable:", summary['stable'])

```

```

profil sayısı: 5
  attracting_fixed_point : stable
  repelling_fixed_point  : unstable
  neutral_fixed_point    : neutral
  brouwer_fixed_point    : not_applicable
  periodic_point_n      : not_applicable
stable: ['attracting_fixed_point']

```

15.6 Alıştırmalar

Kodlama

- K1. Rasyonel sayılar $\mathbb{Q}$ (sembolik) için `is_complete` sonucunu kontrol edin.
- K2. Kendi 4-noktalı metrik uzayınızı oluşturun ve `metric_compactness_check` uygulayın.
- K3. `analyze_metric_completeness` çalıştırın ve `value` sözlüğünü inceleyin.
- K4. `rational_metric()` oluşturun; `is_complete(...).status` ile `'not_complete'` etiketinin uzayın `tags` kümesinde olup olmadığını yazdırın. Statü neden `unknown`, etiket neden `True`?
- K5. `get_fixed_point_profiles()` listesini gezin ve `stability == 'stable'` olan profilin `key` alanını yazdırın. Bu profil hangi teoreme (Banach mı?) karşılık gelir?

Teori

- T1. Banach sabit-nokta teoremini sözlü olarak açıklayın ve bir uygulama verin.
- T2. Kapalı $[0, 1]$ 'in tam, açık $(0, 1)$ 'in tam olmadığını gösterin. (Hint: $1/n$ dizisi Cauchy ama $0 \notin (0, 1)$.)

- T3. “Tam uzayın kapalı alt-uzayı tamdır” teoremini ispatlayın. Ayrıca tam bir uzayda **açık** bir alt-uzayın tam olması gerekmediğini bir örnekle gösterin. (Hint: $\mathbb{R}$ tam, ama $(0, 1) \subseteq \mathbb{R}$ açık ve tam değil.)
- T4. $\mathbb{Q}$ ’nun tam olmadığını $\sqrt{2}$ ’nin ondalık açılımı üzerinden ispatlayın: dizinin Cauchy olduğunu, fakat $\mathbb{Q}$ içinde limitinin bulunmadığını gösterin. Tamlama kavramıyla bu “boşluğun” nasıl kapatıldığını açıklayın.

Bölüm 16

Metrik Fonksiyonlar ve Sözleşmeler

Bu bölümde metrik uzaylar arasındaki fonksiyon türleri (izometri, Lipschitz, sürekli) ve bunların sınıflandırılması incelenmektedir.

16.1 Konu

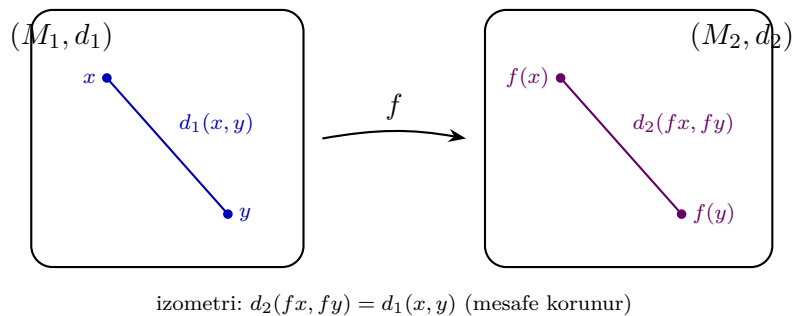
16.1.1 Metrik Fonksiyon Sınıfları

$f : (M_1, d_1) \rightarrow (M_2, d_2)$ fonksiyonu:

Sınıf	Koşul
Genişlemez (non-expansive)	$d_2(f(x), f(y)) \leq d_1(x, y)$
Lipschitz	$\exists K > 0 : d_2(f(x), f(y)) \leq K \cdot d_1(x, y)$
Üniform sürekli	$\forall \varepsilon > 0 \exists \delta > 0 : d_1(x, y) < \delta \Rightarrow d_2(f(x), f(y)) < \varepsilon$
Sürekli	Her noktada ε - δ sürekliliği
İzometri	$d_2(f(x), f(y)) = d_1(x, y)$
Benzerlik	$\exists c > 0 : d_2(f(x), f(y)) = c \cdot d_1(x, y)$
Homeomorfizma	Bijektif sürekli; ters de sürekli

Tablo 16.1: Metrik fonksiyon sınıfları

Sıralama: İzometri $\Rightarrow$ Genişlemez $\Rightarrow$ Lipschitz $\Rightarrow$ Üniform sürekli $\Rightarrow$ Sürekli.



Şekil 16.1: İzometri: f , x ile y arasındaki mesafeyi korur — $d_2(f(x), f(y)) = d_1(x, y)$.

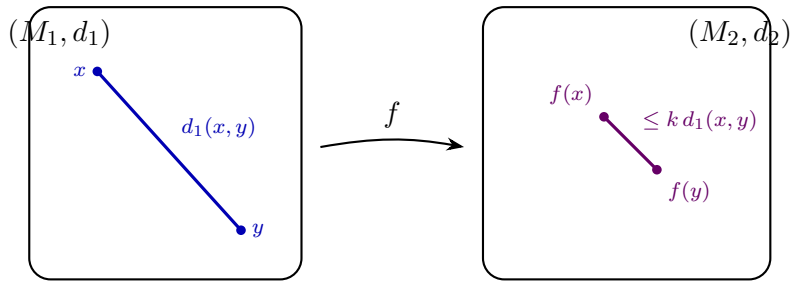
Sezgi

Bir izometriyi, bir kâğıt parçasını *buruşturmadan ve germeden* başka bir yere taşıyan bir hareket gibi düşünün: döndürebilir, kaydırabilir, yansıtabilirsiniz, ama iki nokta arasındaki mesafe asla değişmez. Düzlemde öteleme, dönme ve yansıma izometridir. Benzerlik ise kâğıdı oranını koruyarak büyüten/küçülten bir fotokopi makinesidir ($c = 2 \Rightarrow$ her mesafe iki katına çıkar); $c = 1$ alındığında benzerlik tekrar izometriye iner.

16.1.2 Lipschitz Sabiti

$$K = \sup_{x \neq y} \frac{d_2(f(x), f(y))}{d_1(x, y)}.$$

İzometri: $K = 1$ ve eşitlik; benzerlik: $K = c$ sabit.



büzülme: $d_2(fx, fy) \leq k d_1(x, y)$, $k < 1$ (mesafe daralır)

Şekil 16.2: Büzülme: $k < 1$ olduğunda f noktaları birbirine yaklaştırır — $d_2(f(x), f(y)) \leq k d_1(x, y)$.

Sezgi

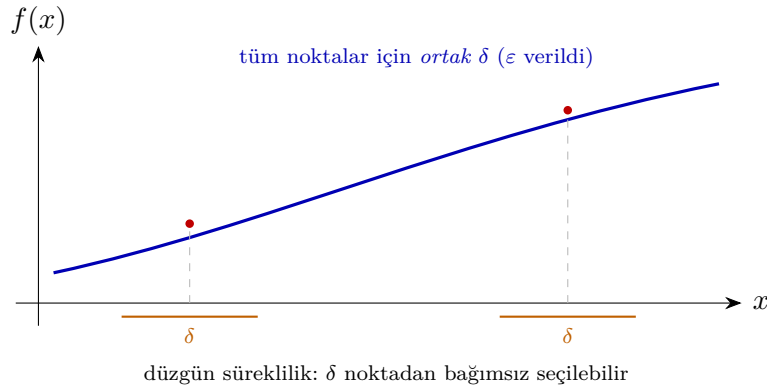
Lipschitz sabiti K , fonksiyonun “en dik eğimi” gibidir: hiçbir nokta çiftinde mesafeyi K katından fazla büyütemez. $K < 1$ olduğunda her adımda mesafe kesin olarak küçülür — buna **büzülme (contraction)** denir. Büzülmeyi tekrar tekrar uyguladığınızda tüm noktalar tek bir noktaya (sabit noktaya) doğru hunilenir; Banach sabit-nokta teoreminin kalbi budur.

Karşı-örnek

Her sürekli fonksiyon düzgün (üniform) sürekli *değildir*. $\mathbb{R}^+$ üzerinde $f(x) = 1/x$ sürekli, ama düzgün sürekli değildir: x sifıra yaklaştıkça eğim sınırsız diklenir, bu yüzden verilen bir ε için *tek* bir δ bütün noktalarda iş göremez ($x \approx 0.001$ civarında çok küçük bir δ gerekirken $x \approx 10$ civarında çok daha büyüğü yeterlidir). Aynı şekilde $\mathbb{R}$ üzerinde $f(x) = x^2$ sürekli, ama düzgün sürekli değildir. Düzgün süreklilik δ 'nın *noktadan bağımsız* seçilebilmesini ister; Lipschitz fonksiyonlar bunu $\delta = \varepsilon/K$ ile otomatik sağlar.

16.2 Teoremler

Teorem 16.1. *İzometri $\Rightarrow$ homeomorfizma.*



Şekil 16.3: Düzgün süreklilik: ε verildiğinde aynı δ tüm noktalarda iş görür (noktadan bağımsız).

İspat eskizi. Önce **injektiflik**: f bir izometri ve $f(x) = f(y)$ olsun. O zaman $d_1(x, y) = d_2(f(x), f(y)) = 0$, dolayısıyla metriğin ayırma aksiomundan $x = y$: izometri daima injektiftir. Bijektif kabul edersek f^{-1} vardır ve o da bir izometridir. İzometri her mesafeyi koruduğu için ε - δ sürekliliğini $\delta = \varepsilon$ ile sağlar: hem f hem f^{-1} süreklidir. Bijektif + sürekli + sürekli ters = homeomorfizma. $\square$

Teorem 16.2. Benzerlik, $c = 1 \Rightarrow$ İzometri.

Teorem 16.3. Lipschitz $\Rightarrow$ Üniform sürekli $\Rightarrow$ Sürekli.

İspat eskizi. f sabiti $K > 0$ olan Lipschitz olsun. Verilen $\varepsilon > 0$ için $\delta = \varepsilon/K$ seçin; $d_1(x, y) < \delta$ ise $d_2(f(x), f(y)) \leq K d_1(x, y) < K(\varepsilon/K) = \varepsilon$. Seçilen δ noktaya bağlı değildir, dolayısıyla f düzgün süreklidir; bu özel olarak sürekliliği de içerir. $\square$

Teorem 16.4. Kompakt metrik uzaydan metrik uzaya her sürekli fonksiyon üniform süreklidir.

16.3 Algoritmalar

16.3.1 classify__finite__metric__map — $O(|X|^2)$

Karmaşıklık: $O(|X|^2)$.

16.4 pytop API

```

1 from pytop.metric_spaces import FiniteMetricSpace
2 from pytop.metric_map_taxonomy import (
3     classify_finite_metric_map,
4     metric_map_profile,
5     MetricMapProfile,
6 )

```

`classify_finite_metric_map(fms1, fms2, f) → MetricMapProfile` döner (Result değil).

Algorithm 21 `ClassifyMap( $X, d_1, Y, d_2, f$ )`

```

1:  $K \leftarrow 0$ ; isometry  $\leftarrow \text{True}$ ; similarity_ratio  $\leftarrow \text{None}$ 
2: for each pair  $(x_1, x_2)$  with  $x_1 \neq x_2$  do
3:   ratio  $\leftarrow d_2(f(x_1), f(x_2))/d_1(x_1, x_2)$ 
4:    $K \leftarrow \max(K, \text{ratio})$ 
5:   if  $d_2(f(x_1), f(x_2)) \neq d_1(x_1, x_2)$  then
6:     isometry  $\leftarrow \text{False}$ 
7:   end if
8:   if similarity_ratio is None then
9:     similarity_ratio  $\leftarrow \text{ratio}$ 
10:  else if ratio  $\neq$  similarity_ratio then
11:    similarity_ratio  $\leftarrow \text{None}$ 
12:  end if
13: end for
14: return MetricMapProfile(lipschitz_constant =  $K$ , isometry = isometry, ...)

```

MetricMapProfile alanları: isometry, similarity, similarity_ratio, lipschitz_constant, non_expansive, bijective, lipschitz, continuous, uniformly_continuous, homeomorphism, name, certification.

16.5 Örnekler

Örnek 5.1 — Özdeşlik: İzometri

```

1 f_id = {1: 1, 2: 2, 3: 3, 4: 4}
2 prof_id = classify_finite_metric_map(fms, fms, f_id)
3 print("isometry:", prof_id.isometry)
4 print("lipschitz_constant:", prof_id.lipschitz_constant)

```

```

isometry: True
lipschitz_constant: 1.0

```

Örnek 5.2 — $2\times$ Ölçekleme: Benzerlik

```

1 f_scale = {1: 2, 2: 4, 3: 6, 4: 8}
2 prof_scale = classify_finite_metric_map(fms_line1, fms_line2, f_scale
3 )
4 print("similarity:", prof_scale.similarity)
5 print("similarity_ratio:", prof_scale.similarity_ratio)
6 print("lipschitz_constant:", prof_scale.lipschitz_constant)

```

```

similarity: True
similarity_ratio: 2.0
lipschitz_constant: 2.0

```

Örnek 5.3 — Sabit Fonksiyon: $K = 0$

```

1 f_const = {1: 1, 2: 1, 3: 1, 4: 1}
2 prof_const = classify_finite_metric_map(fms, fms, f_const)
3 print("lipschitz_constant:", prof_const.lipschitz_constant)
4 print("bijective:", prof_const.bijective)

```

```

lipschitz_constant: 0.0
bijective: False

```

Örnek 5.6 — Büzülme (Contraction): $K = 1/2$

Öklid doğrusunda her mesafeyi yarıya indiren bir eşleme bir **büzülmedir** ($K < 1$): Banach teoreminin sözleşme koşulunu sağlar.

```

1 src = [0, 2, 4, 6]
2 img = [0, 1, 2, 3]
3 d_src = {(a, b): abs(a - b) for a in src for b in src}
4 d_img = {(a, b): abs(a - b) for a in img for b in img}
5 M_src = FiniteMetricSpace(carrier=tuple(src), distance=d_src)
6 M_img = FiniteMetricSpace(carrier=tuple(img), distance=d_img)
7 f_half = {0: 0, 2: 1, 4: 2, 6: 3}
8 prof_half = classify_finite_metric_map(M_src, M_img, f_half)
9 print("lipschitz_constant:", prof_half.lipschitz_constant)
10 print("similarity_ratio:", prof_half.similarity_ratio)
11 print("isometry:", prof_half.isometry)

```

```

lipschitz_constant: 0.5
similarity_ratio: 0.5
isometry: False

```

Örnek 5.7 — Ayırık Metrikte Döngü: İzometri ve Homeomorfizma

Ayrık metrikte her permütasyon mesafeyi korur: bir izometri ve (bijektif olduğundan) homeomorfizmadır.

```

1 dp = [1, 2, 3]
2 d_disc = {(a, b): (0 if a == b else 1) for a in dp for b in dp}
3 M_disc = FiniteMetricSpace(carrier=tuple(dp), distance=d_disc)
4 f_cycle = {1: 2, 2: 3, 3: 1}
5 prof_cycle = classify_finite_metric_map(M_disc, M_disc, f_cycle)
6 print("isometry:", prof_cycle.isometry)
7 print("homeomorphism:", prof_cycle.homeomorphism)

```

```

isometry: True
homeomorphism: True

```

16.6 Alıştırmalar

Kodlama

- K1. $\{1, 2, 3\}$ üzerinde $f(1) = 2, f(2) = 3, f(3) = 1$ (döngü) fonksiyonunu ayrık metrikte `classify_finite_metric_map` ile sınıflandırın.
- K2. Öklid metriklı $\{0, 1, 2, 3, 4\}$ üzerinde $f(x) = x+1$ kaydırma fonksiyonunun Lipschitz sabitini hesaplayın.
- K3. Sabit fonksiyon $f(x) = 1$ için $K = 0$ olduğunu doğrulayın.
- K4. Öklid doğrusunda $\{0, 2, 4, 6\} \rightarrow \{0, 1, 2, 3\}$ büzülmesini ($f(x) = x/2$) kurun; `lipschitz_constant`, `similarity_ratio`, `non_expansive` alanlarını yazdırın ve $K < 1$ olduğunu doğrulayın.
- K5. $\{1, 2, 3\}$ üzerinde özdeşlik olmayan bir döngüyü hem ayrık hem Öklid metrikte sınıflandırın; hangi metrikte izometri olduğunu `isometry` alanlarıyla karşılaştırın.

Teori

- T1. İzometri $\Rightarrow$ genişlemez $\Rightarrow$ Lipschitz ($K = 1$) zincirini ispatlayın.
- T2. Lipschitz $\Rightarrow$ üniform sürekli olduğunu $\delta = \varepsilon/K$ seçimi ile ispatlayın.
- T3. İzometrinin daima injektif olduğunu ispatlayın ($f(x) = f(y) \Rightarrow d_2(f(x), f(y)) = 0$); ardından düzlemde sabit bir fonksiyonun neden izometri olamadığını açıklayın.

Ek A

Alıştırma Çözümleri

Bu ek, kılavuz alıştırmalarının tam çözümlerini içerir. Kodlama çözümlerindeki tüm çıktılar gerçek çalıştırmadan alınmıştır; teori çözümleri tam argüman verir. Pilot kapsamında Bölüm 4 ve Bölüm 6 çözümleri yer alır; diğer bölümler yaygınlaştırma aşamasında eklenecektir.

A.1 Bölüm 4 — Topolojik Uzaylar

K1

```
1 from pytop import cofinite_topology, is_t1, is_t2
2 c = cofinite_topology('a', 'b', 'c')
3 print("|tau| =", len(c.topology))
4 print("Etiketler:", sorted(c.tags))
5 print("T1:", is_t1(c).status, "| T2:", is_t2(c).status)
```

Listing A.1: Çıktı

```
|tau| = 8
Etiketler: ['cofinite', 'compact', 'finite', 't1']
T1: true | T2: true
```

Üç elemanlı kümede tümleyeni sonlu olan *her* alt küme kosonlu koşulunu sağlar; $|\tau| = 2^3 = 8 = |\mathcal{P}(X)|$: topoloji ayrıktır. Sonlu kümede $T1 \Rightarrow$ tekiler kapalı $\Rightarrow$ her alt küme kapalı $\Rightarrow$ her alt küme açık; $T1$ olan sonlu uzay zorunlu olarak ayrık, dolayısıyla Hausdorff'tur. $T1$ olup $T2$ olmayan örnek için taşıyıcı sonsuz olmalıdır: kosonlu $\mathbb{N}$ 'de boş olmayan iki açık daima kesişir (Bölüm 6, Örnek 2). (alıştırmaya dön)

K2

```
1 from pytop import topology_from_subbasis
2 s = topology_from_subbasis({1, 2, 3, 4}, [{1, 2}, {3, 4}, {2, 3}])
3 print("|tau| =", len(s.topology))
4 print("Acik kumeler:", sorted(sorted(t) for t in s.topology))
```

Listing A.2: Çıktı

```
|tau| = 9
```

```
Acik kumeler: [[], [1, 2], [1, 2, 3], [1, 2, 3, 4], [2], [2, 3], [2, 3, 4], [3], [3, 4]]
```

Alt-baz çiftlerinin kesişimleri $\{2\}, \{3\}, \emptyset$ yeni baz elemanları üretir; birleşimler $\{1, 2, 3\}, \{2, 3, 4\}$ ve X 'i ekler. Toplam 9 açık küme. [\(alıştırmaya dön\)](#)

K3

```
1 from pytop import finite_chain_space
2 c5 = finite_chain_space(5)
3 print("|tau| =", len(c5.topology))
4 print("Acik kumeler:", sorted(sorted(t) for t in c5.topology))
```

Listing A.3: Çıktı

```
|tau| = 6
Acik kumeler: [[], [1], [1, 2], [1, 2, 3], [1, 2, 3, 4], [1, 2, 3, 4, 5]]
```

Açıklar 6 önektir. “En açık” nokta 1’dir: tek elemanlı $\{1\}$ açığında ve dolayısıyla her boş olmayan açıkta yer alır; 5 yalnız X ’tedir. [\(alıştırmaya dön\)](#)

T1

$X = \{1, 2, 3\}$ üzerinde tam olarak **29** farklı topoloji vardır (homeomorfizma sınıfı olarak 9). Doğrudan yöntem $\mathcal{P}(\mathcal{P}(X))$ ’in $2^8 = 256$ elemanlı aday ailesinin her birinde üç aksiyomu denetlemeyi gerektirir. Baz teoremi işi tersine çevirir: (B1)–(B2)’yi sağlayan küçük aileler seçilir, her biri *otomatik olarak geçerli* bir topoloji üretir; yalnız üretilen topolojilerin tekrarları ayıklanır. Aksiyom denetimi aday başına yapılmaz — üretim doğruluğu Baz Teoremi’nce garantilidir. [\(alıştırmaya dön\)](#)

T2

Ayrık en incedir: Herhangi bir τ topolojisi tanım gereği $\tau \subseteq \mathcal{P}(X) = \tau_{\text{disc}}$ sağlar; hiç bir aksiyoma gerek yoktur, “topoloji X ’in alt kümelerinden oluşur” tanımı yeter. *İndirgenmiş en kabadır*: (T1) aksiyomu her topolojinin $\emptyset$ ve X ’i içermesini zorunlu kılar; $\tau_{\text{ind}} = \{\emptyset, X\} \subseteq \tau$. Kullanılan tek aksiyom (T1)’dir. $\square$ [\(alıştırmaya dön\)](#)

A.2 Bölüm 6 — Ayrılma Aksiyomları

K1

```
1 from pytop import make_topology, separation_chain
2 k = make_topology({1, 2, 3}, {1}, {2}, {1, 2})
3 print("Acik kumeler:", sorted(sorted(t) for t in k.topology))
4 for prop, r in separation_chain(k).items():
5     print(f" {prop:20s}: {r.status}")
```

Listing A.4: Çıktı

```

Acik kumeler: [[], [1], [1, 2], [1, 2, 3], [2]]
t0              : true
t1              : false
hausdorff       : false
urysohn         : false
t3              : false
tychonoff       : unknown
t4              : false
completely_normal : false
perfectly_normal : false

```

T0 sağlanır: (1,2) çiftini {1}, (1,3) çiftini {1}, (2,3) çiftini {2} ayırır. T1 düşer: 3'ü içerip 1'i dışlayan açık yoktur (3 yalnız X 'te). `tychonoff`: `unknown` — sürekli fonksiyon ayırması açık-küme taramasıyla karara bağlanmaz; kütüphane dürüstçe “bilinmiyor” der. ([alıştırmaya dön](#))

K2

```

1 from pytop import finite_chain_space, separation_chain
2 for prop, r in separation_chain(finite_chain_space(3)).items():
3     print(f" {prop:20s}: {r.status}")

```

Listing A.5: Çıktı

```

t0              : true
t1              : false
hausdorff       : false
urysohn         : false
t3              : false
tychonoff       : unknown
t4              : false
completely_normal : false
perfectly_normal : false

```

En yüksek sağlanan aksiyom **T0**'dır. Önek yapısı her çifti “daha solda olan” lehine ayırır: (1,2) için {1}, (2,3) için {1,2}. T1 imkânsızdır: 2'yi içerip 1'i dışlayan önek yoktur. ([alıştırmaya dön](#))

K3

```

1 from pytop import two_point_indiscrete_space, is_t0, is_t1, is_t2
2 tp = two_point_indiscrete_space()
3 print("T0:", is_t0(tp).status, "| T1:", is_t1(tp).status, "| T2:",
      is_t2(tp).status)

```

Listing A.6: Çıktı

```
T0: false | T1: false | T2: false
```

Boş olmayan tek açık X 'tir ve her iki noktayı da içerir; hiçbir çift hiçbir yönden ayrılamaz. Zincirin tamamı en alt basamakta düşer. ([alıştırmaya dön](#))

T1

$T2 \Rightarrow T1$: $x \neq y$ verilsin. T2 ile ayrık $U \ni x, V \ni y$ açıkları vardır. $U \cap V = \emptyset$ olduğundan $y \notin U$ ve $x \notin V$: hem “ x ’i içerip y ’yi dışlayan” hem “ y ’yi içerip x ’i dışlayan” açık bulundu — T1’in iki yönlü koşulu sağlandı. $T1 \Rightarrow T0$: T0 yalnız *en az bir* yönlü ayrım ister; T1’in verdiği iki yönlü ayrımın herhangi biri yeter. $\square$ ([alıŖtırmaya dön](#))

T2

($\Rightarrow$) X sonlu ve T1 olsun. T1 geređi her x için ve her $y \neq x$ için $y \in U_x, x \notin U_y$ olan açık U_x vardır; $X \setminus \{x\} = \bigcup_{y \neq x} U_y$ açıktır, yani $\{x\}$ kapalıdır. Herhangi $A \subseteq X$, sonlu sayıda tekilin birleşimi olarak kapalıdır; o halde tümleyenini $X \setminus A$ açıktır. A keyfi olduğundan *her* alt küme açıktır: topoloji ayrıktır. ($\Leftarrow$) Ayrık topolojide her $\{x\}$ açıktır; $x \neq y$ için $\{x\}$ ve $\{y\}$ iki yönlü ayrımı doğrudan verir, T1 (hatta T2) sağlanır. $\square$ ([alıŖtırmaya dön](#))

Dizin

Açık top Bkz. Bölüm 14.

Banach sabit-nokta teoremi Bkz. Bölüm 15.

Bağlantılı uzay Bkz. Bölüm 8.

Cauchy dizisi Bkz. Bölüm 15.

Heine-Borel teoremi Bkz. Bölümler 7, 15.

Homeomorfizma Bkz. Bölümler 4, 16.

İzometri Bkz. Bölüm 16.

Kompaktlık Bkz. Bölüm 7.

Lipschitz sabit Bkz. Bölüm 16.

Metrik aksiyomları Bkz. Bölüm 14.

Sayılabilirlik aksiyomları Bkz. Bölüm 9.

Tamlık Bkz. Bölüm 15.

Topoloji aksiyomları Bkz. Bölüm 4.

Urysohn fonksiyon teoremi Bkz. Bölüm 6.